Operating System Engineering

Mihir Rao

Contents

1 Introduction and Examples **5**
- 1.1 Reading and writing: file descriptors . . . 6
- 1.2 What happens on a system call . . . 6
- 1.3 The shell, fork, and exec . . . 7
- 1.4 I/O redirection . . . 8
- 1.5 Pipes . . . 8
- 1.6 Directories . . . 9

2 C and gdb **10**
- 2.1 Memory in C . . . 10
- 2.2 Pointers . . . 10
- 2.3 Arrays . . . 11
- 2.4 Bitwise operators, casts, and headers . . . 11
- 2.5 Debugging with gdb . . . 12

3 OS Organization and System Calls **14**
- 3.1 Strawman: no operating system . . . 14
- 3.2 The UNIX interface . . . 14
- 3.3 Hardware support for isolation . . . 14
- 3.4 Entering the kernel . . . 15
- 3.5 The kernel as trusted computing base . . . 16
- 3.6 Monolithic kernels and microkernels . . . 16
- 3.7 Xv6 and its machine . . . 17
- 3.8 Booting xv6 and the first system call . . . 17

4 Page Tables **19**
- 4.1 Address spaces . . . 19
- 4.2 Paging hardware . . . 19
- 4.3 Three-level page tables . . . 20
- 4.4 Flags, page faults, and the TLB . . . 21
- 4.5 The xv6 kernel address space . . . 21
- 4.6 Process address spaces . . . 22
- 4.7 Code: building the kernel page table . . . 23
- 4.8 Code: user address spaces and sbrk . . . 24

5 Calling Conventions and Stack Frames RISC-V **25**
- 5.1 RISC-V and x86 . . . 25
- 5.2 The RISC-V abstract machine . . . 25
- 5.3 Calls and returns . . . 26
- 5.4 The calling convention . . . 27
- 5.5 Stack frames . . . 27
- 5.6 Mixing C and assembly . . . 28

5.7 Structs . . . 29
5.8 Debugging with gdb . . . 29

6 Isolation and System Call Entry/Exit **30**
6.1 What a trap must do . . . 30
6.2 From write() to ecall . . . 31
6.3 ecall and the trampoline . . . 31
6.4 Saving the user registers . . . 32
6.5 usertrap() and syscall() . . . 33
6.6 Returning: usertrapret(), userret, and sret . . . 33

7 Q&A Labs **35**
7.1 Reading a page table . . . 35
7.2 A kernel page table per process . . . 36
7.3 Mapping user memory into the kernel page table . . . 36
7.4 The system-call and utilities labs . . . 37

8 Page Faults **38**
8.1 Page faults as a mechanism . . . 38
8.2 Lazy allocation . . . 38
8.3 Zero-fill on demand . . . 39
8.4 Copy-on-write fork . . . 39
8.5 Demand paging . . . 40
8.6 Memory-mapped files . . . 40
8.7 TLB management . . . 40

9 Interrupts **42**
9.1 Where interrupts come from . . . 42
9.2 RISC-V interrupt registers . . . 43
9.3 Console output: printing `$` . . . 43
9.4 Interrupts and concurrency . . . 44
9.5 Console input: reading `1` . . . 45
9.6 Interrupt evolution and polling . . . 46

10 Multiprocessors and Locking **47**
10.1 What goes wrong without locks . . . 47
10.2 The lock abstraction . . . 48
10.3 Deadlock and modularity . . . 49
10.4 Locks and parallelism . . . 49
10.5 Case study: the UART driver . . . 49
10.6 Implementing a spin lock . . . 50
10.7 Memory ordering . . . 51
10.8 Spin locks and blocking locks . . . 51

11 Thread Switching **53**
11.1 Threads in xv6 . . . 53
11.2 A pre-emptive switch, step by step . . . 54
11.3 Questions . . . 56

13 Sleep and Wakeup **58**
13.1 Thread switching and `p->lock` . . . 58
13.2 Sequence coordination . . . 58
13.3 Lost wakeups . . . 59
13.4 How `sleep()` prevents lost wakeups . . . 61
13.5 Terminating a thread . . . 62

13.6 Killing a process . . . 62

14 File Systems **64**
14.1 The file system API . . . 64
14.2 Software layers and storage devices . . . 65
14.3 On-disk layout . . . 65
14.4 Inodes and directories . . . 66
14.5 Creating, writing, and deleting a file . . . 66
14.6 Concurrency and the buffer cache . . . 67
14.7 Pathname lookup . . . 68

15 Crash Recovery **69**
15.1 The problem . . . 69
15.2 Logging . . . 70
15.3 xv6's log . . . 70
15.4 The code path . . . 71
15.5 Recovery . . . 72
15.6 Challenges . . . 72
15.7 Assessment . . . 73

16 File System Performance and Fast Crash Recovery **74**
16.1 Review of xv6 logging . . . 74
16.2 ext3's log . . . 74
16.3 Asynchrony, batching, and concurrency . . . 75
16.4 System calls and commit . . . 76
16.5 Recovery . . . 77
16.6 Tricky details . . . 77
16.7 Ordered data mode . . . 78
16.8 More corner cases . . . 79

17 Virtual Memory for Applications **80**
17.1 User-level VM primitives . . . 80
17.2 Implementation . . . 81
17.3 Example: a huge memoization table . . . 81
17.4 Concurrent garbage collection . . . 81
17.5 Are the primitives good enough? . . . 83

18 OS Organization **84**
18.1 Traditional kernels . . . 84
18.2 Microkernels . . . 85
18.3 L4 . . . 86
18.4 Fast IPC . . . 87
18.5 Running Linux on L4 . . . 87
18.6 Evaluation . . . 89
18.7 The current situation . . . 89

19 Virtual Machines **90**
19.1 Trap-and-emulate . . . 90
19.2 Hardware-supported virtualization . . . 92
19.3 Dune . . . 93

20 Kernels and HLL **95**
20.1 Why consider a high-level language? . . . 95
20.2 Biscuit . . . 95
20.3 Heap exhaustion . . . 96

20.4 Experience with Go . . . 98
20.5 Performance . . . 99
20.6 Conclusions . . . 101

21 Networking **102**
21.1 Packet formats and protocols . . . 102
21.2 The kernel network stack . . . 103
21.3 Receive livelock . . . 104
21.4 The polling solution . . . 104

22 Meltdown **106**
22.1 The core of the attack . . . 106
22.2 Speculative execution . . . 106
22.3 Caches . . . 107
22.4 Meltdown in detail . . . 108
22.5 Defenses . . . 109

23 RCU **110**
23.1 Read-heavy shared data . . . 110
23.2 Read/write locks . . . 110
23.3 Reading without locks . . . 111
23.4 The Linux RCU interface . . . 112
23.5 Performance and examples . . . 114
23.6 Limitations . . . 114
23.7 An efficient `synchronize_rcu()` . . . 115
23.8 Write-heavy shared data . . . 115

Lecture 1: Introduction and Examples

An operating system serves several purposes at once. It

- abstracts the hardware, for convenience and portability;
- multiplexes the hardware among many applications;
- isolates applications from each other, to contain bugs;
- allows sharing among cooperating applications, and controls that sharing for security;
- stays out of the way of high performance, and supports a wide range of applications.

The usual organization is layered. User applications (`vi`, `gcc`, a database, ...) run on top of the kernel, which provides services on top of the hardware (CPU, RAM, disk, network). Typical kernel services are processes (running programs), memory allocation, file contents, file names and directories, and access control, plus many others such as users, inter-process communication, networking, time, and terminals. Both the interface the kernel offers and its internal structure matter.

Applications see the kernel through system calls. In C on a UNIX system (Linux, macOS, FreeBSD, ...) they look like

```
1 fd = open("out", 1);
2 write(fd, "hello\n", 6);
3 pid = fork();
```

These look like function calls, but they are not: each one transfers control into the kernel.

Operating system design and implementation are hard, and interesting, for several reasons. The environment is unforgiving: hardware is quirky and kernels are hard to debug. There are many design tensions: efficiency against abstract, portable, general-purpose interfaces; powerful against simple interfaces; flexibility against security. Features interact (what should `fd = open(); fork()` do with the descriptor?). Uses vary widely (laptops, phones, cloud servers, virtual machines, embedded devices), and the hardware keeps evolving (non-volatile RAM, multi-core processors, fast networks).

The examples below run on xv6, a small teaching kernel with the same structure as UNIX systems such as Linux but simple enough to read in full. UNIX is a good model because it is open source, well documented, cleanly designed, and widely used. Xv6 runs on the RISC-V architecture, here under the `qemu` machine emulator, and serves as an example of core kernel functions: virtual memory, multi-core, interrupts, and so on.

1.1 Reading and writing: file descriptors

The program `copy.c` copies its input to its output:

```
int main() {
  char buf[64];
  while(1){
    int n = read(0, buf, sizeof(buf));
    if(n <= 0)
      break;
    write(1, buf, n);
  }
  exit(0);
}
```

`read()` and `write()` are system calls. Their first argument is a FILE DESCRIPTOR (FD), which tells the kernel which open file to use; the file must have been opened earlier. An FD can refer to a file, a device, a socket, and so on, and a process can have many open FDs. By convention FD 0 is STANDARD INPUT and FD 1 is STANDARD OUTPUT. The second argument of `read()` points to memory into which to read, and the third is the maximum number of bytes to read; `read()` may read fewer, never more. The return value is the number of bytes actually read, or −1 on error. `copy.c` does not care about the format of the data: UNIX I/O is a stream of 8-bit bytes, and its interpretation (database records, C source, . . .) is up to the application.

File descriptors come from calls such as `open()`:

```
int fd = open("output.txt", O_WRONLY | O_CREATE);
write(fd, "ooo\n", 4);
```

`open()` creates the file if needed and returns an FD, or −1 on error. An FD is a small integer that indexes a per-process table maintained by the kernel, so different processes have different FD name spaces: FD 1 often means different things in different processes. (These examples ignore errors; real programs must not.)

1.2 What happens on a system call

A system call such as `open()` looks like a function call but is actually a special instruction. The hardware saves some user registers, raises the privilege level, and jumps to a known entry point in the kernel. Kernel C code then calls the system call's implementation: `open()` looks up the name in the file system, possibly waiting for the disk, and updates kernel data structures (a cache, the FD table). Finally the kernel restores the user registers, reduces the privilege level, and jumps back to the calling point in the program, which resumes.

1.3 The shell, fork, and exec

The SHELL is UNIX's command-line interface: it prints the `$` prompt and runs command-line utilities such as `ls`, `ls > out`, and `grep x < out`. UNIX supports other styles of interaction too (window systems, servers, routers), but time-sharing through the shell was its original focus, and the shell exercises many system calls.

The shell creates a new process for each command. The `fork()` system call creates a new PROCESS by copying the calling one: its instructions, data, registers, file descriptors, and current directory. The original is the PARENT and the copy the CHILD. The only difference is the return value: `fork()` returns the child's PID (process ID, an integer the kernel assigns uniquely to each process) in the parent and 0 in the child. So code after `fork()` executes in *both* processes, and a test `if(pid == 0)` lets each one tell which it is.

A program is stored in a file containing instructions and initial memory, produced by the compiler and linker. `exec(filename, argv)` replaces the calling process's memory with the executable file: it discards the old instruction and data memory, loads instructions and memory from the file, and *preserves* the file descriptors. The array `argv` holds the command-line arguments, which `exec` passes to the new program's `main(argc, argv)`; a successful `exec` does not return.

The shell combines the two in a common UNIX idiom: `fork()` a child, `exec()` the command in the child, and have the parent `wait()` for the child to finish:

```
int pid, status;
pid = fork();
if(pid == 0){
  char *argv[] = { "echo", "THIS", "IS", "ECHO", 0 };
  exec("echo", argv);
  printf("exec failed!\n");
  exit(1);
} else {
  printf("parent waiting\n");
  wait(&status);
  printf("the child exited with status %d\n", status);
}
```

The status passed to `exit(status)` is returned through `wait(&status)`; by convention 0 means success and 1 that the command encountered an error. The shell does fork, exec, and wait for every command and prints the next prompt after `wait()` returns; for a background command (`&`) it skips the wait. Note that `fork()` copies the parent's memory only for `exec()` to discard it. This looks wasteful, and the copy can be eliminated transparently with copy-on-write.

1.4 I/O redirection

For `echo hello > out` the shell forks, changes FD 1 in the child, and then execs `echo`:

```
if(fork() == 0){
  close(1);
  open("output.txt", O_WRONLY|O_CREATE);   // gets FD 1
  char *argv[] = { "echo", "this", "is", "redirected", "echo", 0 };
  exec("echo", argv);
  printf("exec failed!\n");
  exit(1);
} else {
  wait((int *) 0);
}
```

`open()` always chooses the lowest unused FD, which is 1 because of the preceding `close(1)`. Fork, FDs, and exec interact nicely here. Separating fork from exec gives the child a chance to change its FDs before running the command; FDs provide indirection, so commands just use FDs 0 and 1 without knowing where they lead; and exec preserves the FDs the shell set up. Thus only the shell has to know about I/O redirection, not each program.

It is worth asking "why" about each of these decisions. Why these I/O and process abstractions? Why provide a file system rather than letting programs use the disk their own way? Why FDs, instead of passing a file name to `write()`? Why are files streams of bytes rather than disk blocks or formatted records? Why not combine `fork()` and `exec()`? The UNIX design works well, but other designs exist.

1.5 Pipes

An FD can refer to a PIPE as well as a file. The `pipe(fds)` system call creates two FDs: data written to `fds[1]` can be read from `fds[0]`. The kernel keeps a buffer for each pipe; `write()` appends to the buffer and `read()` waits until there is data. Pipes combine well with `fork()`, since the child inherits both ends:

```
int fds[2];
char buf[100];
pipe(fds);
if(fork() == 0){
  write(fds[1], "this is pipe2\n", 14);
} else {
  int n = read(fds[0], buf, sizeof(buf));
  write(1, buf, n);
}
```

To run `ls | grep x` the shell creates a pipe, forks twice, connects `ls`'s FD 1 to the pipe's write end and `grep`'s FD 0 to its read end, and execs the two programs (Figure 1). Pipes are a separate abstraction, but they combine well with fork and FDs.

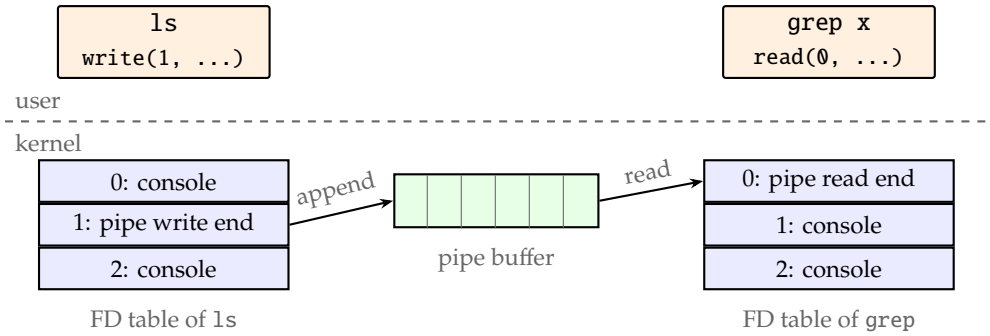


Figure 1: The shell's plumbing for `ls | grep x`. Each process has its own FD table in the kernel; `ls`'s FD 1 refers to the write end of a pipe and `grep`'s FD 0 to its read end. The kernel buffers the bytes in between.

1.6 Directories

A program such as `ls` lists a directory by opening it and reading it like a file, which yields its entries. In xv6 each entry is a fixed-size record, and `"."` is a pseudo-name for the process's current directory:

```c
struct dirent {
  ushort inum;
  char name[14];
};

int fd = open(".", 0);
struct dirent e;
while(read(fd, &e, sizeof(e)) == sizeof(e)){
  if(e.name[0] != '\0')      // skip unused entries
    printf("%s\n", e.name);
}
```

Together these examples show UNIX's I/O, file system, and process abstractions. The interfaces are simple (just integers and I/O buffers), and the abstractions combine well, for example to implement I/O redirection.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 1 (Fall 2020).

Lecture 2: C and gdb

2.1 Memory in C

Even in Python, names can alias memory. After `one = 1; two = one; two += 1` the two names print 1 and 2, but after `abc = ['a', 'b', 'c']; abcdef = abc; abcdef += ['d', 'e', 'f']` both names print `['a', 'b', 'c', 'd', 'e', 'f']`: the two names refer to the same list. C makes such questions explicit, because the programmer manages memory directly. A C program uses three kinds of memory.

- Static memory: global variables, accessible throughout the program; these are the variables defined at global scope or with the `static` keyword.
- Stack memory: local variables of functions, destroyed when the function returns.
- Heap memory: variables whose creation and destruction the program controls with `malloc()` and `free()`. Mistakes lead to memory leaks and use-after-free bugs.

2.2 Pointers

A pointer is a 64-bit integer whose value is an address in memory. Every variable has an address, so one can take a pointer to any variable, including a pointer to a pointer (and so on).

```c
int x = 5;
int *x_addr = &x;     // same as int* x_addr = &x; e.g. 0x7ffd2766a948
*x_addr = 6;          // * accesses the underlying value
int x_value = *x_addr;   // dereferencing: gives 6
int arr1[10];         // an array
int *arr2[20];        // array of pointers, so arr2 acts as an int**
void *myPtr;          // pointer to memory of unspecified type
```

Pointers make it easy to refer to memory that no longer exists:

```c
char* makeABC() {
  char y[3] = {'a', 'b', 'c'};
  return y;   // bug: y lives on makeABC's stack and dies on return
}
```

Pointers support arithmetic with `++`, `--`, `+`, and `-`, in units of the size of the type pointed to. If `char *c` is `0x100002`, then `c++` gives `0x100003` and `c += 4` then `0x100007`. If `int *i` is `0x100002`, then `i++` gives `0x100006` and `i += 4` then `0x100016`: an `int` is 4 bytes.

2.3 Arrays

A C array is a contiguous block of memory holding elements of one type, and the array's name acts as a pointer to its first element: for `char myString[40]` the name behaves as a `char*`, for `char* myArrayOfStrings[20]` as a `char**`, and for `int counting[5] = {1, 2, 3, 4, 5}` as an `int*`. The bracket operator is syntactic sugar for pointer arithmetic, so for `int arr[4] = {5, 6, 7, 8}` the statements `arr[2] = 50`, `*(arr + 2) = 50`, and even `2[arr] = 50` are equivalent (addition is commutative). C does no bounds checking: indexing out of bounds silently reads or modifies whatever memory is there, with unexpected results, so always use the size variables.

Example: *Pointer arithmetic on a stack array.* With `int a[4]`, `int *b`, and `int *c = a`, the following statements print the values in the comments (on a little-endian machine such as RISC-V or x86):

```
for (i = 0; i < 4; i++)
  a[i] = 100 + i;
c[0] = 200;                          // 200 101 102 103
c[1] = 300; *(c + 2) = 301; 3[c] = 302;   // 200 300 301 302
c = c + 1; *c = 400;                 // 200 400 301 302
c = (int *) ((char *) c + 1);
*c = 500;                            // 200 128144 256 302
b = (int *) a + 1;                   // b == a + 4 bytes
c = (int *) ((char *) a + 1);        // c == a + 1 byte
```

After the cast through `char *`, `c` points one byte into `a[1]`, so storing 500 = `0x1F4` overwrites the upper three bytes of `a[1]` and the low byte of `a[2]`: `a[1]` becomes `0x1F490` = 128144 and `a[2]` becomes `0x100` = 256. The last two lines show again that adding 1 to an `int*` advances 4 bytes but adding 1 to a `char*` advances one.

2.4 Bitwise operators, casts, and headers

C lets programs manipulate bits directly. In binary:

Operator	Example	Result
& (and)	`10001 & 10000`	`10000`
| (or)	`10001 | 10000`	`10001`
^ (xor)	`10001 ^ 10000`	`00001`
~ (complement)	`~10000`	`01111`
<< (left shift)	`1 << 4`	`10000` = 16
>> (right shift)	`10101 >> 3`	`10`

xv6 combines these to manage flags. With bit offsets `flag0 = 0`, `flag1 = 1`, `flag2 = 2`, the assignment `flags = (1 << flag0) | (1 << flag2)` sets flags 0 and 2, giving `101`, and `if(flags & (1 << flag1))` tests flag 1: `101 & 010` is 0, so the test is false.

A cast `(newType)variable` converts a value, for example `(char*)myVoidPtr` or `(uint64)(2 + 3)` and `(uint64)myVoidPtr`. The xv6 kernel uses casts heavily when it treats addresses as numbers, as in `kfree()`:

```
1 extern char end[];   // first address after kernel

2 void kfree(void *pa) {
3   struct run *r;
4   if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
5     panic("kfree");
6   ...
```

Header (`.h`) files contain declarations, the specifications; `.c` files contain definitions, the implementations. One should essentially never `#include` a `.c` file. Include guards handle nested or duplicate includes (xv6 rarely uses them), and the `extern` keyword declares a function or variable defined in another file, making it visible throughout the program.

2.5 Debugging with gdb

The GNU debugger can attach to xv6 running under QEMU. The provided `.gdbinit` file sets gdb up for this automatically; gdb must be run from the xv6 directory, and `~/.gdbinit` must be edited to allow other `.gdbinit` files to load. Running `make qemu-gdb` (or `make qemu-nox-gdb`) starts QEMU waiting for the debugger, and gdb is then started in a second shell; `make qemu` runs without gdb. `help <command>` explains any command, and commands may be abbreviated when unambiguous (`c` = `co` = `cont` = `continue`; also `s` = `step`, `si` = `stepi`).

The most useful commands:

- `step` and `next` run one line of C, stepping into (`step`) or over (`next`) function calls; `stepi` and `nexti` do the same for one assembly instruction. All four take a repeat count, and pressing Enter repeats the previous command.
- `continue` runs until a breakpoint or Control-C; `finish` runs until the current function returns; `advance <loc>` runs until the pc reaches the location.
- `break <loc>` sets a breakpoint at an address (`*0x7c00`), a function, or a line (`file.c:71`); `delete`, `disable`, and `enable` manage breakpoints. `break <loc> if <cond>` breaks only when the condition holds, and `cond <n> <cond>` adds a condition to an existing breakpoint.
- `watch <expr>` stops whenever the expression's value changes, `watch -l <addr>` whenever the contents of that memory address change, and `rwatch [-l] <expr>` whenever the value is read.
- `x/<fmt> <addr>` prints raw memory in a chosen format (`x/x` hexadecimal, `x/i` instructions); `print <expr>` evaluates a C expression and prints the result with its type, which is often more

useful: `p *((struct elfhdr *) 0x10000)` prints an ELF header field by field, while `x/13x 0x10000` prints thirteen anonymous words.

- `info registers` prints every register, `info frame` the current stack frame, `list <loc>` the source around a location, and `backtrace` the chain of calls.
- `layout <name>` switches the text user interface, which shows source, disassembly, and registers in a curses window.
- `set var <x> = <v>` changes a variable during execution, and `symbol-file <file>` loads another program's symbols, which is needed to see function and variable names in a program other than the kernel.

Example: *Reading a stack.* The boot loader of the 32-bit x86 version of xv6 sets the stack pointer to `0x7c00` (`movl $start, %esp`) and calls `bootmain()`, which eventually calls the kernel's entry point taken from the ELF header (`entry = (void(*)(void))(elf->entry); entry();`). The `call` pushes a return address; the prologue of `bootmain()` builds a stack frame by pushing `%ebp`, `%edi`, `%esi`, `%ebx` and reserving `0x1c` bytes of locals; and the call to `entry()` pushes another return address. When the kernel starts at `0x0010000c` the stack therefore reads (the stack grows down):

Address	Value	Meaning
`0x7c00`	`0x8ec031fa`	not the stack (first boot loader instructions)
`0x7bfc`	`0x00007c4d`	`bootmain()` return address
`0x7bf8`	`0x00000000`	old `ebp`
`0x7bf4`	`0x00000000`	old `edi`
`0x7bf0`	`0x00000000`	old `esi`
`0x7bec`	`0x00000000`	old `ebx`
`0x7be8–0x7bd0`	`0x00000000`	local variables (`sub $0x1c,%esp`)
`0x7bcc`	`0x00007db7`	`entry()` return address

Source. 6.S081 course staff, *6.S081: Operating System Engineering*, Lecture 2 (Fall 2020).

Lecture 3: OS Organization and System Calls

Applications such as `sh`, `echo`, and `find` sit above the system call interface (`open`, `close`, ...), which the operating system implements on top of the hardware. The goals of the OS are to run multiple applications, to isolate them, to multiplex the hardware among them, and to let them share.

3.1 Strawman: no operating system

Suppose there is no OS: each application interacts directly with the hardware (CPU cores and registers, DRAM chips, disk blocks, ...), perhaps through a library that abstracts some of it. This design is not conducive to multiplexing. Each application must periodically give up the hardware to the others (cooperative scheduling, which real-time operating systems do use), so isolation is weak: an application that forgets to give up the CPU, or loops forever, stops every other application, and another application cannot even kill it. Nor is the design conducive to memory isolation. All applications share physical memory, so one can overwrite another's memory (if `sh` lives at address `1000`, a buggy `echo` can store to `1000`) or the OS library itself.

3.2 The UNIX interface

The UNIX interface abstracts the hardware in a way that achieves the goals.

- *Processes instead of cores* (`fork`). The OS transparently allocates cores to processes, saving and restoring their registers, and it enforces that processes give the cores up, periodically reallocating them.
- *Memory instead of physical memory* (`exec`). Each process has its "own" memory; the OS decides where to place an application in physical memory, enforces isolation between the memories of different applications, and lets the program image be stored in the file system.
- *Files instead of disk blocks.* The OS provides convenient names and allows sharing of files between processes and users.
- *Pipes instead of shared physical memory.* The OS can stop the sender or the receiver.

3.3 Hardware support for isolation

The OS must be defensive: an application must not be able to crash the OS or break out of its isolation. This requires strong isolation between applications and the OS, typically provided by hardware in two forms.

Definition: Processors provide USER MODE and KERNEL MODE. In kernel mode the CPU can execute PRIVILEGED INSTRUCTIONS, such as setting the kernel/user bit, reprogramming the timer chip, setting up the page table, or disabling clock interrupts. In user mode it can execute only unprivileged instructions (`add`, `sub`, `jr`, branches, ...). The OS runs in kernel mode and applications in user mode. (RISC-V also has a machine mode, which can mostly be ignored.)

Processors also provide VIRTUAL MEMORY: hardware page tables translate virtual addresses to physical addresses and so define what physical memory an application can access. The OS gives each process its own page table, set up so that the process can access only its own memory. Every process (`ls`, `echo`, ...) then sees an address space running from 0 to some maximum, and the spaces are isolated from each other.

3.4 Entering the kernel

Applications must still be able to communicate with the kernel: to write to a storage device, which is shared and must therefore be protected inside the kernel, to exit, and so on. The solution is an instruction that changes mode in a controlled way. On RISC-V, `ecall` enters kernel mode at an entry point agreed on in advance, specified by the kernel. A user-level stub for each system call places the call's number n where the kernel expects it and executes `ecall`; in xv6 the number goes in register `a7`:

```
1 fork:
2     li a7, SYS_fork
3     ecall
4     ret
```

The path of a `printf()` is thus app → `printf()` → `write()` → system call → `sys_write()` → ⋯. User-level libraries such as `printf()` are the application's private business, while kernel internal functions cannot be called from user code at all: the only way in is through the entry point, where the kernel's `syscall()` dispatches on the number to the implementation, which lives in a kernel subsystem such as the file system or the process code (Figure 2). The kernel also needs a way to fetch the system call's arguments.

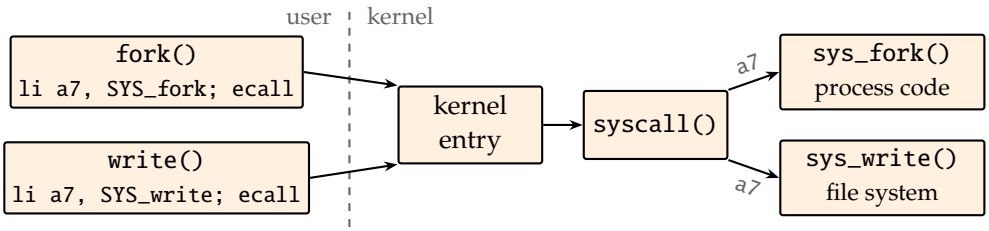


Figure 2: Entering the kernel. Each user-level stub executes `ecall` with its call number in `a7`; the kernel's `syscall()` dispatches on it.

3.5 The kernel as trusted computing base

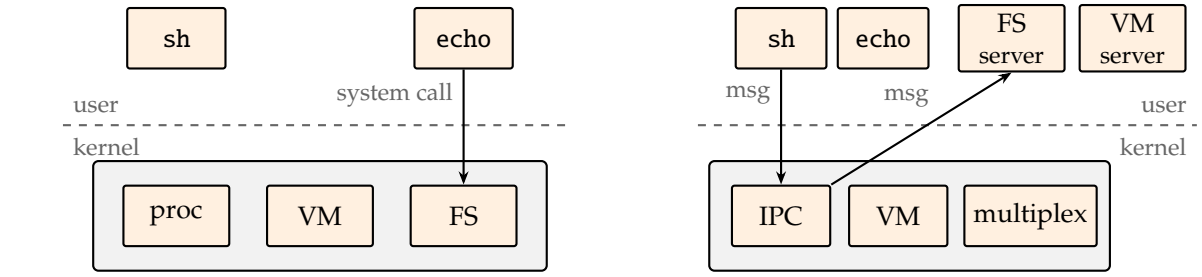
The kernel is the TRUSTED COMPUTING BASE (TCB). It must be correct, since a kernel bug can let user applications circumvent the user/kernel separation; this happens often in practice because kernels are complex (see the CVE lists). The kernel must treat user applications as suspect, even malicious: an application may try to trick the kernel into doing the wrong thing, so the kernel must check arguments carefully, set up the user/kernel boundary correctly, and so on. It is also in charge of separating applications, since one may try to read or write another's memory. All of this requires a security mindset: any bug in the kernel may be a security exploit.

Process isolation is possible without hardware user/kernel mode and virtual memory, using a strongly typed programming language (as in the Singularity OS); the compiler then becomes the TCB. Hardware user/kernel mode is by far the most popular plan.

3.6 Monolithic kernels and microkernels

In a MONOLITHIC KERNEL the whole OS runs in kernel space, as in xv6 and Linux. The kernel interface is the system call interface, and the kernel is one big program containing the file system, drivers, virtual memory, and so on (Figure 3a). Subsystems can easily cooperate (for example, one cache shared by the file system and virtual memory), and the tight integration helps performance. But the interactions are complex, which leads to bugs, and there is no isolation within the kernel.

In a MICROKERNEL design many OS services run as ordinary user programs, for example the file system in a file server (Figure 3b). The kernel implements a minimal mechanism for running services in user space: processes with memory, inter-process communication (IPC), and multiplexing of the hardware. Applications obtain services by sending messages through the kernel to the servers, so the kernel interface is not the system call interface. The kernel is small, which means fewer bugs and more isolation, but good performance may be hard to achieve. Both designs are widely used.



(a) monolithic kernel

(b) microkernel

Figure 3: (a) A monolithic kernel: process management, virtual memory, and the file system all run in kernel mode. (b) A microkernel: the file system and other services run as user-level servers, and applications reach them by IPC messages through a small kernel.

3.7 Xv6 and its machine

Xv6 is a monolithic kernel whose kernel interface is the UNIX system call interface. By convention its source reflects the organization: `user/` holds programs that run in user mode and `kernel/` the code that runs in kernel mode, and `kernel/defs.h` declares the interfaces of the kernel's parts (`proc`, `fs`, ...). The goal is to be able to read the source and understand it.

The Makefile builds the kernel program, the user programs, and `mkfs`, and `make qemu` runs xv6 on QEMU. To build the kernel, `gcc` compiles each `.c` file (`proc.c`, `pipe.c`, ...) to a RISC-V assembly `.S` file, the assembler turns that into a `.o` file, and `ld` links the object files into the kernel. The Makefile keeps a disassembly of the binary, `kernel/kernel.asm`, for inspection.

The emulated RISC-V computer is a very simple board (it has no display, for example):

- a RISC-V processor with 4 cores and 128 MB of RAM;
- interrupt support (the PLIC and CLINT controllers);
- a UART, which lets xv6 write to the console and read from the keyboard;
- an e1000 network card, attached through PCIe.

QEMU's "virt" machine is close to SiFive's HiFive Unleashed board (whose FU540-C000 chip contains the RISC-V cores, an L2 cache, a DDR memory controller, UARTs, and clock control), but uses virtio for the disk. Developing under QEMU is more convenient than on real hardware.

To *emulate* a RISC-V computer, QEMU is a C program that faithfully implements a RISC-V processor. It keeps the processor state (registers `x0`, `x1`, ..., memory) in data structures and loops:

```
for (;;) {
  read next instruction
  decode instruction        (add? sub? ...)
  execute instruction       (e.g. add a0, 7, 1: update the state)
}
```

The big idea is that software can be hardware.

3.8 Booting xv6 and the first system call

Running `make CPUS=1 qemu-gdb` boots xv6 under gdb with one core. QEMU starts xv6 at `_entry` in `kernel/entry.S`, where the linker script `kernel/kernel.ld` places the kernel. Following the boot in gdb:

1. A breakpoint at `_entry` shows the first kernel instruction and the registers (`info reg`).
2. A breakpoint at `main` lets one walk through the kernel's initialization.

3. Stepping into `userinit()` shows the creation of the first process: a `struct proc` (`proc.h`) obtained from `allocproc()`, whose memory holds the small program `initcode.S`.
4. A breakpoint at `forkret()` leads to `userret`, where the kernel returns to user space for the first time and runs `initcode`.
5. A breakpoint at `syscall()` catches the first system call: `num` is the number of `exec`, and `syscalls[num]` is its implementation.

The first user program does nothing but execute `/init`:

```
# exec(init, argv)
start:
    la a0, init         # first argument: "/init"
    la a1, argv         # second argument
    li a7, SYS_exec     # system call number
    ecall
```

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 3 (Fall 2020).

Lecture 4: Page Tables

4.1 Address spaces

Suppose the shell has a bug that sometimes makes it write to a random memory address. If all programs and the kernel share physical memory (the kernel low, then `sh` at addresses `1000`–`2000`, then `cat`, ...), a store such as `sd $7, (a0)` with `a0 = 1000` can wreck the kernel or another process. The goal is ISOLATED ADDRESS SPACES: each process has its own memory, which it can read and write, and it cannot read or write anything else. Each address space starts at 0 and runs to some maximum. The challenge is to multiplex several memories over one physical memory while maintaining isolation between them. Xv6 uses RISC-V's paging hardware to implement address spaces.

4.2 Paging hardware

Paging provides a level of indirection for addressing. The CPU issues VIRTUAL ADDRESSES (VAs), and a MEMORY MANAGEMENT UNIT (MMU) translates each to a PHYSICAL ADDRESS (PA) before it reaches RAM:

$$\text{CPU} \xrightarrow{\text{VA}} \text{MMU} \xrightarrow{\text{PA}} \text{RAM}.$$

Software can load and store only through virtual addresses. The kernel tells the MMU how to map each virtual address to a physical one; the MMU essentially has a table indexed by VA that yields the PA, called a PAGE TABLE, and there is one page table per address space (so every application has its own map). The MMU can also restrict which virtual addresses user code may use. By programming the MMU, the kernel has complete control over the VA → PA mapping, which allows many interesting OS features and tricks.

RISC-V maps memory in 4 KB PAGES, aligned to start on 4 KB boundaries; 4 KB = 2^{12} bytes, so the low 12 bits of an address are the offset within the page. The RISC-V used by xv6 has 64-bit addresses, so the page-table index would be the top $64 - 12 = 52$ bits of the VA, but the top 25 of those are unused (no RISC-V machine has that much memory now; the address space can grow in the future). The index is therefore 27 bits, and a virtual address space spans $2^{27} \cdot 2^{12} = 2^{39}$ bytes = 512 GB.

The MMU uses the index bits of the VA to find a PAGE TABLE ENTRY (PTE), then forms the PA from the PHYSICAL PAGE NUMBER (PPN) in the PTE followed by the 12-bit offset from the VA. Each PTE is 64 bits, of which 54 are used: the top 44 of these are the PPN, the top bits of the physical address, and the low 10 are flags (Figure 4). A physical address therefore has $44 + 12 = 56$ bits; virtual and physical addresses need not have the same size. Page tables are stored in RAM: the MMU loads (and stores) PTEs, and the OS reads and writes PTEs as ordinary memory locations.

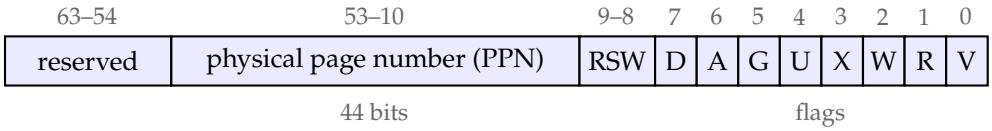


Figure 4: A RISC-V (Sv39) page table entry: a 44-bit physical page number and 10 flag bits. V valid, R readable, W writable, X executable, U accessible in user mode, G global, A accessed, D dirty; RSW is reserved for supervisor software.

4.3 Three-level page tables

Could the page table simply be an array of PTEs indexed by the 27-bit index? It would have $2^{27} \approx 134$ million entries of 8 bytes each, about 1 GB per page table, with one page table per address space and one address space per application. Small programs need mappings for only a few hundred pages, so almost all of that memory would be wasted.

RISC-V instead uses a THREE-LEVEL PAGE TABLE. A PAGE DIRECTORY is one 4096-byte page holding $4096/8 = 512$ PTEs, each of which either points to another page directory or is a leaf; there are $512 \cdot 512 \cdot 512 = 2^{27}$ leaf PTEs in total. Entries can be invalid, in which case the page directories below them need not exist, so the page table for a small address space is small. The `satp` register holds the physical address of the top-level directory; the directory pages can be anywhere in RAM and need not be contiguous, and the kernel rewrites `satp` when it switches to another address space.

To translate a VA, the hardware starts at the L2 directory named by `satp`. The top 9 index bits select a PTE there, which gives the PA of an L1 directory; the next 9 bits select a PTE in that, giving the PA of an L0 directory; the last 9 bits select the leaf PTE, whose PPN together with the low 12 bits of the VA forms the PA (Figure 5).

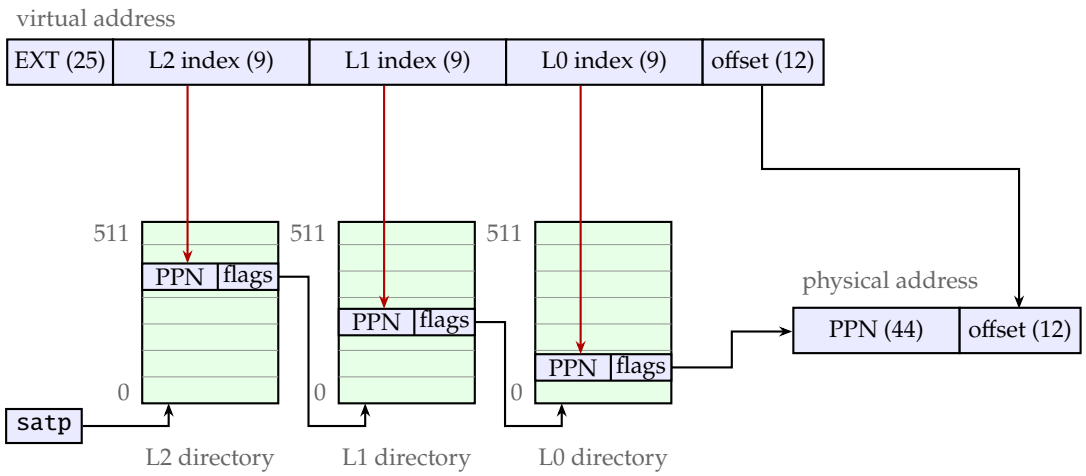


Figure 5: Sv39 translation. `satp` names the L2 page directory; the three 9-bit index fields of the virtual address select one PTE in each of three page directories, and the PPN of the last PTE followed by the 12-bit offset is the physical address. Each directory is one page of 512 PTEs.

Note. The sizes follow from the levels: one L0 PTE maps 4 KB, one L1 PTE covers $512 \cdot 4$ KB = 2 MB, one L2 PTE covers $512 \cdot 2$ MB = 1 GB, and the whole table covers 512 GB. Even a single mapping needs three page-table pages (one per level).

4.4 Flags, page faults, and the TLB

The flag bits V, R, W, X, and U are all used by xv6. If an access hits a PTE whose V bit is clear, or stores to a page without W set, the hardware raises a PAGE FAULT, which forces a transfer into the kernel (`trap.c` in xv6). The kernel can simply report an error and kill the process (xv6 prints `usertrap(): unexpected scause ... pid=... sepc=... stval=...`), or it can install a PTE and resume the process, for example after loading the page from disk.

Walking three levels on every access would be slow, so the CPU caches recent translations in a TRANSLATION LOOK-ASIDE BUFFER (TLB), a cache of [VA, PA] pairs taken from PTEs. When the kernel switches page tables it must flush the TLB, which on RISC-V is the `sfence.vma` instruction.

Because page tables are a level of indirection under the OS's control, paging solves many problems: physical memory need not be contiguous (which avoids fragmentation), and it enables lazy allocation, copy-on-write fork, and many more techniques.

Why does the kernel itself use virtual memory? It is clearly good to have page tables for user processes, and the kernel could run using only physical addresses, but most standard kernels use virtual addresses. The reasons are not fundamental. The hardware makes paging hard to turn off (on entering a system call one would have to disable it). And the kernel benefits from virtual addresses itself: it can mark text pages executable but not data (which helps catch bugs), leave the page below each kernel stack unmapped (also helps catch bugs), and map a page into both the user and the kernel address space (which helps the user/kernel transition).

4.5 The xv6 kernel address space

The kernel's page table maps virtual addresses mostly one-to-one onto physical addresses (Figure 6). The machine's devices sit at fixed physical addresses below `0x80000000`, and the kernel maps them at the same virtual addresses because it controls devices by loading from and storing to their registers. RAM (128 MB) starts at `KERNBASE` = `0x80000000`; the kernel's text is mapped R-X, its data and the free memory up to `PHYSTOP` RW-. Because physical address x is mapped at virtual address x, it is easy for the kernel to read and write physical memory. At the top of the address space the kernel maps the trampoline page a second time and each process's kernel stack, with an unmapped guard page below each stack.

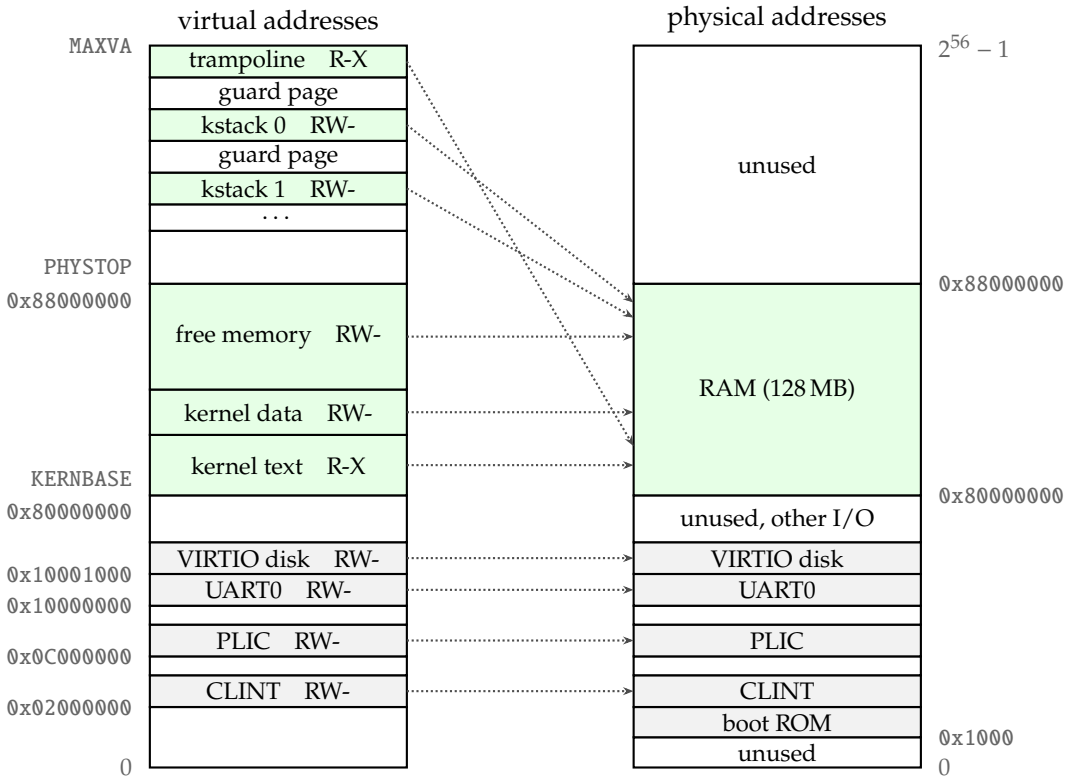


Figure 6: The xv6 kernel address space (not to scale). Devices, kernel text, kernel data, and free memory are mapped at virtual addresses equal to their physical addresses; the trampoline page and the kernel stacks are also mapped high, each stack above an unmapped guard page.

4.6 Process address spaces

Each process has its own address space and its own page table (Figure 7), and the kernel switches page tables (sets `satp`) when it switches processes. The arrangement has several reasons.

- User virtual addresses start at zero; of course user VA 0 maps to a different PA in each process.
- The heap has room to grow contiguously up to `MAXVA` (2^{38} bytes in xv6, i.e. 256 GB), yet needs no contiguous physical memory, so there is no fragmentation problem.
- Both the kernel and every user page table map the trampoline page (and each user table its process's trapframe page) at the top, which eases the transition from user to kernel and back. These pages lack the U flag, so the user process cannot touch them.
- The kernel does not map user memory, so it is not easy for the kernel to read or write user memory: it must translate the user virtual address itself. That is good for isolation (consider the Spectre attacks).

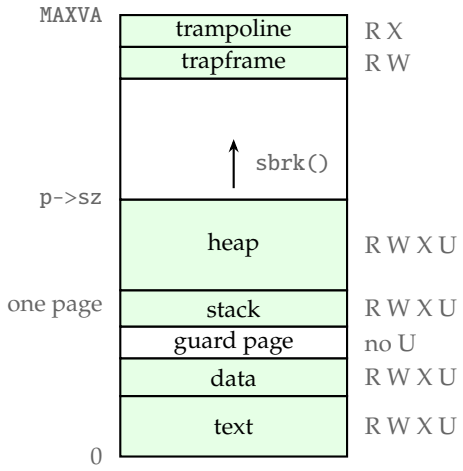


Figure 7: An xv6 process's address space, with the flags of its PTEs. Text and data start at 0; the one-page stack sits above a guard page that user code cannot access (no U flag); the heap grows upward as `sbrk()` raises `p->sz`; the trapframe and trampoline pages occupy the top.

4.7 Code: building the kernel page table

At boot, `kvminit()` builds the kernel page table with a series of `kvmmap()` calls, one per region of Figure 6. Printing the table after each call answers some sizing questions:

- After the first `kvmmap()` (for the UART at `0x10000000` = 256 MB), the table already uses three pages, one per level.
- The CLINT occupies 16 pages, and the PLIC 1024 pages, which fill two lowest-level page-table pages.
- The kernel text was 8 pages in the build examined; the whole kernel mapping, `KERNBASE` to `PHYSTOP`, is 128 MB = 64 · 2 MB, i.e. 64 L1 entries.
- The trampoline is mapped twice: at the top of the address space (just below `MAXVA`) and, as part of the kernel text, in the direct map.

`kvminithart()` then writes `satp` and executes `sfence.vma`. The instruction after the `satp` write is fetched through the new page table, and it is found at the same address only because the kernel text is mapped at virtual addresses equal to its physical addresses.

`mappages(pagetable, va, size, pa, perm)` adds mappings for a range of virtual addresses to the corresponding physical addresses. It rounds the range to page boundaries, since some callers pass unaligned addresses, and for each page it calls `walk()` to find the *address* of the PTE (not just its contents, since it must modify the PTE), stores the PA and permissions there, and marks the PTE valid with `PTE_V`.

`walk()` mimics in software how the paging hardware finds the PTE for an address. `PX(level, va)`

extracts the 9 index bits for a level, so `&pagetable[PX(level, va)]` is the address of the relevant PTE. If that PTE is valid, the next-level page-table page exists and `PTE2PA` extracts its address from the PPN; otherwise `walk()` allocates a zeroed page-table page and fills in the PTE with its PPN (`PA2PTE`). After two levels, the wanted PTE is in the lowest-level page:

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) {
  for(int level = 2; level > 0; level--) {
    pte_t *pte = &pagetable[PX(level, va)];
    if(*pte & PTE_V) {
      pagetable = (pagetable_t)PTE2PA(*pte);
    } else {
      if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
        return 0;
      memset(pagetable, 0, PGSIZE);
      *pte = PA2PTE(pagetable) | PTE_V;
    }
  }
  return &pagetable[PX(0, va)];
}
```

`procinit()` in `proc.c` allocates a page for each process's kernel stack and maps it high, with a guard page below it.

4.8 Code: user address spaces and sbrk

`allocproc()` allocates an empty top-level page table for a new process; `fork()` copies the parent's memory and mappings with `uvmcopy()`; `exec()` builds a new page table, allocating memory with `uvmalloc()` and loading program segments into it with `loadseg()`, and replaces the process's old table with it.

A process calls `sbrk(n)` to ask for n more bytes of heap memory; the user memory allocator in `user/umalloc.c` calls it to get memory. Each process has a size, `p->sz`, and the kernel adds new memory at the process's end and increases the size: it allocates physical memory, maps it into the process's page table, and returns the starting address of the new memory. In the kernel `growproc()` does this with `uvmalloc()`, and when the process returns to user space, `satp` is loaded with the updated page table. `uvmalloc()` starts at `PGROUNDUP(oldsz)`, because the page containing the old end of the process is already mapped, and for each new page it allocates and zeroes a physical page and maps it user-accessible and readable, writable, and executable with `mappages()`.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 4 (Fall 2020).

Lecture 5: Calling Conventions and Stack Frames RISC-V

C code is compiled to machine instructions. The machine's instruction set architecture (ISA) defines those instructions: a C program is translated to assembly (`.S` files), which is assembled into binary object files of instructions such as `add` and `mul`. Knowing how this translation works, and how C and assembly interact, matters because some code (for example, kernel entry) cannot be expressed in C.

5.1 RISC-V and x86

RISC-V is a reduced instruction set (RISC) architecture. The x86-64 ISA of most personal computers (Intel, AMD) is a complex instruction set (CISC): its manual fills three full books, it gains about three new instructions per month, and it has around 15,000 instructions. RISC-V has fewer and simpler instructions and is open source. ARM is also a RISC architecture, used for example in Qualcomm Snapdragon processors (Android) and in Apple's iOS devices; RISC-V is aimed in particular at integrated devices.

5.2 The RISC-V abstract machine

The machine has no C-like control flow, no concept of variables, and no types. The base ISA has a program counter and 32 general-purpose registers `x0`–`x31`. Programs load values from memory into registers, operate on registers, and store registers back to memory. The registers have conventional names and roles (Table 1): a caller-saved register is not preserved across a function call, so a caller that needs its value must save it, while a callee-saved register is preserved across calls, so a function that uses it must save and restore it.

Reg.	Name	Saver	Role	Reg.	Name	Saver	Role
`x0`	`zero`		hard-wired zero	`x9`	`s1`	callee	saved register
`x1`	`ra`	caller	return address	`x10`–`11`	`a0`–`1`	caller	arguments / return values
`x2`	`sp`	callee	stack pointer	`x12`–`17`	`a2`–`7`	caller	arguments
`x3`	`gp`		global pointer	`x18`–`27`	`s2`–`11`	callee	saved registers
`x4`	`tp`		thread pointer	`x28`–`31`	`t3`–`6`	caller	temporaries
`x5`–`7`	`t0`–`2`	caller	temporaries	`pc`			program counter
`x8`	`s0/fp`	callee	saved register / frame pointer				

Table 1: RISC-V integer registers and their roles in the calling convention.

The floating-point registers `f0`–`f31` follow the same pattern: temporaries `ft0`–`ft11` and arguments `fa0`–`fa7` (with `fa0`–`fa1` also returning values) are caller-saved, and `fs0`–`fs11` are callee-saved.

The running example sums the integers up to n:

```c
int sum_to(int n) {
  int acc = 0;
  for (int i = 0; i <= n; i++) {
    acc += i;
  }
  return acc;
}
```

```
# sum_to(n): expects argument in a0, returns result in a0
sum_to:
  mv t0, a0          # t0 <- a0
  li a0, 0           # a0 <- 0
loop:
  add a0, a0, t0     # a0 <- a0 + t0
  addi t0, t0, -1    # t0 <- t0 - 1
  bnez t0, loop      # if t0 != 0: pc <- loop
  ret
```

The assembly adds $n, n-1, \ldots, 1$, which matches the C code for $n \geq 1$. Assembly offers only limited abstractions: no typed, positional arguments, no local variables, only registers. The machine does not even see assembly: it sees the binary encoding of each instruction, 16 or 32 bits long (`mv t0, a0` is encoded as `0x82aa`). The correspondence between assembly and encoding is close to, but not quite, one-to-one.

5.3 Calls and returns

Another function calls `sum_to` by putting the argument in `a0` and executing `call`:

```
main:
  li a0, 10          # a0 <- 10
  call sum_to
```

`call label` sets `ra` ← `pc + 4` (the address of the next instruction) and `pc` ← `label`; `ret` sets `pc` ← `ra`. The machine does not understand labels: they are translated into pc-relative or absolute jumps.

A function that itself calls another function breaks this scheme. If `sum_then_double` simply does `call sum_to` and then doubles `a0`, its own `ret` jumps back into itself, an infinite loop, because the inner `call` overwrote `ra`. Saving `ra` in another register only defers the problem; the solution is to save it on the stack:

```
sum_then_double:
  addi sp, sp, -16   # prologue: make space on the stack,
  sd ra, 0(sp)       #   save registers
  call sum_to
  li t0, 2           # t0 <- 2
  mul a0, a0, t0     # a0 <- a0 * t0
  ld ra, 0(sp)       # epilogue: restore registers,
  addi sp, sp, 16    #   restore the stack pointer
  ret
```

The stack grows toward lower addresses, so the prologue subtracts from `sp` and the epilogue adds the same amount back.

5.4 The calling convention

These functions worked because one author wrote all of them; arguments could just as well have been passed in `t2` and results returned in `t3`. The rules that let independently written code interoperate form the CALLING CONVENTION:

- arguments are passed in `a0`, `a1`, . . . , `a7`, and the rest on the stack;
- values are returned in `a0` and `a1`;
- each register is designated caller-saved or callee-saved (Table 1).

C code generated by GCC follows the convention, and so must hand-written assembly; then everyone's code interoperates, including C with assembly. Compiled code has a PROLOGUE, a body, and an EPILOGUE. A LEAF FUNCTION, one that calls no other function, need not save `ra`. Compiled with `-fno-omit-frame-pointer`, every function also maintains `s0`/`fp` as a frame pointer.

5.5 Stack frames

Each call creates a STACK FRAME holding the return address, the caller's frame pointer, saved registers, and local variables (Figure 8). The stack pointer `sp` points to the bottom of the stack, the end of the current frame, and the frame pointer `fp` points to the top of the current frame. The return address is at `fp`−8 and the previous frame pointer at `fp`−16, so the saved frame pointers link the frames into a chain that a debugger can follow.

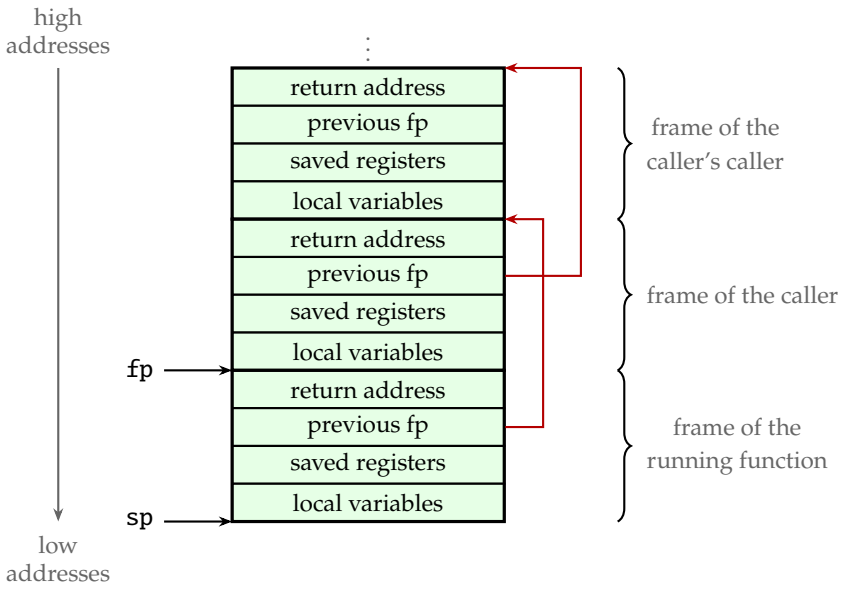


Figure 8: Stack frames. Each frame stores the return address and the caller's frame pointer at its top; `fp` points to the top of the current frame and `sp` to its bottom. The saved frame pointers (red) chain each frame to its caller's.

Example: In a program where `main` calls `f` and `f` calls `g`, stopping in `g` after its prologue gives `sp` = `0x2fb0` and `fp` = `0x2fc0`, and the stack holds:

Address	Contents	Frame
`0x2fd8`	garbage return address	`main`
`0x2fd0`	garbage frame pointer	`main`
`0x2fc8`	return address into `main`	`f`
`0x2fc0`	`0x2fe0` (frame pointer of `main`)	`f`
`0x2fb8`	return address into `f`	`g`
`0x2fb0`	`0x2fd0` (frame pointer of `f`)	`g`

In gdb, `x/g $fp-16` prints `0x2fd0`, the frame pointer of `f`, and `x/g 0x2fd0-16` prints `0x2fe0`, that of `main`. Gdb automates this walk (`backtrace`, `info frame`, `frame 1`, ...), and with debug information it can also reason about leaf functions.

5.6 Mixing C and assembly

Calling assembly from C, or C from assembly, works as long as both sides follow the calling convention; a function prototype tells the C compiler how to call an assembly function. An assembly function that calls into C, such as `sum_squares_to`, keeps the values it needs after those calls in callee-saved registers like `s0` and `s1` (saving and restoring them in its own prologue and epilogue) rather than in `t0` and `t1`, which the called C code may overwrite. C also supports inline assembly.

5.7 Structs

C lays out the fields of a struct in memory in declaration order, following alignment rules, because misaligned loads and stores can be slow or unsupported, depending on the platform. The attribute `__attribute__((packed))` removes the padding. Structs are generally passed to assembly by reference, and assembly that manipulates a struct must know its layout, that is, the offset of each field.

5.8 Debugging with gdb

- `x/nfu addr` examines memory: n units of size u printed in format f.
- `step` runs to the next line of C code, `next` does the same but steps over function calls, and `finish` runs until the current function returns.
- `stepi` and `nexti` do the same for assembly instructions.
- `layout next` (or `layout split`) cycles through text layouts showing source, assembly, and registers; `info registers` and `p $a0` show register values; `advance` runs to a location.
- Conditional breakpoints stop only when a condition holds; watchpoints stop when a memory location changes value.

Source. 6.S081 course staff, *6.S081: Operating System Engineering*, Lecture 5 (Fall 2020).

Lecture 6: Isolation and System Call Entry/Exit

System calls, faults, and interrupts all enter the kernel in the same way, through a TRAP. The user-to-kernel transition matters for both isolation and performance, and it involves careful design and much important detail. (The system call implementation itself must also be careful and secure, but the focus here is the transition.)

6.1 What a trap must do

While a program runs, the CPU's resources are set up for user execution: the 32 registers (including `sp`), the `pc`, the privilege mode, and control registers such as `satp`, `stvec`, and `sepc`. To execute a system call such as `write()` the kernel must

- save the 32 user registers and the `pc`,
- switch to supervisor mode,
- switch to the kernel page table and to a kernel stack,
- and jump to kernel C code.

Two high-level goals constrain how. User code must not be able to interfere with the transition (for example, user code must never execute in supervisor mode), and the transition must be transparent: the user code resumes afterwards without being disturbed.

Switching from user to supervisor mode grants only two things. Supervisor code can use the CPU control registers: `satp` (the page table's physical address), `stvec` (where `ecall` jumps in the kernel; it points to the trampoline), `sepc` (where `ecall` saves the user `pc`), and `sscratch` (here, the address of the trapframe). And supervisor code can use PTEs that lack the `PTE_U` flag. It has no other powers; in particular it cannot use addresses that are not in the current page table, so the kernel must set things up carefully before it can work.

The path of a system call through xv6 is shown in Figure 9. The rest of the unit follows the shell's `write(2, "$ ", 2)`, which prints its prompt, along this path in gdb.

6.2 From write() to ecall

The user-space `write()` is a stub in `user/usys.S`:

```
write:
  li a7, SYS_write   # system call number, 16
  ecall              # at 0xde6 in the shell
  ret
```

Register `a7` tells the kernel which system call is wanted, and `ecall` triggers the transition. Stopped at the `ecall` (`b *0xde6`), the `pc` and `sp` are low addresses, since user memory starts at zero, and the arguments are in the C argument registers: `a0` is the FD (2), `a1` the buffer (`x/2c $a1` shows `$` and a space), and `a2` the count (2). Printing `satp` is not very useful, but qemu's `info mem` shows that the user page table maps just six pages: instructions, data, a stack guard page (without `PTE_U`), the stack, and two high pages, the trapframe and the trampoline. There are no mappings for kernel memory, devices, or physical memory.

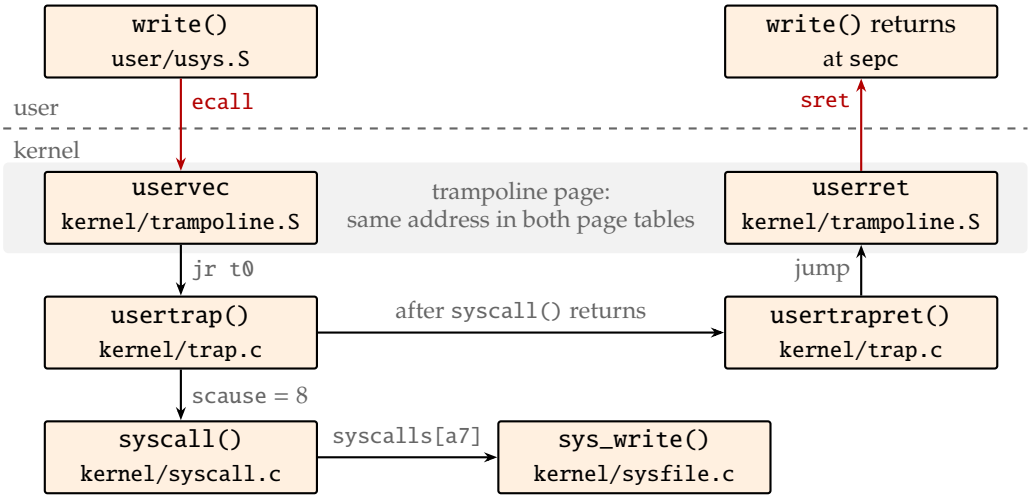


Figure 9: A system call's path through xv6. `ecall` enters the trampoline code `uservec`, which calls `usertrap()`; it dispatches to `sys_write()` through `syscall()`. On the way out, `usertrapret()` jumps to `userret` in the trampoline, whose `sret` resumes the user program.

6.3 ecall and the trampoline

After `stepi` over the `ecall`, the `pc` is `0x3ffffff000`, a very high virtual address in the last page of the user address space, and the instructions there are the start of the kernel's trap handler, `uservec` in `kernel/trampoline.S`. The registers still hold the user values (except `pc`), and the user page table is still in use. Since `ecall` does not switch page tables, the kernel's first trap instructions must be mapped in every user page table: the kernel maps the TRAMPOLINE page at the top of each one, protected by the absence of `PTE_U`, and sets `stvec` to its virtual address. That the CPU executes from a page without `PTE_U` without faulting shows it is now in supervisor mode.

`ecall` did exactly three things: it changed the mode from user to supervisor, saved the `pc` in `sepc`, and jumped to `stvec` (which the kernel set before entering user space). So `ecall` lets user code switch to supervisor mode, but the kernel immediately gains control at an address it chose; the user program itself never executes as supervisor.

Everything else is left to software: saving the 32 user registers, switching to the kernel page table, setting up a stack for kernel C code, and jumping to it. `ecall` does as little as possible to give operating system designers room for very fast system calls, faults, and interrupts: a kernel might handle some traps without switching page tables, map both user and kernel into a single page table, skip saving some registers, or need no stack for simple system calls. Many clever kernel entry schemes exist, with different divisions of work between CPU and software, because performance here is often very important.

6.4 Saving the user registers

The user registers cannot simply be written to some convenient physical address, since even supervisor mode must go through the page table. Nor can `uservec` first load the kernel page table into `satp`: it does not know that table's address, and it would need a free register even to execute `csrw satp`. Xv6 solves the problem in two parts.

1. It maps a second kernel page, the TRAPFRAME, into every user page table, at `0x3ffffe000` just below the trampoline. It has room for the saved registers (`struct trapframe` in `kernel/proc.h`), and each process has its own.
2. RISC-V provides the `sscratch` register. Before entering user space the kernel puts the trapframe's address in `sscratch`; supervisor code can swap any register with `sscratch`, thereby both obtaining the trapframe address and saving that register's user value.

So `uservec` begins with `csrrw a0, sscratch, a0`; afterwards `a0` holds the trapframe's address and `sscratch` holds `0x2`, the user's `a0` (the FD argument). Then 32 stores save the user registers into the trapframe through `a0`.

Next `uservec` prepares to run C code. At this point the only kernel data it can reach is the trapframe, so it loads everything from there, where the kernel stored it before the previous return to user space:

```
uservec:
  csrrw a0, sscratch, a0   # a0 <- trapframe, sscratch <- user a0
  sd ra, 40(a0)            # save the user registers ...
  # ...
  ld sp, 8(a0)             # top of this process's kernel stack
  ld tp, 32(a0)            # hart ID
  ld t0, 16(a0)            # address of usertrap()
  ld t1, 0(a0)             # kernel page table
  csrw satp, t1            # switch to the kernel page table
  sfence.vma zero, zero    # flush stale TLB entries
  jr t0                    # jump to usertrap()
```

Switching page tables while executing does not crash, because the trampoline page is mapped at the same virtual address in the kernel page table as in every user page table. With the kernel page table installed, kernel functions and data are usable, and `jr t0` jumps to `usertrap()`.

6.5 usertrap() and syscall()

`usertrap()` in `kernel/trap.c` handles all traps from user space: errors, device interrupts, and system calls. It reads the `scause` register to find the cause; `scause` = 8 is a system call, so it calls `syscall()` in `kernel/syscall.c`. There `myproc()` uses `tp` to find the current `struct proc`, and `p->trapframe` points to the saved registers: `p->trapframe->a7` holds 16 (`SYS_write`) and `a0`, `a1`, `a2` hold the FD, buffer, and count. `syscall()` dispatches through `syscalls[num]`, a table of functions, to `sys_write()`, which from here on is fairly ordinary C code.

When `sys_write()` returns (the prompt has appeared on the console), `syscall()` assigns its return value to `p->trapframe->a0`. Because the RISC-V C calling convention returns values in `a0`, this eventually becomes the return value of the user's `write()`: here 2, for the two characters.

6.6 Returning: usertrapret(), userret, and sret

`usertrapret()` starts the return to user space. It first prepares for the next user-to-kernel transition: it sets `stvec` to `uservec` in the trampoline, for the next `ecall`, and fills in the trapframe fields that `uservec` will need: the kernel page table, the top of the process's kernel stack, the address of `usertrap()`, and the hart ID (from `tp`). The final `sret` instruction uses two registers, which it also prepares: `sstatus`, whose "previous mode" bit is set to user, and `sepc`, set to the user program counter saved at trap entry. Switching to the user page table cannot happen in `usertrapret()`, since that code is not mapped in the user page table, so it jumps to `userret` in the trampoline, the one page mapped in both. `userret` receives `TRAPFRAME` in `a0` and the user page table in `a1`:

```
userret:
  csrw satp, a1            # switch to the user page table
  sfence.vma zero, zero
  ld t0, 112(a0)           # saved user a0 (the return value)
  csrw sscratch, t0        #   into sscratch, for the final swap
  ld ra, 40(a0)            # restore the user registers ...
  # ...
  csrrw a0, sscratch, a0   # a0 <- user a0, sscratch <- trapframe
  sret                     # user mode, pc <- sepc
```

After the swap, `a0` holds the return value of `write()` and `sscratch` holds the trapframe pointer, which is what `uservec` expects at the next trap. `sret` switches to user mode and jumps to `sepc`: the program resumes at `0xdea`, the instruction after the `ecall` (`0xde6` + 4), with `a0` = 2.

System call entry and exit is far more complex than a function call. Much of the complexity comes from the requirement of isolation and from the desire for simple and fast hardware mechanisms. Open design questions remain: can an evil program abuse the entry mechanism, can the hardware or software be made simpler, and can traps be made faster?

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 6 (Fall 2020).

Lecture 7: Q&A Labs

This session walks through solutions to the first labs, starting with the page-table lab, the hardest: it needs few lines of code but produces difficult-to-debug failures. In the worst case qemu and xv6 simply stop running; in the "best" case the kernel panics. There are so many possible causes that such bugs are hard to track down even for experienced staff, and the first virtual-memory lab has historically been the most challenging one.

7.1 Reading a page table

The first part of the lab prints a page table. For the `init` process the output is

```
page table 0x0000000087f67000
..0: pte 0x0000000021fd8c01 pa 0x0000000087f63000 fl 0x0000000000000001
.. ..0: pte 0x0000000021fd8801 pa 0x0000000087f62000 fl 0x0000000000000001
.. .. ..0: pte 0x0000000021fd901f pa 0x0000000087f64000 fl 0x000000000000001f
.. .. ..1: pte 0x0000000021fd840f pa 0x0000000087f61000 fl 0x000000000000000f
.. .. ..2: pte 0x0000000021fd801f pa 0x0000000087f60000 fl 0x000000000000001f
..255: pte 0x0000000021fd9801 pa 0x0000000087f66000 fl 0x0000000000000001
.. ..511: pte 0x0000000021fd9401 pa 0x0000000087f65000 fl 0x0000000000000001
.. .. ..510: pte 0x0000000021fdd807 pa 0x0000000087f76000 fl 0x0000000000000007
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000 fl 0x000000000000000b
```

Each line is a valid PTE; the indentation gives its level and the number its index in that page-table page. The physical address is the PTE's physical page number (`pte >> 10`) shifted left by 12: for example `0x21fd8c01` gives `0x87f63000`. The flag bits are `V` = 1, `R` = 2, `W` = 4, `X` = 8, `U` = `0x10`. Entries at the top two levels have only `V` set: they point to the next page-table page. The leaves, read against the user address-space layout, are:

Indices	Virtual address	Flags	Page
0, 0, 0	`0x0`	`U X W R V`	text and data
0, 0, 1	`0x1000`	`X W R V`	guard page (no `PTE_U`)
0, 0, 2	`0x2000`	`U X W R V`	stack
255, 511, 510	`0x3fffffe000`	`W R V`	trapframe
255, 511, 511	`0x3ffffff000`	`X R V`	trampoline

The guard page is mapped, but without `PTE_U`, so a user access to it (a stack overflow) faults. The trapframe is readable and writable but not executable, the trampoline readable and executable but not writable, and neither is accessible to user code. Consecutive virtual pages need not be physically

contiguous: the first three leaves map to `0x87f64000`, `0x87f61000`, and `0x87f60000`. All these pages lie near the top of RAM, which occupies 128 MB starting at `0x80000000` (kernel instructions and data at the bottom, free memory above), because the allocator hands out the highest free pages first.

The top-level index of the trampoline is 255, not 511, because xv6 uses only 38 of the 39 virtual-address bits of the RISC-V scheme: `MAXVA` $= 2^{38}$, so the highest page lies at top-level index $2^8 - 1 = 255$. (Using one bit fewer avoids having to sign-extend addresses with the high bit set.)

7.2 A kernel page table per process

The second part gives each process its own kernel page table, an exact copy of the global `kernel_pagetable`, in preparation for the third part. This is harder than it seems:

1. xv6's code is specialized to a single kernel page table, with no function that creates one;
2. `kvminit()` is not the full story, since `procinit()` (kernel stacks) and `virtio_disk.c` also add mappings;
3. cleanup must be done carefully, for instance without freeing the kernel stacks;
4. small errors are easy to make, and lead to bugs that are time-consuming to track down.

There are two approaches. One generalizes `kvminit()` to build a new kernel page table per process. The other shares as much as possible with `kernel_pagetable`: a top-level entry covers 1 GB, so `kvmcreate()` can copy top-level entries 1–511 (which then point to the same lower-level page-table pages as the global table) and add only the few devices that live in entry 0, below 1 GB (Figure 10). Then `kvmfree()` need only consider entry 0, freeing its intermediate page-table pages but not the pages they map; this avoids a whole class of bugs, such as freeing kernel stacks. `scheduler()` and `usertrapret()` must use the process's kernel page table, and `kvmfree()` must be called when a process is freed. In either approach it pays to proceed in small incremental steps that keep the existing code working, which makes it easy to compare old and new behavior and to roll back.

7.3 Mapping user memory into the kernel page table

The third part maps each process's user page table into the bottom of its kernel page table. This is convenient for kernel programmers: with user addresses valid in the kernel, `copyin()` can be replaced by `copyin_new()`, which is just a `memmove()` after checks that the range lies within the process's memory and does not wrap around. Instead of the software walks in `walkaddr()`, the paging hardware does the walks.

A function `kvmmapuser()` copies the user PTEs into the kernel page table, adjusting the permissions: the copies must have `PTE_U` cleared, because supervisor code cannot use pages marked `PTE_U`. It is

called wherever user mappings change: `userinit()`, `exec()` (which need not replace the kernel page table), `fork()`, and `sbrk()`. Because user memory now shares entry 0 with the devices, user processes may grow only up to the PLIC at `0x0C000000`.

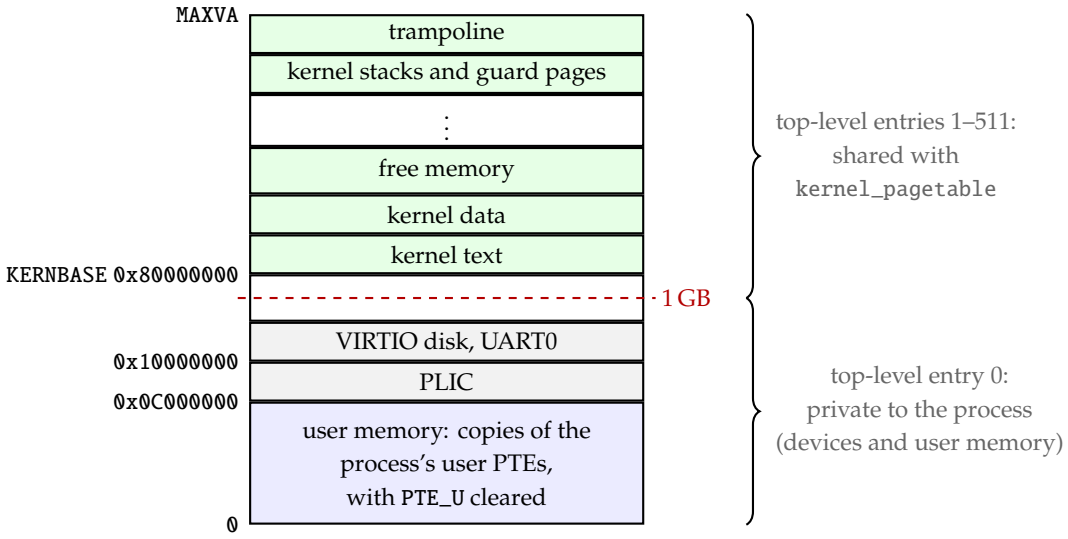


Figure 10: A per-process kernel page table (not to scale). Above 1 GB it shares the global kernel page table's top-level entries; entry 0 holds the low device mappings and, below the PLIC, copies of the process's user mappings.

Two questions about other designs have short answers. Linux, in response to the Meltdown attack, adopted KERNEL PAGE-TABLE ISOLATION (KPTI): while user code runs, its page table omits the kernel's mappings, as xv6's separate user page tables already do. And the PLIC limit on user memory could be removed by mapping the devices, such as the VIRTIO disk, at high virtual addresses near the kernel stacks instead of at their physical addresses.

7.4 The system-call and utilities labs

System calls look like function calls, but they are not: they cross the user/kernel boundary. In xv6, `user/usys.pl` generates the user-space stubs (the declarations live in `user/user.h` and the numbers in `kernel/syscall.h`), and `kernel/syscall.c` demultiplexes system calls through its table of implementation functions. The `trace` call records its mask in the process structure (`kernel/proc.h`), and `fork()` must copy it to the child; `sysinfo` returns its result to user memory with `copyout()`.

In the utilities lab, `primes` must close the file descriptors it does not use so that the pipeline terminates correctly (a reader sees end of file only when every write end of its pipe is closed), and `xargs` must set up the `argv` array it passes to `exec`.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 7 (Fall 2020).

Lecture 8: Page Faults

Virtual memory has two views. Its primary purpose is isolation: each process has its own address space. But a page table is also a level of indirection between virtual and physical addresses, and that indirection gives the kernel an opportunity to do clever things. The uses seen so far are static: the trampoline page shared by every address space and the guard pages that catch stack overflows. Page faults make the mapping dynamic, which enables lazy allocation, copy-on-write fork, demand paging, and memory-mapped files, both for better performance and efficiency and for new features.

8.1 Page faults as a mechanism

The key idea is to change the page table on a page fault. A page fault is a trap, like a system call, and causes a controlled transfer into the kernel; on RISC-V, 3 of the 16 exception causes are page faults. Xv6 gives up when one occurs (it kills the faulting user process, or panics on a fault in the kernel), but it does not have to: the kernel can instead update the page table and restart the faulting instruction, returning through `userret` as after any trap. The combination of page faults and page-table updates is powerful.

To do something useful the kernel needs three pieces of information:

1. the virtual address that caused the fault, which the hardware puts in the `stval` register;
2. the type of the fault, in `scause`: an instruction, load, or store page fault (cause codes 12, 13, and 15);
3. the instruction and mode where the fault occurred: the user program counter is saved in `sepc` and then in `p->trapframe->epc`, and the mode is implicit in whether `usertrap()` or `kerneltrap()` handles the trap.

8.2 Lazy allocation

With `sbrk()` as in xv6, allocation is eager: `sbrk(n)` allocates and maps physical memory for the whole request immediately. Applications tend to over-ask, for example allocating for the largest possible input while typically using less, so eager allocation spends memory that may never be used, and a large request can be expensive (if memory is full, the kernel must first evict pages to free some).

Modern operating systems allocate lazily. `sbrk(n)` only raises `p->sz` by n and allocates nothing. When the application touches the new memory, the access causes a page fault. If the faulting address is below `p->sz` (and above the stack), the kernel allocates one page, zeroes it, maps it in the

page table, and restarts the instruction. A process that never touches memory never faults for it, so less memory may be used, and the cost of allocation is spread over the page faults instead of being paid up front. In xv6 this touches three files: `sys_sbrk()` in `sysproc.c` only adjusts the size, `usertrap()` in `trap.c` handles the page fault, and the unmapping code in `vm.c` must tolerate pages in $[0, \texttt{p->sz})$ that were never mapped.

8.3 Zero-fill on demand

Applications often have large parts of memory that must start out zero, such as global arrays in the BSS ("block starting symbol") segment, so the kernel must often fill pages with zeros. Instead, the kernel can `memset` *one* physical page to zero and map it, read-only, at every virtual page that should be zero-filled; at `exec` all the BSS pages then point to that single page. On a write, the store faults; the kernel allocates a page, copies the zero page into it, maps it read/write in the application's address space in place of the shared page, and restarts the instruction.

8.4 Copy-on-write fork

Xv6's `fork()` copies all of the parent's pages, but fork is often immediately followed by `exec()`, which discards the copy (the shell forks a child only for the child to exec `echo`). The COPY-ON-WRITE (COW) idea is to share the address space between parent and child instead. `fork()` maps each page into both page tables read-only, marking the PTEs as COW with one of the bits reserved for supervisor software (RSW). When either process writes a COW page, the store faults; the kernel allocates a new page, copies the old one into it, maps the copy read/write in the faulting process, and restarts the instruction (Figure 11). Because a physical page may now be referenced by several page tables, the kernel keeps a reference count per physical page and frees a page only when its last reference goes away.

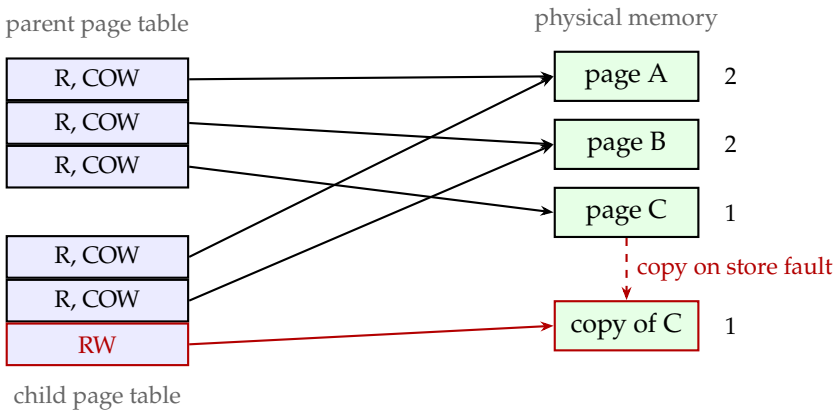


Figure 11: Copy-on-write fork. After `fork()` both page tables map the same physical pages read-only and marked COW. A store by the child to the third page faults; the kernel copies the page, maps the copy read/write in the child (red), and restarts the store. Numbers are reference counts.

8.5 Demand paging

Xv6's `exec()` loads the complete text and data segments of the program file into memory and maps them eagerly. That is expensive (the file may be on a slow disk) and may be unnecessary (the program may not use the whole file). With DEMAND PAGING, `exec` creates page-table entries but marks them on-demand (not valid), so the first instruction fetch at the entry point faults. On a fault the kernel reads the page from the file into memory, maps it, and restarts the instruction. The kernel must keep metadata about where each page lives on disk, typically in a structure called a VIRTUAL MEMORY AREA (VMA).

A remaining challenge is a file, or more generally an address space, larger than physical memory. The kernel can then store less frequently used parts of the address space on disk and page them out and in transparently. When memory is full and a fault needs a page, the kernel evicts a page (writing it back to its file if it is dirty, so non-dirty pages are cheaper victims), uses the just-freed page, and restarts the instruction. This works well when the working set fits in physical memory. The most popular replacement strategy is least recently used (LRU); the hardware helps by setting the A (accessed) bit of a PTE on each access and the D (dirty) bit on each write. On Linux, `top` reports each process's virtual size (VIRT) separately from the memory actually resident (RES).

8.6 Memory-mapped files

Memory-mapped files allow access to a file with ordinary loads and stores instead of `read()` and `write()`, which makes it easy to read and write parts of a file without moving the offset with `lseek`. UNIX systems provide the system call

```
1 void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

which maps `length` bytes of the file `fd`, starting at `offset`, into the address space with protection `prot`. The kernel records the mapping in a VMA and pages the file in lazily, on demand. When memory is full it pages out file pages that are not used frequently, and `munmap(addr, length)` writes dirty pages back to the file.

Shared virtual memory extends the idea across machines: processes on different machines share a virtual address space, which gives the illusion of physical shared memory across a network. Pages that are only read are replicated; a write invalidates the other copies.

8.7 TLB management

CPUs cache page-table translations in the TLB for speed. Xv6 flushes the entire TLB on every user/kernel transition, because each transition switches page tables and the old translations no longer apply. RISC-V allows more sophisticated schemes:

- the `PTE_G` bit marks a global translation that is the same in every address space (a candidate is a page mapped at the same address everywhere, such as the trampoline);
- TLB entries can be tagged with an address-space identifier (ASID): `satp` takes an ASID, and `sfence.vma` also takes one, so the kernel can flush selectively;
- large pages of 2 MB and 1 GB reduce the number of translations needed.

Virtual memory is still evolving. Recently, Linux adopted kernel page-table isolation (KPTI) to defend against the Meltdown side channel; xv6, with separate kernel and user page tables, basically implements KPTI already. Somewhat recent changes include support for five-level page tables (57 address bits) and for ASIDs; less recent ones include large pages and the no-execute (NX) PTE flag, `PTE_X` on RISC-V. Page tables together with traps and page faults make a powerful and elegant basis for such features.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 8 (Fall 2020).

Lecture 9: Interrupts

An interrupt occurs when hardware wants attention now: a packet has arrived, or the clock has ticked. Software must set aside its current work, process the interrupt, and then resume. On RISC-V interrupts use the same trap mechanism as system calls and exceptions, but they raise new issues:

- *asynchrony*: an interrupt stops whatever is running, and its handler may not run in the context of the process that caused it;
- *concurrency*: devices and processes run in parallel;
- *programming devices*: devices can be difficult to program.

9.1 Where interrupts come from

On the SiFive FU540 board that `qemu`'s `virt` machine resembles, the RISC-V cores are connected to two interrupt controllers (Figure 12). The PLIC (platform-level interrupt controller) collects interrupts from devices such as the UART and routes each one to a core; the CLINT (core-local interruptor) supplies each core's timer and software interrupts. The UART (universal asynchronous receiver/transmitter) carries the console: `qemu` emulates a 16550-style UART, slightly different from the one on the real board. The kernel tells which device interrupted by its IRQ (interrupt request) number, a unique source number defined by the hardware platform: UART0 is IRQ 10 on `qemu` (see `kernel/memlayout.h`), and a different number on the SiFive board.

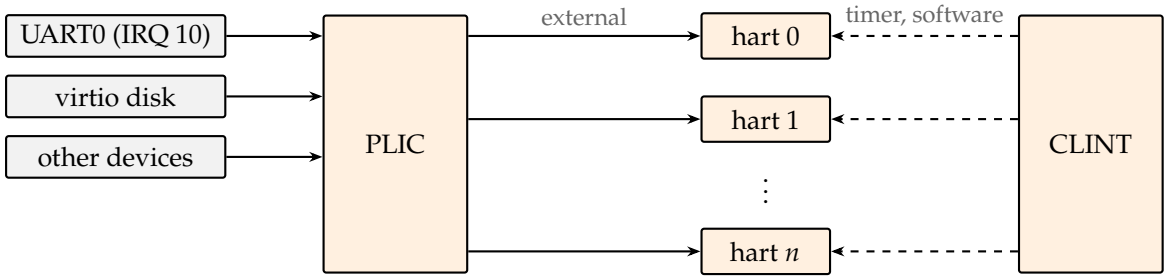


Figure 12: Interrupt routing. Device interrupts go through the PLIC, which routes each one to some core; the CLINT gives every core its timer and software interrupts.

The interrupt only tells the kernel that the device wants attention; the driver in the kernel knows how to tell the device what to do. Often the interrupt handler calls the relevant driver, though other arrangements are possible (scheduling a thread, or polling). The kernel programs a device through memory-mapped I/O: the device's control registers appear at physical addresses, and ordinary `ld` and `st` instructions read and write them. A driver typically has two halves around a buffer

(Figure 13): the TOP HALF is called through `read()` and `write()` system calls and runs in the context of the calling process, and the BOTTOM HALF is the interrupt handler.

9.2 RISC-V interrupt registers

The supervisor-mode registers involved are:

Register	Role
`sie`	interrupt enable, one bit each for external, software, and timer interrupts
`sstatus`	status; its SIE bit enables or disables interrupts as a whole
`sip`	interrupt pending
`scause`	cause of the current trap
`stvec`	trap vector: where the hardware jumps on a trap
`mideleg`	machine-mode register that delegates interrupts to supervisor mode

Xv6 sets them up in three places:

```
// start(), still in machine mode: enable external, timer, software
w_sie(r_sie() | SIE_SEIE | SIE_STIE | SIE_SSIE);

// main()
consoleinit();      // calls uartinit() to configure the UART
plicinit();         // configure the PLIC
scheduler();        // calls intr_on() to enable interrupts on this CPU:
                    //   w_sstatus(r_sstatus() | SSTATUS_SIE);
```

9.3 Console output: printing $

For `$ ls`, two interrupt-driven activities occur: the driver puts `$` into the UART, which raises an interrupt when the character has been sent; and the keyboard delivers `l` and `s` to the UART, which raises an interrupt for each received character.

The shell starts with FDs 0, 1, and 2 connected to the console, set up by `init`: UNIX presents the console device as a file. To print its prompt the shell calls `printf()`, which calls `putc()`, which calls `write(2, "$", 1)`. In the kernel the call chain is `sys_write()` → `filewrite()` → `consolewrite()` → `uartputc()`, which adds `$` to the driver's buffer and calls `uartstart()` to kick off sending: it puts the character in the UART's send FIFO and returns. The system call returns to user space while, at the same time, the UART sends the character to the console, and the shell goes on to call `read()` to wait for input.

When the UART finishes sending, it raises an interrupt, and the PLIC passes the interrupt to a CPU. The CPU then performs these steps:

1. If the trap is a device interrupt and the SIE bit is clear, do none of the following (the interrupt stays pending).
2. Disable interrupts by clearing SIE, so the handler is not itself interrupted.
3. Copy the `pc` to `sepc`.
4. Save the current mode (user or supervisor) in the SPP bit of `sstatus`.
5. Set `scause` to reflect the interrupt's cause.
6. Set the mode to supervisor.
7. Copy `stvec` to the `pc`. In user space `stvec` holds `uservec` in the trampoline, which leads to `usertrap()`; in the kernel it holds `kernelvec`.
8. Start executing at the new `pc`.

This is the same mechanism used for system calls and page faults. Both `usertrap()` and `kerneltrap()` call `devintr()` to check for device interrupts. For an external interrupt it calls `plic_claim()` to learn which device interrupted, then `uartintr()`, which sends more characters if any are waiting (possibly several), then `plic_complete()`. Returning from `kernelvec` or `usertrap()` resumes the interrupted computation.

If several interrupts arrive, the PLIC distributes them among the cores, so interrupts can be handled in parallel. If no CPU claims an interrupt, it stays pending; eventually each interrupt is delivered to some CPU.

9.4 Interrupts and concurrency

Interrupts expose three forms of concurrency.

1. *Between the device and the CPU.* This is producer/consumer parallelism.
2. *An interrupt may interrupt the CPU* while it is running, for example while it is returning to the shell, still in the kernel. Code that must be atomic disables interrupts.
3. *An interrupt may run on a different CPU* in parallel with the shell (or with its return path). This needs locks.

For printing, the shell is the producer and the device is the consumer. To decouple them the driver keeps a buffer: in xv6, `uart_tx_buf`, a circular array of 32 characters with a write index advanced by `uartputc()` and a read index advanced as the UART takes characters (Figure 13). The top half puts characters into the buffer, waiting if there is no room, and runs in the context of the calling process. The bottom half removes characters from the buffer, and the interrupt handler wakes up waiting producers; it may not run in the context of the shell. The two halves may run in parallel on different CPUs.

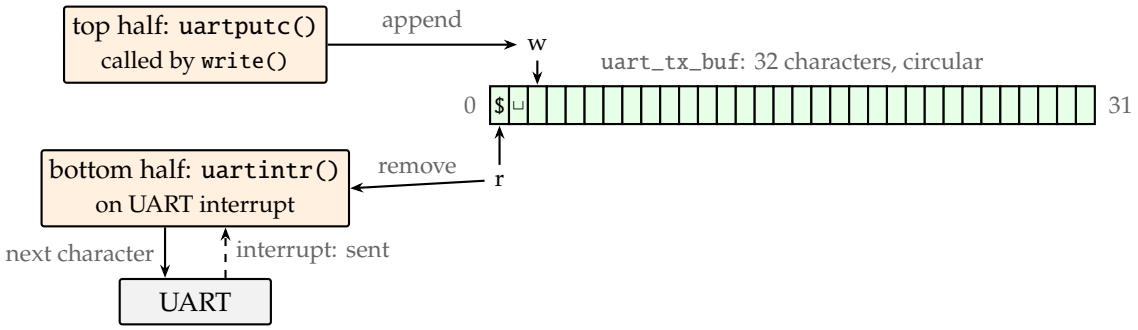


Figure 13: The UART driver's transmit buffer. The top half (`uartputc()`, called from `write()`) appends at the write index; the bottom half (`uartintr()`, on each transmit-complete interrupt) hands the character at the read index to the UART.

Interrupts run between the instructions of other code. Given

```
addi sp, sp, -48
sd   ra, 40(sp)
```

other code may run between the two instructions: a timer or UART interrupt can arrive in between. For user code that is not so bad, because the kernel resumes the user code in the same state. Kernel code is harder, since the handler may update state the interrupted code can observe. If the code sets `x = 0` and then executes `if (x == 0) f();` while an interrupt handler sets `x = 1`, then `f()` may or may not run. To make a block of code atomic with respect to interrupts, the kernel turns them off with `intr_off()`, which is `w_sstatus(r_sstatus() & ~SSTATUS_SIE)`. RISC-V itself turns interrupts off on every trap (interrupt or exception); this is why the trampoline code, which runs with the user state only partly saved, is not interrupted. This is a first glimpse of concurrency, which locks address in general.

9.5 Console input: reading 1

After printing its prompt, the shell is in the `read()` system call waiting for console input. In the kernel, `usertrap()` has set `stvec` to `kernelvec`; `consoleread()` calls `sleep()`, which switches to `scheduler()`, which turns interrupts on with `intr_on()`.

The user types `1`, which causes a UART interrupt. Since the CPU is in the kernel, the trap enters `kernelvec`, which saves the registers on the *current* stack, here the scheduler thread's stack, and calls `kerneltrap()`. That calls `devintr()`, which calls `uartintr()`; it reads the character with `c = uartgetc()` and passes it to `consoleintr(c)`. `consoleintr()` handles control characters, echoes the character to the console with `uartputc_sync()`, puts `c` in the console's buffer, and wakes up the reader. Then `kernelvec` restores the registers, and `sret` returns to wherever the interrupt happened, in this case the scheduler loop. The scheduler then runs the shell, which collects the `1`. For keyboard input the roles are reversed: the device produces and the shell consumes.

9.6 Interrupt evolution and polling

Interrupts used to be relatively fast; now they are slow. The old approach was simple hardware and smart software, with an interrupt for every event; the new approach is hardware that completes a lot of work before interrupting. Some devices generate events faster than one per microsecond: gigabit Ethernet can deliver 1.5 million small packets per second, while an interrupt costs on the order of a microsecond (saving and restoring state, cache misses).

Polling is the other way of interacting with a device: the processor spins until the device wants attention. That wastes processor cycles if the device is slow (one example in xv6 is `uartputc_sync()`), but it is inexpensive if the device is fast: there is no saving of registers, and if events are always waiting there is no need to keep alerting the software. So high-rate devices are polled, while low-rate devices such as the keyboard interrupt, since constant polling would waste the CPU. Systems can switch between the two automatically, interrupting when the rate is low and polling when it is high. A related direction is faster forwarding of interrupts to user space, for page faults and user-handled devices: hardware that delivers directly to user code without kernel intervention, or a faster forwarding path through the kernel.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 9 (Fall 2020).

Lecture 10: Multiprocessors and Locking

Applications want to use multi-core processors for parallel speedup. Microprocessor trend data show why: since about 2005 single-thread performance has plateaued and clock frequency has stayed roughly constant, while the number of cores per chip keeps growing. So the kernel must handle parallel system calls, and thus parallel access to kernel data (the buffer cache, processes, and so on). Locks help with correct sharing of data, but locks can also limit parallel speedup.

10.1 What goes wrong without locks

Delete the `acquire()` and `release()` calls in xv6's `kalloc.c`. The kernel still boots, and `usertests` passes, except that some pages are lost. The reason is a race between two cores calling `kfree()` at the same time on a shared-memory multiprocessor. `kfree()` pushes the page `r` onto the free list with

```
1 r->next = kmem.freelist;
2 kmem.freelist = r;
```

If both cores execute the first line before either executes the second, both freed pages point to the old head, and the second store to `kmem.freelist` overwrites the first, so one page is lost (Figure 14). We need locks for correctness, but they cost performance: with a lock, calls to `kfree()` are serialized.

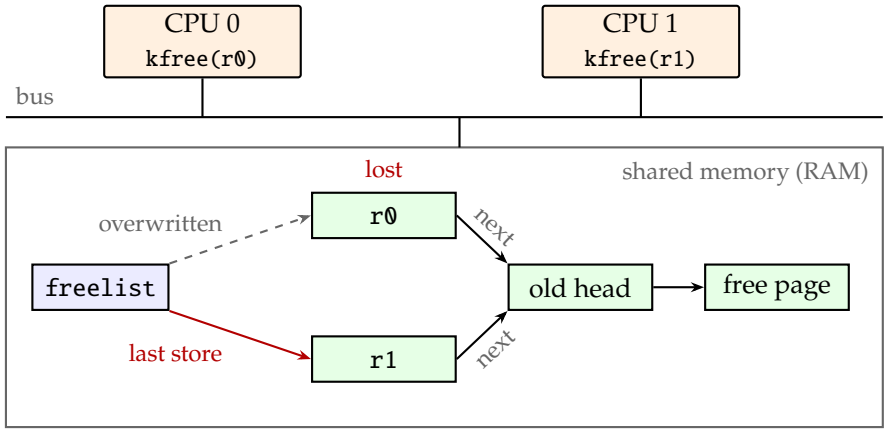


Figure 14: A race in `kfree()` without a lock. CPU 0 and CPU 1 each free a page and both set `next` to the old head of the free list; CPU 1's store to `freelist` lands last, so CPU 0's page is no longer reachable.

10.2 The lock abstraction

A LOCK is an object with two operations:

```
acquire(&l);
x = x + 1;     // critical section
release(&l);
```

If multiple threads call `acquire(l)`, only one returns right away; the others wait ("block") until `release()`. The code between is the CRITICAL SECTION. A program typically has lots of data and lots of locks; threads that use different data likely hold different locks and can execute in parallel. Note that lock `l` is not tied to the data `x` in any way: the programmer has a plan for the correspondence.

A conservative rule decides when a lock is needed: any time two threads use a memory location and at least one of them writes it, don't touch the shared data unless you hold the right lock. The rule is too strict in some cases (program logic may rule out sharing, and lock-free programming avoids locks) and too loose in others (`printf()` output from several threads; not every lock corresponds simply to some data).

Could locking be automatic, with the language associating a lock with every data object and the compiler adding `acquire` and `release` around every use? That leaves less room for the programmer to forget, but it is often too rigid. Consider `rename("d1/x", "d2/y")` done as "lock d1, erase x, unlock d1; lock d2, add y, unlock d2". For a while the file does not exist at all. `rename()` should be atomic: other system calls should see the state before or after it, never in between, or programs become too hard to write. What is needed is "lock d1; lock d2; erase x; add y; unlock d2; unlock d1". The programmer often needs explicit control over the region of code during which a lock is held, to hide awkward intermediate states.

There are three ways to think about what locks achieve:

1. locks help avoid lost updates;
2. locks help create atomic multi-step operations, hiding intermediate states;
3. locks help operations maintain invariants on a data structure: an operation assumes the invariants hold at its start, uses locks to hide their temporary violation, and restores them before releasing the locks.

10.3 Deadlock and modularity

`rename()` holds two locks, which invites DEADLOCK:

Core A	Core B
`rename("d1/x", "d2/y")`	`rename("d2/a", "d1/b")`
lock d1	lock d2
lock d2 . . .	lock d1 . . .

Each core holds one lock and waits forever for the other. (A single thread that acquires a lock it already holds deadlocks too.) The solution is for the programmer to work out a global order for all locks and for all code to acquire locks in that order: predict the locks needed, sort them, and acquire them. This is complex.

Locks also make it hard to hide details inside modules. If a function in module M_1 calls a function in M_2, then to avoid deadlock M_1 must know which locks M_2 acquires, and it may need to acquire them before the call even if it does not use them. Locks are often not the private business of individual modules.

10.4 Locks and parallelism

Locks *prevent* parallel execution. To get parallelism one often has to split up data and locks so that each core uses different data and different locks: FINE-GRAINED LOCKS. Choosing the best split is a design challenge (one lock for the whole file system, per directory or file, or per disk block; for the whole kernel, per subsystem, or per object), and it may require redesigning code to work well in parallel, for example breaking the single free-memory list into per-core free lists if threads wait a lot on its lock. Such rewrites can be a lot of work. Hence the advice: start with big locks, for example one lock protecting an entire module, which gives less opportunity for deadlock and requires less reasoning about invariants and atomicity; measure whether there is a problem (big locks are often enough, since little time may be spent in that module); and redesign for fine-grained locking only if you have to.

10.5 Case study: the UART driver

The xv6 UART driver `uart.c` is typical of many operating systems' device-driver arrangements. Processes call `uartputc()`, which appends characters to `uart_tx_buf`, and `uartintr()`, the interrupt handler, removes characters from the buffer and hands them to the UART hardware for display. The sources of concurrency are processes and the interrupt. The driver has a single, fairly coarse-grained lock, `uart_tx_lock`, which protects three things:

1. there are no races in operations on `uart_tx_buf`;

2. if the queue is not empty, the UART hardware is executing (sending) the head of the queue, the character at the read end;
3. there is no concurrent access to the UART's write registers; they have one writer at a time.

`uartintr()` acquires the lock too, so it may have to wait at interrupt level; it removes a character from `uart_tx_buf`, hands the next queued character to the UART (property 2), and touches the UART registers (property 3).

10.6 Implementing a spin lock

A naive implementation does not work:

```
struct lock { int locked; };

void broken_acquire(struct lock *l) {
  while(1){
    if(l->locked == 0){   // A
      l->locked = 1;      // B
      return;
    }
  }
}
```

Two CPUs can both execute line A, both see `locked == 0`, and both execute line B and return holding the lock. Lines A and B must execute atomically. Hardware provides this with an atomic swap instruction:

```
li  a5, 1
# s1 = &lk->locked
amoswap.w.aq a5, a5, (s1)
```

In hardware, `amoswap addr, r1, r2` locks the address globally (other cores cannot use it), sets *tmp* ← `*addr`, `*addr` ← `r1`, `r2` ← *tmp*, and unlocks the address. RISC-V hardware thus provides a notion of locking a memory location; different CPUs have implemented it differently, at the granularity of a cache line or of the entire bus, depending on the memory system. The problem is pushed down to the hardware, whose memory lock forces concurrent swaps to run one at a time rather than interleaved.

Xv6's spin lock uses the swap through a compiler built-in:

```
void acquire(struct spinlock *lk) {
  push_off();   // disable interrupts
  while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
    ;
  __sync_synchronize();
  ...
}
```

If `lk->locked` is already 1, `__sync_lock_test_and_set` sets it to 1 again and returns 1, and the loop continues. If it is 0, at most one test-and-set sees the 0: it sets the lock to 1 and returns 0, and every other one returns 1. Waiting cores "spin" in the loop, hence the name SPIN LOCK. `acquire()` first disables interrupts with `push_off()`: otherwise an interrupt handler on the same CPU (such as `uartintr()`) could try to acquire a lock that the interrupted code holds, and spin forever. `release()` sets `lk->locked` to 0 and re-enables interrupts.

10.7 Memory ordering

Suppose two cores use a lock, with a naive implementation, to guard a counter `x`:

Core A	Core B
`locked = 1`	
`x = x + 1`	`while(locked == 1) ...`
`locked = 0`	`locked = 1`
	`x = x + 1`
	`locked = 0`

Executed serially this is fine, but the compiler *and* the CPU reorder memory accesses: they do not obey the source program's order of memory references. The compiler might generate `locked = 1; locked = 0; x = x + 1` for core A, moving the increment outside the critical section. The legal behaviors are called the MEMORY MODEL. `release()`'s call to `__sync_synchronize()` prevents such reordering: the compiler will not move a memory reference past it, and it may emit a memory-barrier instruction telling the CPU not to either. `acquire()`'s call has a similar effect. Code that uses locks does not need to understand the memory-ordering rules; they matter only for exotic lock-free code.

10.8 Spin locks and blocking locks

Spin locks waste CPU while waiting. Why not give up the CPU and let another process run, especially when the thread holding the lock needs to run? The guidelines for spin locks are to hold them for very short times and never to yield the CPU while holding one. For longer critical

sections, systems provide BLOCKING LOCKS, whose waiting threads yield the CPU, at typically higher overhead.

In summary, locks are good for correctness but can be bad for performance, and they complicate programming. Don't share if you don't have to; start with a few coarse-grained locks; instrument the code to find which locks prevent parallelism; use fine-grained locks only as needed for parallel performance; and use an automated race detector.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 10 (Fall 2020).

Lecture 11: Thread Switching

Systems support multiple tasks for time-sharing (many users or many running programs), for program structure (for example a prime-number sieve built from cooperating processes), and for parallel speedup on multi-core hardware. Threads are an abstraction that simplifies programming with many tasks. A thread is an independent serial execution, with its own registers, program counter, and stack. A threading system interleaves the execution of multiple threads in two ways: multiple CPUs each run a different thread, and each CPU "switches" among threads, running one at a time.

Threads may or may not share memory. Xv6's kernel threads share kernel memory, and therefore need locks; xv6's user processes have one thread each, so they share nothing; Linux supports multiple threads sharing a user process's memory. There are other techniques for interleaving many tasks, such as event-driven programming and state machines; threads are not the most efficient, but they are usually the most convenient.

Thread design raises three challenges: how to interleave many threads on a few CPUs, transparently to the threads (scheduling is the process of choosing which thread to run next); what to save while a thread is not running; and how to cope with compute-bound threads.

For compute-bound threads, each CPU has timer hardware that interrupts periodically, and the kernel uses timer interrupts to grab control from looping threads. It saves the thread's state, switches to another thread, and eventually resumes the first by restoring the saved state, so the switch is transparent. This forced yield of unaware code is pre-emptive scheduling, as opposed to cooperative scheduling, in which code yields voluntarily. A pre-empted thread goes from RUNNING to RUNNABLE.

A thread that is not running must have its state set aside: its registers and its stack (its memory stays where it is). So the implementation gives each thread a stack and a register save area, and tracks each thread's status: RUNNING, RUNNABLE, or SLEEPING.

11.1 Threads in xv6

Each xv6 process has two threads: a user thread with a user stack, and a kernel thread with a kernel stack. A process is executing *either* its user thread *or*, inside a system call or interrupt, its kernel thread. The kernel threads share kernel memory and data structures, so the kernel is a parallel program. Below, "process", "kernel thread", and "thread" are used as synonyms.

Getting from one process to another involves several transitions (Figure 15). The trapframe (TF) holds saved user registers; the context (CTX) holds saved kernel-thread registers.

1. user → kernel: the trap saves the user registers in the trapframe;
2. kernel thread → scheduler thread: `swtch()` saves the kernel thread's registers in its context;
3. scheduler thread → another kernel thread: `swtch()` restores that thread's registers from its context;
4. kernel → user: the return from the trap restores the user registers from the trapframe.

A CONTEXT SWITCH is the switch from one thread to another.

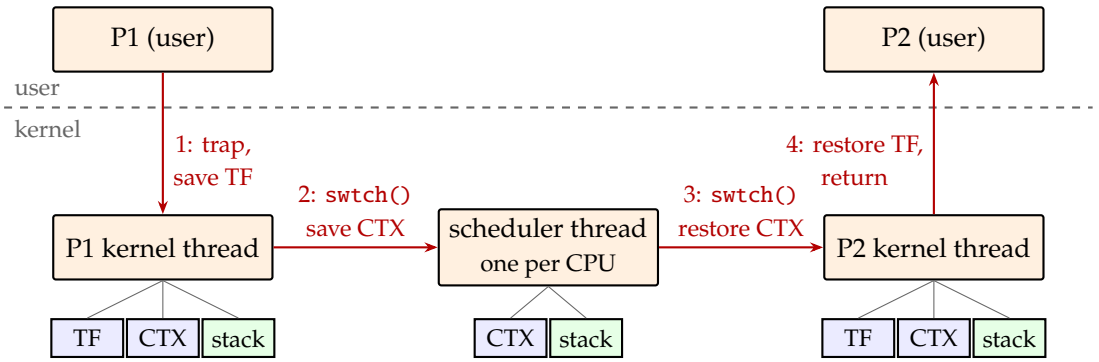


Figure 15: Switching a CPU from process P1 to process P2. The trap saves P1's user registers in its trapframe (TF); `swtch()` saves P1's kernel registers in its context (CTX) and switches to this CPU's scheduler thread, which later uses `swtch()` to restore P2's kernel thread; P2 then returns to user space, restoring its user registers from its trapframe.

There is one SCHEDULER THREAD per core, each with its own stack and `struct context`. Kernel threads always switch to the current core's scheduler thread, which switches to another kernel thread if one is RUNNABLE; there are never direct switches from one kernel thread to another. The scheduler's separate stack simplifies cases such as switching away from an exiting process. The scheduler thread keeps scanning the process table until it finds a RUNNABLE thread; there may not be one, in which case the scheduler is idle. At any moment each core is running either its scheduler thread or some other thread, and runs only one thread at a time; each thread is either running on exactly one core or has its registers saved in its context; and the saved context of a thread that is not running refers to a call to `swtch()`.

The relevant fields of `struct proc` (in `proc.h`) are `p->trapframe` (the saved user registers), `p->context` (the saved kernel-thread registers), `p->kstack` (the kernel stack), `p->state` (RUNNING, RUNNABLE, SLEEPING, . . .), and `p->lock`, which protects `p->state` and other fields.

11.2 A pre-emptive switch, step by step

The program `spin.c` runs two CPU-bound processes on a single-CPU `qemu`. Their output alternates even though both run continuously, because xv6 switches its one CPU between them.

A breakpoint at the timer-interrupt case of `usertrap()` shows a device interrupt from the timer

(`timerinit()` in `kernel/start.c` configures the RISC-V timer hardware). The current process is one of the `spin` processes, and its saved `p->trapframe->epc` lies in the increment loop of `spin`: the timer interrupted user code, as expected. `usertrap()` calls `yield()`, which acquires `p->lock` and changes `p->state` from RUNNING to RUNNABLE: the process gives up the CPU but wants to run again. The lock is needed because `yield()` modifies `p->state`, and to prevent another CPU from running this RUNNABLE thread too early. `yield()` calls `sched()`, which makes some sanity checks and then calls `swtch(&p->context, &mycpu()->context)`: a context switch from the process's kernel thread to the scheduler thread. `swtch()` saves the current registers in its first argument and restores the registers previously saved in its second; the saved `ra` in `cpus[0].context` points into `scheduler()` in `proc.c`.

```
# void swtch(struct context *old, struct context *new);
swtch:
  sd ra, 0(a0)       # save into old (a0)
  sd sp, 8(a0)
  sd s0, 16(a0)
  ...                # s1 through s11
  ld ra, 0(a1)       # restore from new (a1)
  ld sp, 8(a1)
  ld s0, 16(a1)
  ...                # s1 through s11
  ret
```

`swtch()` neither saves nor restores the program counter. It saves `ra`, the address it was called from, and its final `ret` jumps to the `ra` it restored, so the target thread resumes just after *its* earlier call to `swtch()`. And it saves only 14 registers (`ra`, `sp`, `s0`–`s11`), not all 32. All 32 user registers are already in the trapframe; what `swtch()` saves are kernel-thread registers, and since `swtch()` is called like an ordinary C function, the calling convention lets it ignore the caller-saved registers (`t0`–`t6`, `a0`–`a7`), which the compiler has already saved where needed; `zero` never changes, and `tp` holds the CPU's hart ID, which belongs to the CPU rather than the thread. At the start of `swtch`, `$ra` points into `sched()`; just before its `ret`, `$ra` points into `scheduler()` and `$sp` into `stack0`, the stack that `entry.S` set up at boot.

After the `ret` we are in `scheduler()`, in the scheduler thread on the scheduler's stack. `scheduler()` has just returned from a call to `swtch()` that it made a while ago to switch to the spin process's kernel thread. That call saved `scheduler()`'s registers, and the process's own call to `swtch()` has now restored them; the variable `p` refers to the interrupted process. The scheduler now releases `p->lock`, which `yield()` acquired. The code of `scheduler()` *looks* like an ordinary acquire/release pair, but in fact the scheduler acquires and `yield()` releases, then `yield()` acquires and the scheduler releases: unusually, the lock is released by a different thread than the one that acquired it.

Why hold `p->lock` across `swtch()` instead of releasing it just before? Because it makes two groups of steps atomic.

- Setting `p->state = RUNNABLE`, saving the registers in `p->context`, and ceasing to use `p`'s kernel stack: another CPU's scheduler must not start running `p` until all three are done.
- Setting `p->state = RUNNING` and moving the registers from the context into the RISC-V registers: an interrupt in between must not `yield()` and save not-yet-initialized registers into the context.

The scheduler's loop looks at all processes in turn for one that is RUNNABLE, looping (possibly idle for a while) until it finds one; here it finds the other `spin` process. It locks the process and sets its state to RUNNING, so that no other CPU's scheduler will run it, and calls `swtch(&c->context, &p->context)`. The new thread's saved `p->context.ra` points into `sched()`. So when this `swtch()` reaches its `ret`, execution continues in `sched()`, in a timer interrupt of the *other* spin process: that process was interrupted in the past and called `yield()`, `sched()`, and `swtch()`, and now it resumes and returns to user space.

Only `swtch()` writes contexts (apart from initialization), and only `sched()` and `scheduler()` call `swtch()`. So a kernel thread's `context.ra` always points into `sched()`, and a scheduler thread's always points into `scheduler()`. When `sched()` calls `swtch()`, `swtch()` returns to `sched()`, but typically in a *different* thread. `sched()` and `scheduler()` are CO-ROUTINES: each knows what it is switching to and where the return from `swtch()` comes from, and they cooperate about `p->lock` and `p->state`. In ordinary thread switching, by contrast, neither party typically knows which thread comes before or after.

11.3 Questions

What is the scheduling policy? When several threads are RUNNABLE, each CPU's scheduler runs the next RUNNABLE process it finds in its scan of the process table, so the processes take turns (round robin). Whether this is a good policy is worth pondering.

Is there pre-emptive scheduling of kernel threads? Yes: a timer interrupt can occur while in the kernel, and `kerneltrap()` in `kernel/trap.c` then calls `yield()`. The registers of the interrupted kernel code cannot be saved in `p->trapframe`, which already holds the user registers, nor in `p->context`, since the thread is about to call `yield()` and `swtch()`; instead `kernelvec.S` pushes them on the kernel stack, since the thread is already in the kernel. Pre-emption in the kernel is not critical in xv6, but it is valuable if some system calls do a lot of computation or if a strict notion of thread priority is needed.

Why does `scheduler()` briefly enable interrupts with `intr_on()`? There may be no RUNNABLE threads, because all are waiting for I/O (disk or console, say). Enabling interrupts gives a device the chance to signal completion and so wake up a thread; otherwise the system would freeze.

Why does `sched()` forbid holding locks other than `p->lock` when yielding the CPU (it checks that `noff == 1`)? Suppose process P1 holds lock L1 and yields the CPU, and process P2 then runs and

tries to acquire L1. P2's `acquire()` spins with interrupts turned off, so no timer interrupt occurs and P2 never yields the CPU, so P1 never runs and never releases L1.

Could the per-CPU scheduler thread be eliminated, with `sched()` itself looking for the next process and switching directly to it? That would be faster, saving one of the `swtch()` calls. It can be done, but the scheduling loop would then run on a thread's kernel stack, which raises problems: what if that thread is exiting, what if another CPU wants to run the thread, and what if there are fewer threads than CPUs, so that there are too few stacks?

In summary, xv6 provides a convenient thread model for kernel code. It is pre-emptive, via timer interrupts, and transparent, via switching registers and stacks. Multi-core operation requires careful handling of stacks and locks.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 11 (Fall 2020).

Lecture 13: Sleep and Wakeup

13.1 Thread switching and p->lock

In xv6, `yield()` acquires `p->lock`, sets `p->state = RUNNABLE`, and calls `swtch()` to the core's scheduler thread, which runs on its own stack and releases `p->lock` when its `swtch()` returns:

```
1 // yield(), on the thread's kernel stack
2 acquire(&p->lock);
3 p->state = RUNNABLE;
4 swtch(...);            // to the scheduler thread
5 // scheduler(), on the scheduler's stack
6 swtch(...);            // returns here once the thread has switched away
7 release(&p->lock);
```

The lock is held across `swtch()` so that no other core's scheduler can see `p->state == RUNNABLE` and start running the thread until the original core has stopped executing it and has stopped using its kernel stack. Otherwise two cores could run on one stack.

`sched()` also checks that no spinlock other than `p->lock` is held when a thread gives up the CPU (it requires `noff == 1`). On a single core, suppose P_1 acquires a lock L and calls `sched()`, and then P_2 tries to acquire L. P_2 spins until P_1 releases L and never yields the CPU (acquiring a spinlock disables interrupts, so not even the timer can switch it away), while P_1 cannot release L until it runs again: deadlock. With several cores the same deadlock can arise once more spinlocks are involved. The rule is therefore never to hold a spinlock while yielding the CPU.

13.2 Sequence coordination

Threads often wait for a specific event or condition: a disk read to complete (signaled by an interrupt), a pipe writer to produce data (signaled by another thread), or any child to exit. Coordination is a basic building block of thread programming. It is usually easy to use but, like locking, follows rules that sometimes produce hard puzzles.

A reader could spin in a loop such as `while(buffer is empty) {}` until a writer puts data in the buffer, but that wastes a CPU that other threads could use. Coordination primitives instead let the waiting thread yield the CPU; examples are barriers, semaphores, and event queues. xv6 uses sleep and wakeup.

The console's UART driver illustrates the idea (in a modified form of xv6's `uartwrite()` and `uartintr()`). The UART accepts only one byte (really a few) of output at a time and takes a long

time, perhaps a millisecond, to send each one; it interrupts after it has sent each character. A process that writes to the console must wait until the previous character has gone out, and it should give up the CPU until the interrupt arrives. So `write()` calls `uartwrite()`, which sends a byte if it can and otherwise calls `sleep()` to wait for the interrupt; the interrupt handler `uartintr()` calls `wakeup()`. The address `&tx_chan` passed to both links the sleep with the wakeup. The scheme is simple and flexible: sleep and wakeup need not understand what is being waited for, and no explicit coordination objects have to be allocated.

13.3 Lost wakeups

One would like `sleep()` to mean just "wait for this event," but it cannot be designed that cleanly because of a problem called LOST WAKEUPS, which lurks behind every sequence coordination scheme. Consider a sleep that takes only a channel:

- `broken_sleep(chan)` sleeps on a CHANNEL, a number or address that identifies the condition or event being waited for. It acquires `p->lock`, sets `p->chan = chan` and `p->state = SLEEPING`, and calls `sched()`.
- `wakeup(chan)` wakes every thread sleeping on `chan` (possibly more than one): for each process `p`, if `p->state == SLEEPING` and `p->chan == chan`, it sets `p->state = RUNNABLE`.

The UART code would loop `while(not done) sleep(&done)` before sending each character and then clear `done`, and the interrupt would set `done = true` and call `wakeup(&done)`. Here `done` is the condition being waited for, and `&done` is merely the channel.

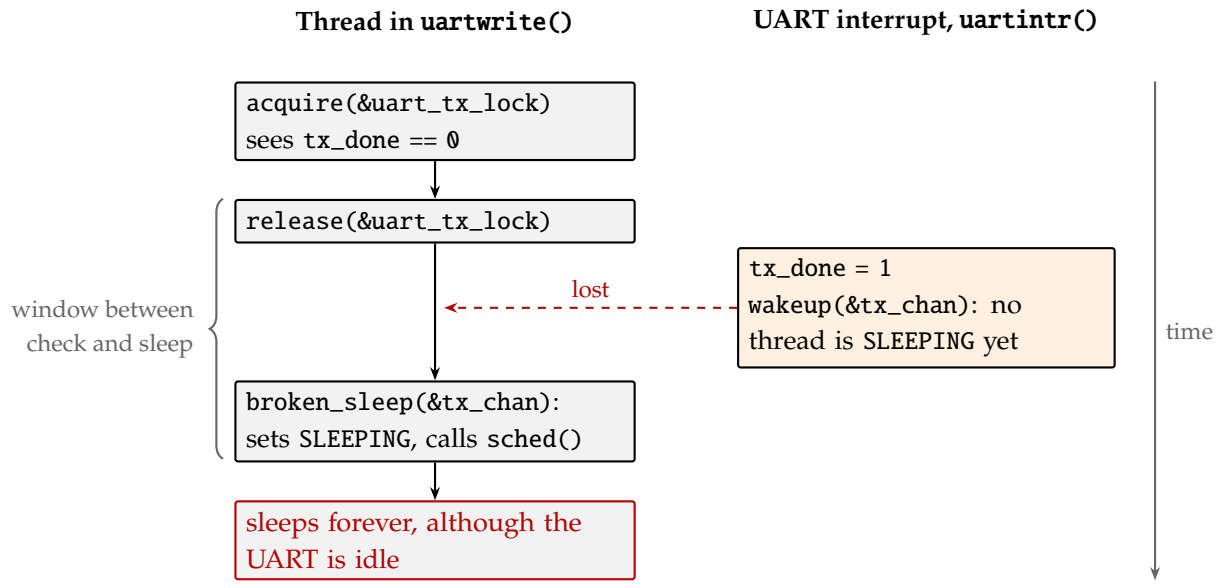


Figure 16: A lost wakeup. The interrupt's `wakeup()` runs in the window after `uartwrite()` has checked the condition and released the lock but before it is marked `SLEEPING`.

Both `uartwrite()` and `uartintr()` must lock the driver's data (such as `done`) and the UART hardware. `uartwrite()` cannot hold the lock for its whole sequence, since then `uartintr()` could never acquire it to set `done`. So suppose `uartwrite()` releases the lock just before sleeping:

```
release(&uart_tx_lock);
broken_sleep(&tx_chan);
acquire(&uart_tx_lock);
```

This fails (Figure 16). `uartwrite()` sees that the previous character is not yet sent and releases the lock; the interrupt occurs after `release()` but before `broken_sleep()`, and its `wakeup()` finds no sleeping thread. `uartwrite()` then goes to sleep even though transmission is done, and nothing will ever wake it.

The window between checking the condition and being marked asleep must be closed, using a lock to keep `wakeup()` from running anywhere inside it. xv6 therefore changes the interface: there must be a CONDITION LOCK that protects the condition, and the callers of both `sleep()` and `wakeup()` must hold it. `sleep(chan, lock)` requires the caller to hold `lock`, releases it while sleeping, and re-acquires it before returning; the caller of `wakeup(chan)` also holds the lock. The repaired UART driver:

```
static int tx_done; // has the UART finished sending?
static int tx_chan; // &tx_chan is the "wait channel"

void
uartwrite(char buf[], int n)
{
  acquire(&uart_tx_lock);
  int i = 0;
  while(i < n){
    while(tx_done == 0){
      // UART is busy sending a character; wait for it to interrupt.
      sleep(&tx_chan, &uart_tx_lock);
    }
    WriteReg(THR, buf[i]);
    i += 1;
    tx_done = 0;
  }
  release(&uart_tx_lock);
}

void
uartintr(void)
{
  acquire(&uart_tx_lock);
  if(ReadReg(LSR) & LSR_TX_IDLE){
```

```
    // UART finished transmitting; wake up any sending thread.
    tx_done = 1;
    wakeup(&tx_chan);
  }
  release(&uart_tx_lock);
  // ... then read and process any incoming characters
}
```

13.4 How sleep() prevents lost wakeups

`wakeup(chan)` in `proc.c` scans the process table, acquiring each `p->lock` while it checks for `SLEEPING` and a matching channel. Its caller already holds the condition lock, so `wakeup()` holds *both* the condition lock and the `p->lock` of the process it examines.

`sleep()` must release the condition lock, since no lock other than `p->lock` may be held across `swtch()`. To keep `wakeup()` from running after that release, it acquires `p->lock` *before* releasing the condition lock. Because `wakeup()` needs both locks, it is enough for `sleep()` to hold either one to force `wakeup()` to spin rather than look at this process. `p->lock` is released only by the scheduler after `swtch()` completes, so `wakeup()` is guaranteed to find `p->state == SLEEPING` and `p->chan == chan`: no wakeup is lost. In simplified form:

```
void
sleep(void *chan, struct spinlock *lk)
{
  struct proc *p = myproc();

  acquire(&p->lock);  // hold p->lock before giving up the condition lock,
  release(lk);        // so wakeup() cannot run in between
  p->chan = chan;
  p->state = SLEEPING;
  sched();
  p->chan = 0;
  release(&p->lock);
  acquire(lk);        // re-acquire the condition lock before returning
}
```

`uartwrite()` wraps `sleep()` in a loop that re-checks the condition after `sleep()` returns and may sleep again. There are two reasons: with multiple waiters, another thread may have consumed the event first, and `kill()` wakes processes even when the condition is not true. All uses of `sleep()` are wrapped in such a loop.

Pipes follow the same pattern. In `piperead()` the condition is that data is waiting (`nread != nwrite`), and `pipewrite()` calls `wakeup()` after adding data. With `broken_sleep()` the same race would appear: a reader could find the pipe empty and release the pipe's lock, a writer could then

add data and call `wakeup()`, and only afterwards would the reader go to sleep. The loop around `sleep()` matters because several processes may read the same pipe. `piperead()` also ends with a `wakeup()`, because reading frees buffer space for writers that are sleeping until the pipe is no longer full.

The sleep/wakeup rules are a little complex: `sleep()` need not understand the condition, but it does need the condition lock. The interface is flexible though low level. Other schemes, such as counting semaphores, are cleaner but perhaps less general, and all of them must cope with lost wakeups one way or another.

13.5 Terminating a thread

Termination is another coordination puzzle, because it frees resources that might still be in use. One thread cannot simply destroy another: the victim might be executing on another core, might hold locks, or might be in the middle of a complex update to important data structures. Nor can a thread free all of its own resources, such as the stack it is running on or the `struct context` it needs in order to call `swtch()`. xv6 gets rid of processes in two ways, `exit()` and `kill()`.

In the ordinary case a process quits voluntarily with the `exit()` system call, and the freeing is split between `exit()` and the parent's `wait()`. `exit()` closes the process's open files, makes `init` (PID 1) the parent of its children, wakes up its parent if it is waiting, sets `p->state = ZOMBIE`, and switches to the scheduler. A ZOMBIE is dying but not yet dead: it will not run again and will not yet be reallocated by `fork()`, and its kernel stack and `proc[]` entry are still allocated. The parent (or `init`) eventually calls `wait()`, which sleeps until some child exits, scans `proc[]` for children in state `ZOMBIE`, and calls `freeproc()` on one with its `p->lock` held; this frees the trapframe, the page table, and so on, and sets the state to `UNUSED`. So `wait()` is needed by the kernel, not only by applications: every process must be waited for, which is why an exiting process's children are re-parented. Some complexity comes from a child exiting concurrently with its own parent, and from locking parent before child to avoid deadlock.

13.6 Killing a process

`kill(pid)` cannot safely terminate its target directly. The target might be executing in the kernel, using its kernel stack, page table, `proc[]` entry, and trapframe, or it might hold locks, for example in the middle of `fork()`, and must finish to restore invariants. So `kill()` only sets the flag `p->killed`. The target checks the flag itself and calls `exit()`: `usertrap()` contains `if(p->killed) exit(-1);` at points where no locks are held, so exiting there is safe.

A target that is in `sleep()` holds no locks and is not executing, but destroying it at once is still not always safe. It might be fine if it is waiting for console input, but not if it is waiting for the disk midway through creating a file. xv6's `kill()` therefore changes a `SLEEPING` target to `RUNNABLE`, as `wakeup()` does, so its `sleep()` returns, probably before the condition is true. Some sleep loops

check `p->killed`, for example in `piperead()` and `consoleread()`; otherwise a read could hang indefinitely for a killed process. Others do not, for example in `virtio_disk.c`, which is acceptable because disk reads are quick. A killed process may thus continue for a while, but `usertrap()` exits it once the system call finishes.

Note. xv6's specification of `kill`: a target in user space dies the next time it makes a system call or takes a timer interrupt; a target in the kernel will never execute another user instruction, but may spend quite a while longer in the kernel.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 13 (Fall 2020).

Lecture 14: File Systems

File systems are useful because they give *durability* across restarts, *naming and organization* through user-friendly pathnames, and *sharing* of files among programs and users. They are interesting to design because of crash recovery, performance and concurrency (storage devices are slow, which motivates a buffer cache and concurrent operations), sharing, security, and disk layout. The file abstraction itself is useful well beyond disks: pipes, devices, `/proc`, `/afs`, and Plan 9 expose objects as files, so file-oriented applications work with many kinds of objects.

14.1 The file system API

An example in the UNIX/POSIX/Linux/xv6 API:

```
fd = open("x/y", -);
write(fd, "abc", 3);
link("x/y", "x/z");
unlink("x/y");
write(fd, "def", 3);
close(fd);
// file x/z contains abcdef
```

Pathnames are human readable, the write offset is implicit (the file descriptor maintains it), `link` gives a file multiple names, and `fd` refers to an underlying object. The API reflects a set of high-level choices, and other file system APIs, sometimes quite different, make other ones:

Aspect	UNIX choice	Alternatives
objects	files	virtual disk, database
content	byte array	80-byte records, B-tree
naming	human-readable names	object IDs
organization	name hierarchy	
synchronization	none	locking, versions

With no synchronization, `link()` and `unlink()` can change the name hierarchy concurrently with an `open()`.

The API has implications for the implementation. A file descriptor refers to something that is preserved even if the file's name changes, or if the file is deleted while open. A file can have multiple links, that is, occur in multiple directories, and no one occurrence is special, so the file's information must be stored somewhere other than in a directory. Hence the file system records each file's information in an on-disk INODE, independent of any name, and refers to it by an I-NUMBER (an

internal analogue of a file descriptor). The inode needs a link count, to know when to free the file, and a count of open file descriptors; deallocation is deferred until both are zero.

14.2 Software layers and storage devices

xv6's file system is built in layers, from top to bottom: system calls; name and file descriptor operations; inodes (reading and writing file content); the inode cache (synchronization); the log (crash recovery); the buffer cache; and the `virtio_disk` driver. Everything above the driver lives in memory; the disk is the persistent device below.

Data lives on a persistent medium that keeps it without power, attached to the CPU and memory through a bus such as PCIe. The two common media are hard disk drives (big, slow, inexpensive) and solid state drives (smaller, fast, more expensive):

	HDD	SSD
random access	5–10 ms	100 μs
sequential	100 MB/s	500 MB/s

A hard disk has concentric tracks, each a sequence of sectors with an error-correcting code; the head must seek and the platter must rotate, so random access is slow. It can only read or write whole sectors, so writing part of a sector requires a read-modify-write. An SSD stores data in non-volatile flash and is internally complex in ways that are hidden except sometimes in performance: flash must be erased before it is rewritten and each flash block tolerates a limited number of writes, so the SSD remaps blocks through a level of indirection. For both kinds, sequential access is much faster than random access and large transfers are faster than small ones; both facts shape file system design.

Disks were historically read and written in 512-byte SECTORS. Most operating systems use BLOCKS of several sectors (for example 4 KB blocks of 8 sectors) to reduce bookkeeping and seek overheads; xv6 uses 2-sector blocks, so `BSIZE` = 1024 bytes.

14.3 On-disk layout

xv6 treats the disk as an array of blocks, ignoring its physical properties. Its `mkfs` program writes the layout of Figure 17 for an empty file system, and the layout stays fixed for the file system's lifetime: block 0 is unused, block 1 is the SUPERBLOCK (file system size, number of inodes), blocks 2–31 hold the log for transactions, blocks 32–44 hold the array of inodes packed into blocks, block 45 is the bitmap of blocks in use (0 free, 1 used), and blocks from 46 to the end hold file and directory content. Everything on disk other than file content (the superblock, inodes, bitmap, and directory content) is METADATA.

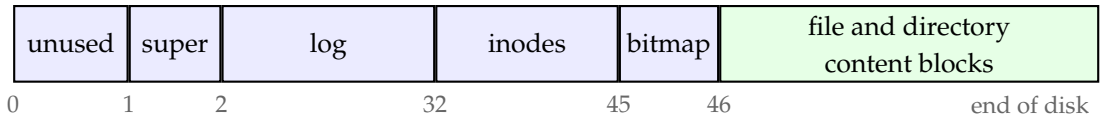


Figure 17: xv6's on-disk layout; numbers are the first block of each region.

14.4 Inodes and directories

An on-disk inode is 64 bytes and holds the file's `type` (free, file, directory, or device), `nlink`, `size`, and `addrs[12+1]`: twelve DIRECT block numbers and one INDIRECT block number, which names a block holding $1024/4 = 256$ further block numbers (Figure 18). The maximum file size is therefore $(12 + 256) \times 1024$ bytes, or 268 KB. To find byte 8000 of a file, compute the logical block $\lfloor 8000/1024 \rfloor = 7$, look up `addrs[7]`, and read offset $8000 - 7 \cdot 1024 = 832$ within that block.

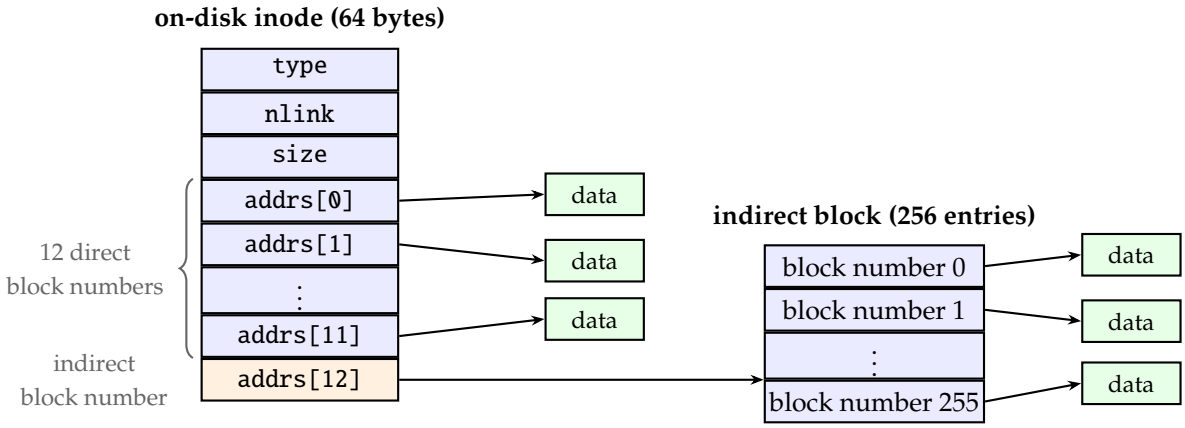


Figure 18: An xv6 on-disk inode with its direct and indirect block numbers.

Turning an i-number into an inode is easy: inode i starts at byte $32 \cdot \texttt{BSIZE} + 64\,i$ of the disk, that is, in block $32 + \lfloor 64\,i/1024 \rfloor$, with 16 inodes per block.

A DIRECTORY is much like a file, except that users cannot write it directly. Its content is an array of directory entries (dirents), each a 2-byte i-number and a 14-byte name; an entry is free if its i-number is zero. Looking up `/y/x` starts at the root directory's inode (i-number 1), scans its blocks for an entry named `y` (say with i-number 251), reads inode 251, and scans its blocks for `x`.

A file system is thus best viewed as an on-disk data structure: a tree of directories, inodes, and blocks, with two allocation pools, inodes and blocks. xv6's is a very simple one.

14.5 Creating, writing, and deleting a file

Instrumenting xv6 to print each disk write (on a fresh file system) shows how operations update these structures. Table 2 lists the writes for `echo hi > x` followed by `rm x`.

Block	Written by	Purpose
create `x` *(*`open` *with create and truncate)*		
33	`ialloc`	allocate an inode for `x` (set its type)
33	`iupdate`	initialize the inode (e.g. set `nlink` to 1)
46	`writei`	add the entry `x` to the root directory's content (`dirlink`)
32	`iupdate`	update the root directory's inode, whose size may have changed
33	`iupdate`	`itrunc` of the new inode, though nothing changed
write `"hi"`		
45	`balloc`	mark a free block allocated in the bitmap
595	`bzero`	zero the newly allocated block
595	`writei`	write `hi` into it
33	`iupdate`	record the new size and `addrs[0]` in `x`'s inode
write `"\n"`		
595	`writei`	write the newline
33	`iupdate`	record the new size
rm x		
46	`writei`	clear `x`'s directory entry (`sys_unlink`)
32	`iupdate`	update the directory's inode after writing its content
33	`iupdate`	decrement `x`'s link count (`sys_unlink`)
45	`bfree`	free the data block in the bitmap (`itrunc`, from `iput`)
33	`iupdate`	record the zeroed length (`itrunc`)
33	`iupdate`	mark the inode free (`iput`)

Table 2: Disk writes for `echo hi > x` and then `rm x`.

Block 33 holds `x`'s inode and block 32 holds the root directory's inode; block 46 is the root directory's content and block 45 the bitmap. Creation writes block 33 twice because `ialloc` first claims a free inode by setting its type on disk, and `create()` in `sysfile.c` then initializes it. The call path is `sys_open` → `create` → `ialloc`, `iupdate`, and `dirlink` → `writei` → `iupdate`, followed by `itrunc` → `iupdate`. Writing data goes `sys_write` → `filewrite` → `writei`, which calls `bmap`; for a block not yet allocated, `bmap` calls `balloc`, which updates the bitmap and calls `bzero`. `writei` finishes with `iupdate` because the file's length and `addrs[]` have changed. Deletion goes `sys_unlink` → `writei` and `iupdate`, then `iunlockput` → `iput` → `itrunc` (which calls `bfree` and `iupdate`) and a final `iupdate`; its four `iupdate`s record the directory's inode, the link count, the truncation, and the freeing of the inode.

14.6 Concurrency and the buffer cache

xv6 has modest concurrency goals: parallel reads and writes of different files, and parallel pathname lookup. Even these pose correctness challenges. For example, could concurrent calls to `ialloc` get the same inode? No: `ialloc` reads the inode block with `bread`, modifies it, writes it, and releases it with `brelse`; `bread` locks the block (perhaps waiting) and `brelse` unlocks it, so the two calls examine the block one at a time.

The BUFFER CACHE (`bio.c`) holds a few recently used blocks. The file system calls `bread`, which calls `bget`. If the block is already cached, `bget` locks the buffer and returns it; the lock is a SLEEP LOCK, which puts the caller to sleep until the process currently using the buffer releases it. If the block is not cached, `bget` reuses an existing buffer. Incrementing `b->refcnt` keeps a buffer from being recycled while a process waits for it. The cache maintains the invariant that there is only one copy of each disk block in memory. There are two levels of locking: `bcache.lock` protects the description of what is in the cache, and `b->lock` protects just one buffer.

The buffers form a doubly linked list through `bcache.head`. `brelse` moves a released buffer to the front (`bcache.head.next`), and `bget` recycles the buffer at the tail (`bcache.head.prev`), so replacement is least recently used (LRU). Whether LRU is the best policy, whether the double copy of I/O (disk to buffer cache, then buffer cache to user space) could be avoided for better performance, and how much RAM to dedicate to disk buffers are design questions that xv6 answers in the simplest way.

14.7 Pathname lookup

Lookup traverses a pathname one element at a time, touching potentially many blocks: the inode of the top directory, that directory's data, the inode of the next level down, and so on. Each may miss in the cache, and disk accesses are expensive, so xv6 allows parallel lookups: if one process blocks on the disk, another may proceed. This is challenging because an `unlink` may run concurrently with a lookup.

`namex()` in `fs.c` locks the current directory with `ilock()`, finds the next directory's inode, unlocks the current directory, and only then locks the next one. Another process may unlink the next inode in between, but the inode will not be deleted, because the lookup holds a reference to it (its `refcnt` is positive). Unlocking the current directory before locking the next also avoids deadlock when both are the same inode, as in a lookup of "`.`". The key idea is that taking a reference to an inode is separate from locking it.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 14 (Fall 2020).

Lecture 15: Crash Recovery

15.1 The problem

Suppose the power fails while the file system is being written (say, during a `make`). After a reboot, is the file system still usable? File system operations are multi-step disk operations, and a crash in the middle of one may leave the on-disk file system with its invariants violated. After the reboot the result can be bad (the kernel crashes again because the file system is corrupt) or worse (no crash, but reads and writes return incorrect data).

Recall the disk writes for creating a file with `echo hi > x`: block 33 to allocate `x`'s inode, block 33 to initialize it, block 46 to record `x` in the root directory's content, block 32 to update the root directory's inode, and block 33 again. A crash after the inode writes but before the directory write leaves an inode that is allocated but used by no file, which is not so bad: an inode is lost. If the file system instead wrote blocks 46 and 32 first and then 33, a crash between 32 and 33 would leave a directory entry pointing to a free inode, which is a disaster: the system may crash again, or worse, the inode may be allocated to another file.

Writing data is similar. Writing `"hi"` to `x` sets the allocation bit in the bitmap (block 45), writes the data block (595), and updates the inode's size and `addrs[0]` (block 33). A crash after the bitmap write but before the inode write leaves a block that is neither free nor used, losing a data block (not so bad). With the opposite order, inode first and bitmap second, a crash in between leaves an inode referring to a free block, which may later be allocated to another file and so be shared among several files (a disaster). `unlink` likewise updates the bitmap, frees the inode, and erases the directory entry.

After rebooting and running recovery code, what can we hope for?

1. The file system's internal invariants hold; for example, no block is both on the free list and in a file.
2. All but the last few operations are preserved on disk: data written yesterday survives, but data being written at the time of the crash might not, so the user might have to check the last few operations.
3. There are no order anomalies: after `echo 99 > result ; echo done > status`, recovery must not produce `status` containing `done` without `result` containing `99`.

Correctness and performance often conflict. Disk writes are slow; safety argues for writing to disk as soon as possible, while speed argues for not writing (batching, a write-back cache, sorting writes by track). Crash recovery recurs in every storage system, databases included, and much work over the years has produced many clever trade-offs between performance and correctness.

15.2 Logging

The most popular solution is LOGGING, also called JOURNALING. Its goals are system calls that are atomic with respect to crashes, and fast recovery (no hour-long `fsck`). xv6's log provides safety and fast recovery; Linux's ext3 (Lecture 16) is also fast in normal operation.

We want all of a system call's writes or none of them; call such an atomic operation a TRANSACTION. The system call first records all the writes it *will* do in a log on disk (logging), then records "done" on disk (commit), and only then performs the writes to the file system (install). Recovery after a crash replays all writes in the log if the log says "done," and ignores the log otherwise. This is a WRITE-AHEAD LOG.

Definition: Write-ahead rule. Install *none* of a transaction's writes to disk until *all* its writes are in the log on disk and the logged writes are marked committed.

Once one write has been installed in the on-disk file system, all of the transaction's other writes must also be done to keep it atomic. Since a crash may come right after that first installation, the other writes must still be available after the crash, and so they must already be in the log.

Logging is a kind of magic: crash recovery of complex mutable data structures is generally hard, yet logging can often be layered on top of an existing storage system, and it is compatible with high performance.

15.3 xv6's log

xv6 keeps the data of logged writes in the buffer cache, pinned so that it cannot be evicted, and on each write adds the block number to an in-memory array (Figure 19). On the disk, block 2 is the log header and blocks 3 onward hold logged blocks (the log ends at block 31; inodes start at 32, the bitmap is block 45, and content blocks start at 46). A transaction commits as follows:

1. Write the modified buffers to the log on disk, and *wait* for the disk to complete these writes (they are synchronous).
2. Write the log header sector: the block numbers and a non-zero count `n`.
3. After the commit, install the logged blocks at their home locations in the file system and unpin them.
4. Write zero to `n` in the on-disk header.

The on-disk `n` indicates commitment. Non-zero means the log content is valid and is a complete transaction; zero means it is not committed, may be incomplete, and recovery should ignore it. The write of a non-zero `n` is the COMMIT POINT. On reboot, if $n = 0$ recovery does nothing; if $n > 0$ it re-installs the logged blocks and cleans the log.

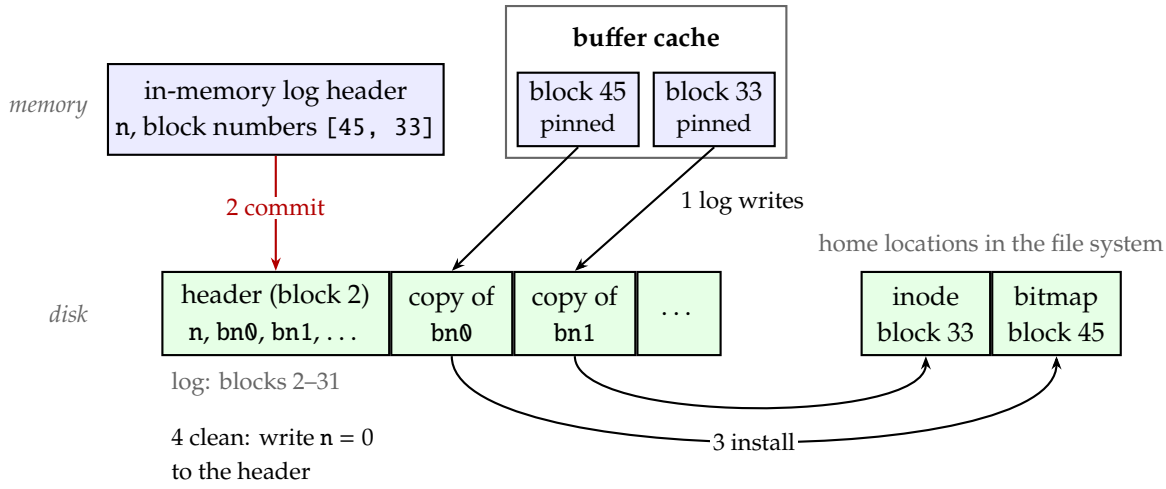


Figure 19: xv6's log. Logged blocks stay pinned in the buffer cache, and the in-memory header records their block numbers. Commit copies them to the on-disk log (1), writes the header with non-zero `n` (2, the commit point), installs them at their home locations (3), and zeroes `n` (4).

With `bwrite()` instrumented to print the low-level disk writes made at commit, `echo a > x` produces three transactions:

Transaction	Log writes	Commit	Install	Clean
create `x`	3, 4, 5 (copies of 33, 46, 32)	2	33, 46, 32	2
write `"a"`	3, 4, 5 (copies of 45, 595, 33)	2	45, 595, 33	2
write `"\n"`	3, 4 (copies of 595, 33)	2	595, 33	2

In the second transaction, the `bzero` of newly allocated block 595 and the write of `a` into it are absorbed into a single logged block.

15.4 The code path

`sys_write` calls `filewrite` in `file.c`, which computes how many blocks it can write before the log is full and writes that many in one transaction. Each transaction brackets the writes made in `fs.c`'s `writei`:

```
begin_op();
  // writei():
  bmap(...);        // may allocate: balloc writes the bitmap,
                    //   log_write of the bzero'ed new block
  bp = bread(...);
  // modify bp->data
  log_write(bp);    // absorbs the bzero
  iupdate(ip);      // writes the inode
end_op();
```

- `begin_op()` in `log.c` marks the start of a group of writes that must be atomic. It checks whether the log is currently being committed, and whether this operation's writes will fit in the remainder of the log.
- `log_write()` adds the block's sector number to the in-memory array and calls `bpin()` to pin the block in the buffer cache, so that `bio.c` will not evict it.
- `end_op()` commits if there are no other outstanding operations.
- `commit()` copies the updated blocks from the cache to the log on disk, records the sector numbers and "done" in the on-disk log header, installs the writes by copying them from the on-disk log to the file system (then `bunpin()` lets them be evicted), and erases "done" from the log.

15.5 Recovery

A crash loses memory, leaving only the disk as of the crash. During boot, before using the file system, the kernel calls `recover_from_log()`, which copies the logged blocks to their real locations if the log header says "done." The on-disk log covers every crash point:

- a crash before commit leaves `n` = 0, so the partial transaction is ignored, as if the system call never happened;
- a crash during commit is decided by the commit point: before the header write nothing happened, after it everything will;
- a crash during `install_trans()` leaves a committed log, so recovery installs every block again;
- a crash during recovery itself just causes recovery to run again at the next boot.

Replaying the log more than once is fine, as long as no other activity intervenes.

xv6 assumes the disk is FAIL-STOP: each write is either done correctly or not done at all (perhaps the last write is lost to a power failure). Thus there are no partial writes (each sector write is atomic), no wild writes, no decay of sectors (no read errors), and no reads of the wrong sector.

15.6 Challenges

Preventing write-back from the cache. A system call may update a *cached* block, but the block must not reach its home location until the transaction commits. This is tricky because the cache may run out of space and be tempted to evict entries to read and cache other data. In the create example, if the file system logs the dirty inode block, logs the directory block, and then evicts the dirty inode block (writing it home) before the commit, the write-ahead rule is violated. xv6 makes the buffer cache big enough, pins dirty blocks in it, and unpins them after the commit.

Each system call's writes must fit in the log. The maximum log size is 30 blocks. xv6 computes an upper bound on the number of blocks each system call writes and makes the log at least that large (and the buffer cache at least as large as the log). It breaks some system calls into several transactions; a large `write()` is split this way, so large writes are not atomic, but a crash leaves a correct prefix of the write.

Concurrent system calls. The log must hold writes from several calls, and a commit must write all of them (a GROUP COMMIT), but it must not write data from calls that are still inside their transaction. xv6 lets no new system call start if its data might not fit in the log; such a call waits for the current calls to complete and commit. When the number of in-progress calls falls to zero, xv6 commits, frees the log space, and wakes the waiting calls. All concurrent operations must fit in the log, which limits the number of concurrent file system calls.

A block written several times. Writes affect only the cached block in memory, so a cached block may reflect several uncommitted transactions. But installation happens only when no transactions are in progress, so installed blocks reflect only committed transactions. Logging such a block once for many writes is good for performance; this is WRITE ABSORPTION.

15.7 Assessment

xv6's log is correct thanks to the write-ahead rule, it has good disk throughput because the log naturally batches writes (though every data block is written twice), and it allows concurrency. It is not very efficient, however:

- every block is written twice, once to the log and once at installation;
- it logs whole blocks even if only a few bytes changed;
- it writes each log block synchronously, where it could write them as a batch and write only the header synchronously;
- log writes and install writes are both eager, where both could be lazy for more write absorption (as long as the log is still written first);
- operations that do not fit in the log are troublesome: `unlink` might dirty many blocks while truncating a file.

Logging thus solves atomicity for multi-step operations, but leaves the question of performance.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 15 (Fall 2020).

Lecture 16: File System Performance and Fast Crash Recovery

Logging is a successful and important idea: it adds crash recovery to almost any storage system, and it is used widely, in databases, file systems, and fault-tolerant distributed systems. Linux's ext3 file system is a case study in what it takes to make logging efficient, and in the real-world details that follow.

16.1 Review of xv6 logging

In xv6 each system call is a transaction (`begin_op()`, writes, `end_op()`), and its writes initially affect only cached blocks in memory. The goal is to make a system call's writes atomic with respect to crashes: apply all of them to the file system or none, for example both marking a block free *and* erasing its block number from an inode. At the end of a system call xv6

1. writes all modified blocks to the log on disk;
2. writes their block numbers to the log header on disk (the commit point);
3. installs the modified blocks at their home locations on disk;
4. zeroes the log header, so that logged blocks of the next transaction do not appear committed.

After a crash (say the computer loses power but the disk is fine), the reboot runs log recovery before any processes: if the header contains block numbers, it replays the logged writes to their home locations and zeroes the header. Two rules make this work. The write-ahead rule: do not start home writes until all of the transaction's writes are committed in the log on disk, since otherwise recovery could not complete the transaction. The freeing rule: do not erase or overwrite the log until all home writes are done.

xv6's logging is slow. Each system call waits for all of its disk I/O (synchronous update); new system calls wait for a commit to finish, which prevents concurrent execution by multiple processes; and every block is written to disk twice, once to the log and once to the file system.

16.2 ext3's log

ext3 adds a log (also called a *journal*) to ext2, an earlier file system without one. In memory it keeps a write-back block cache and, per transaction, a sequence number, the set of block numbers to be logged, and the set of outstanding handles, one per system call. On disk it has the file system and a circular log, which can hold several transactions (Figure 20):

- a LOG SUPERBLOCK holding the offset and sequence number of the earliest valid transaction (a block at the start of the log, not the file system's superblock);
- for each transaction, a DESCRIPTOR BLOCK (magic number, sequence number, and the block numbers being logged), the logged data blocks it describes, and a COMMIT BLOCK (magic number and sequence number).

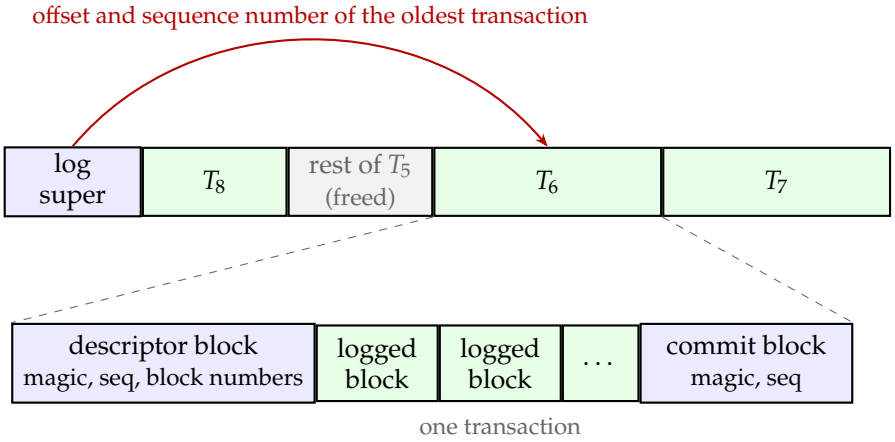


Figure 20: ext3's circular log. The log superblock points to the oldest valid transaction (T_6); transaction T_8 has wrapped around and overwritten the start of the freed T_5. Below: the blocks of one transaction.

16.3 Asynchrony, batching, and concurrency

ext3 gets its performance from three sources.

Asynchronous disk updates. System calls modify only blocks in the cache and do not wait for disk writes. They return quickly, the application can compute while the disk writes (I/O concurrency), and the delay allows batching. Asynchrony changes the crash semantics, however: a crash may "forget" the last few completed system calls. This is a significant difference from xv6 and many file systems without logs: an application may tell a user "I finished writing the file," yet a crash may make the file disappear. Careful applications (databases, mail servers, editors) call `fsync(fd)`, which forces the file's updates into the log and commits them before returning.

Batching. ext3 commits every few seconds, so a transaction typically includes many system calls. Batching spreads the fixed costs of a transaction (the descriptor and commit blocks, seeks and rotation, the barrier between system calls) over many calls; gives WRITE ABSORPTION, since many calls in a batch may modify the same block (a bitmap, a directory block) and need only one disk write; and allows disk scheduling, sorting writes into fewer seeks and longer sequential runs.

Concurrency. Multiple system calls can add to the current transaction, and multiple transactions can be in different stages at once: one *open* transaction accepting new system calls, *committing* transactions doing their log writes, *committed* transactions writing to their home locations, and *old* transactions being freed. This gives CPU concurrency (many processes use the file system in

parallel on a multi-core machine) and I/O concurrency (processes operate on the file system in the cache while earlier transactions write the disk).

16.4 System calls and commit

A system call brackets its updates with the logging interface:

```
sys_unlink() {
  h = start();
  get(h, blockno);  // for each block to be modified
  // modify the block(s) in the cache
  stop(h);
}
```

`start()` tells the logging system about a new system call; the transaction cannot commit until every started call has called `stop()`, and `start()` can block the call if needed. `get()` says the call will modify a cached block, which is added to the current transaction's list of blocks to log. `stop()` does *not* cause a commit; a transaction can commit once all the system calls it includes have called `stop()`.

Committing a transaction:

1. block new system calls;
2. wait for in-progress system calls to `stop()`;
3. open a new transaction and unblock new system calls;
4. write a descriptor with the list of block numbers to the on-disk log;
5. write the modified blocks from the cache to the on-disk log;
6. wait for the descriptor and data log writes to finish;
7. write the commit record to the on-disk log;
8. wait for the commit write to finish: this is the commit point;
9. the modified blocks may now go to their home locations on disk (but are not forced to).

Log space is reused as the log wraps around (for example `SB` T_4 T_1 T_2 T_3). T_2's space can be reused once T_2 has committed, all of T_2's blocks have been written to their home locations (so it never needs to be replayed), and all transactions before T_2 have been freed. Freeing writes the log superblock with the offset and sequence number of the resulting oldest transaction.

16.5 Recovery

A crash loses RAM, and thus the cached blocks, but the disk is assumed to be fine. The crash may interrupt the writing of the last transaction to the log, so the on-disk log may hold a run of complete transactions followed by one partial one. Some cached blocks may also have reached their home locations, but only for fully committed transactions. In Figure 20, the superblock points to T_6, T_5 is free, and T_8 has overwritten the start of T_5. Recovery proceeds:

0. reboot with the disk intact;
1. read the offset and sequence number of the oldest transaction from the log superblock;
2. find the end of the log by scanning until a bad magic number (a missing commit) or an unexpected sequence number (an old entry), and go back to the last valid commit block, ignoring a final partial transaction in case the crash happened midway through writing it;
3. replay all blocks through the last valid commit, in log order.

A block after the last valid commit might look like a descriptor, the start of a new transaction. A descriptor left over from earlier use of the log has too low a sequence number. File data that happens to look like a descriptor cannot occur: ext3 replaces the magic number in journaled data blocks with zero and sets a flag for that block in the descriptor. If there is another crash during replay, recovery after the next reboot replays exactly the same writes.

16.6 Tricky details

Why ext3 delays the start of T_2's system calls until all of T_1's finish. This barrier sacrifices some performance, but prevents the following problem. Let `create(x)` belong to T_1 and `unlink(y)` to T_2, and let the system crash after T_1 commits but before T_2 does. If `unlink` could start before `create` finished, `unlink(y)` could free `y`'s inode (marking it free in the block cache) and `create(x)` could then allocate that same inode, reading `unlink`'s write. After recovery, `y` exists (the `unlink` did not commit) and `x` exists too, using the same inode as `y`. T_1 must not write T_2's updates to the log, so T_2's system calls may not start until T_1's are done. ext3's copy-on-write (below) does not fix this, because it gives T_1 a copy of the blocks as of the end of T_1, and the `unlink` executed before T_1 ended. Nor can T_2 get its own copies of the blocks, separate from T_1's, since later system calls generally should see the effects of earlier ones.

Committing from a snapshot. Suppose `create(d/f)` is in T_1 and `create(d/g)` is in T_2; both write `d`'s directory content block. If T_1 writes its log *after* `create(d/g)` has run, T_1 would log T_2's update too, and a crash plus replay would apply part but not all of T_2: for example the directory entry `d/g` without an initialized inode for `g`. ext3 therefore gives T_1 a private copy of the block cache as it was when T_1 closed, and T_1 commits from this snapshot. The copies are made efficiently with copy-on-write, and they let system calls in T_2 proceed while T_1 commits.

16.7 Ordered data mode

So far ext3 has been in JOURNALED DATA mode, which writes file content blocks to the log as well as metadata (inodes, directory content, free bitmaps), so file content is covered by the atomicity guarantee. A `write()` that grows a file logs a descriptor, the inode, the block bitmap, the content, and a commit. Logging content is slow: every data block is written twice, and data is usually much bigger than metadata.

Could file content be left out of the log entirely, and if so, when should it be written? Not at arbitrary times: if the metadata commits first, a crash may leave an inode pointing to blocks from someone else's deleted file, exposing that file's old content. In ORDERED DATA mode ext3 writes file content blocks to their home locations on disk *before* committing the log, and does not log them. For a `write()`, ext3 writes the content blocks, waits, and then commits the updated inode (with the block number and increased size). If the crash comes after the data write but before the commit, the block on disk holds new data that is not visible, because neither the inode (size and block list) nor the free bitmap was updated: there is no metadata inconsistency, and the blocks are still free. Most people use ordered data mode, since it avoids writing data twice and is much faster.

It makes sense for the log to ignore file content because the real point of the log is to protect the file system's internal invariants (for example, that a block is not both free and used in an inode) so that recovery after a crash is fast; content is irrelevant to that. In any case the file system does not know enough about application writes to tell which groups of them must be atomic. Applications can instead call `fsync()` to make sure data is safe on disk.

Ordered data mode has its own correctness challenges:

A. `rmdir` frees a directory's content block, the block is reused for a `write()` to some file, and the system crashes before the `rmdir` or the write commits. After recovery it is as if the `rmdir` never happened, but the directory's content block has been overwritten. Fix: do not reuse a freed block until the system call that freed it has committed.

B. `rmdir` commits, its block is reused in a file, the ordered file write and its commit follow, and then a crash and recovery replay the `rmdir`'s logged directory block but not the file content (which was never logged). The file is left holding directory content such as `.` and `...` Fix: put REVOKE records in the log that prevent replay of a given block.

Both problems come from changing a block's type (content versus metadata), so another solution would be never to change a block's type.

Note. Summary of ext3's rules. The basic write-ahead rule: do not write a metadata block to the on-disk file system until it is committed in the on-disk log. Wait for all system calls in T_1 to finish before starting T_2. Do not overwrite a block in the buffer cache before it is in the log. Do not free an on-disk log transaction until all its blocks have been written to the file system. In

ordered data mode, also write data blocks to the file system before the commit, do not reuse a free block until the freeing system call has committed, and do not replay revoked blocks.

16.8 More corner cases

An open file that is unlinked. If a process opens a file and then unlinks it, `unlink` removes the directory entry but does not free the inode or blocks, because the file is still open. The unlink commits (just the removal of the entry), and then the system crashes. The log contains no writes that free the inode or blocks, so after recovery they are neither on the free list nor reachable by any name, and will never be freed. The solution is to add the inode to a linked list rooted in the file system's superblock and commit that together with the removal of the directory entry; recovery walks the list and completes the deletions.

Running out of log space. If there is not enough free log space for a transaction's blocks, ext3 frees the oldest transactions by installing their writes in the on-disk file system. So many system calls might also start that the whole log could not hold their writes even with all older transactions freed (unlikely, since each call writes few blocks compared to the log size). So each system call passes the maximum number of blocks it needs to `start()`, which waits if the current transaction's total is too high and works on freeing old transactions.

Reservations also matter for a more subtle reason. T_1 writes block 17 and commits, finishing its log and commit record, but block 17 has not yet been written to its home location. T_2 then writes block 17 in the cache; ext3 does *not* copy block 17 here, since it copies only when needed to write T_1's log. Now a system call in T_2 does a write for which there is no log space. ext3 cannot install T_1's blocks at their home locations and free T_1's log space, because the cache no longer holds T_1's version of block 17 (ext3 could read it back from the log, but does not). Reservations detect the potential exhaustion of log space in advance and keep such a system call from starting.

Checksums. A transaction's log blocks must be on disk before the commit block is written, so ext3 waits for the disk to report "done" before starting the commit write. But disks usually have write caches and reorder writes for performance; this is sometimes hard to turn off (the disk lies), and people often leave reordering enabled for speed. It is bad news if the disk writes the commit block before the rest of the transaction. The solution is for the commit block to contain a checksum of all the transaction's data blocks; recovery computes the checksum of the data blocks and installs the transaction only if it matches the one in the commit block. ext4 has log checksums; ext3 does not.

In sum, logs make multi-step disk updates atomic with respect to crashes, and recovery relies on the write-ahead rule and is fast. Batching and concurrency give ext3 much better performance than xv6, at the cost of many corner cases.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 16 (Fall 2020).

Lecture 17: Virtual Memory for Applications

Operating system kernels use virtual memory in creative ways. Appel and Li (1991) argue that user-level applications can benefit from virtual memory too, and list uses such as concurrent garbage collection, generational garbage collection, concurrent checkpointing, data-compression paging, persistent stores, and shared virtual memory. Most operating systems today provide `mmap()` and user-level page faults.

17.1 User-level VM primitives

The paper asks for six primitives. xv6 supports none of them (its alarm handler is the closest thing to a user-level trap), so one way to read the paper is as a statement of what a good operating system should support. Unix today covers most of them:

Primitive	Meaning	Unix today
Trap	handle page-fault traps in user mode	`sigaction`, `SIGSEGV`
Prot1	decrease the accessibility of a page (read/write to read-only to none)	`mprotect()`
ProtN	decrease the accessibility of *N* pages (saves TLB flushes)	`mprotect()`
Unprot	increase the accessibility of a page	`mprotect()`
Dirty	return the pages dirtied since the previous call	no, but a workaround exists
Map2	map one physical page at two virtual addresses with different protections	`shm_open` and two `mmaps`

`mmap()` maps memory into the address space, with many flags and options. It can map a file, or anonymous memory (preferred over `sbrk`):

```
addr = mmap(NULL, len, PROT_READ|PROT_WRITE, MAP_PRIVATE, fd, offset);
addr = mmap(NULL, len, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
```

`mprotect(addr, len, PROT_READ)` changes a mapping's permission; with `PROT_NONE` any access traps. `munmap(addr, len)` removes a mapping. `sigaction()` configures a signal handler, here for the segmentation-fault signal that a protection violation raises:

```
act.sa_sigaction = handle_sigsegv;
act.sa_flags = SA_SIGINFO;
sigemptyset(&act.sa_mask);
sigaction(SIGSEGV, &act, NULL);
```

Linux has many more VM calls: `madvise()`, `mincore()`, `mremap()`, `msync()`, `mlock()`, `mbind()`, `shmat()`, and others.

17.2 Implementation

An address space consists of page tables and VIRTUAL MEMORY AREAS (VMAs). A VMA is a contiguous range of virtual addresses with the same permissions, backed by the same object (a file or anonymous memory); an address space might have VMAs for 1000–2000 and 2100–2200. VMAs tell the kernel how to handle page faults.

A user-level trap works as follows. The PTE (or TLB entry) is marked protected, invalid or read-only. On an access the CPU saves the user state and jumps into the kernel, which asks the VM system what to do: page in from disk? dump core? The VM system looks at the VMA, and for a protected page generates a signal, an UPCALL into the user process, run lower on the user stack or on a separate stack. The user handler can do anything; it probably must Unprot the referenced page, to avoid a repeated fault. The handler returns to the kernel, which returns to the user program, continuing or restarting the instruction that trapped.

17.3 Example: a huge memoization table

An application wants a precomputed table $\mathtt{n} \mapsto \sqrt{\mathtt{n}}$, so that it can look up `sqrts[n]`, which is very fast when the entry is present. The table is bigger than physical memory. With user-level VM primitives the application allocates a huge range of virtual addresses without backing memory. A lookup in an absent page faults; the handler allocates a page, computes the square roots for all entries on that page, and lets the lookup continue. When much memory is in use, the handler frees some pages again with Prot1 or ProtN, so they will fault and be recomputed if used later. Taken to the extreme, the whole table can occupy just a *single* physical page, which the handler moves to whichever part of the table was last touched.

17.4 Concurrent garbage collection

In a garbage-collected language the application, called the MUTATOR, allocates memory and computes with it but never calls `free`, which is error prone, particularly in concurrent programs. A COLLECTOR concurrently finds free memory: memory no longer reachable from the roots, which are the registers and stack pointers of every thread. Traditional implementations need language and compiler support to instrument loads and stores; user-level VM can avoid this instrumentation and be faster.

A COPYING COLLECTOR divides the heap into a FROM-SPACE and a TO-SPACE. At the start of a collection all objects are in from-space. The collector copies the objects the roots point to into to-space, leaving in each from-space original a FORWARDING POINTER to its copy, and then scans the copies, copying

(forwarding) the objects they point to. At the end, from-space holds only discarded objects and is free memory.

Baker's real-time algorithm makes this incremental. To-space is divided into scanned, unscanned, and new regions, and the world is never stopped for collection: each allocation scans and forwards a few more objects, and each dereference checks whether the pointer is into from-space and, if so, forwards it. This is attractive: allocation is cheap, the heap is compacted so there is no free list, and every allocation does only a little scanning and copying, which is the "real-time" aspect. The costs are a test and branch on every pointer dereference, and difficulty running the collector at the same time as the program: the collector tracing the heap races with program threads, risking two copies of one object.

With VM, the application threads need no explicit checks. After copying the roots, the collector unmaps the unscanned part of to-space (initially a page holding the roots' copies). When a thread touches the unscanned region it takes a page fault; the handler scans just that page, forwarding the pointers in all of its objects, and then Unprots it. There is at most one fault per page and no compiler change: the VM hardware does the pointer check. It is also easy to make concurrent: a collector thread can scan pages in parallel with the application and Unprot each page after scanning it, and the only synchronization needed is over which thread scans which page.

The tricky issue is that the collector must read and write unscanned pages that the application cannot access. The toy collector `baker.c` solves this with Map2 (Figure 21): it creates a shared-memory object with `shm_open` and `mmap`s it twice, once for the mutator and once for the collector. Without VM, the mutator reads every pointer through `readptr()`, which calls `forward()` to copy the target into to-space if it is still in from-space. With VM this check is compiled out (`#ifndef VM`). At the flip, to-space becomes `PROT_NONE` in the mutator's mapping, and a `SIGSEGV` handler does the work instead:

```
1  static void
2  handle_sigsegv(int sig, siginfo_t *si, void *ctx)
3  {
4    uintptr_t fault_addr = (uintptr_t)si->si_addr;
5    double *page_base = (double *)align_down(fault_addr, PGSIZE);
6    pthread_mutex_lock(&lock);
7    scan(page_base);            // forward the pointers of this page's objects
8    pthread_mutex_unlock(&lock);
9    mprotect(page_base, PGSIZE, PROT_READ|PROT_WRITE);   // Unprot
10 }
```

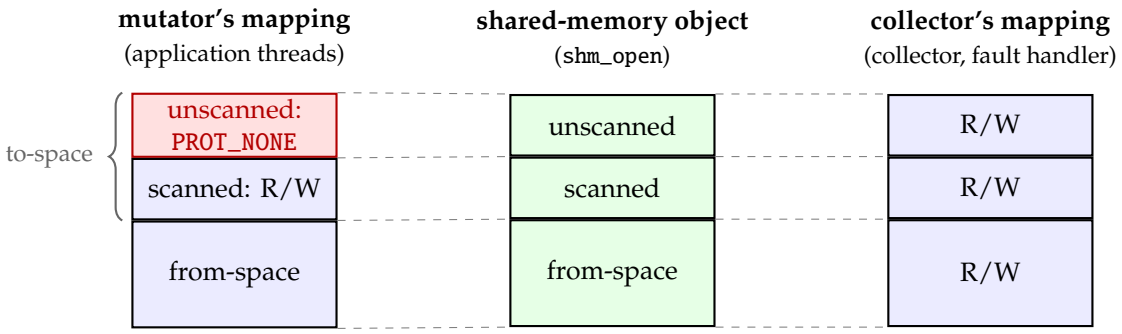


Figure 21: Baker's collector using VM. The heap is a shared-memory object mapped twice. In the mutator's mapping the unscanned part of to-space is inaccessible, so the first access to an unscanned page traps; the collector and the fault handler use a second, read/write mapping of the same pages.

17.5 Are the primitives good enough?

Functionally, Map2 is the only issue for concurrent collection, and not really one, since a page never has to be made accessible twice. Speed is the question. Appel and Li report that traps are not fast enough: 500 μs to scan a page but 1200 μs to take the trap. One could scan three pages per trap, though, and the right comparison is with running Baker's actual algorithm with its checks, against which the VM version might be faster even with slow traps; the time saved by having a second CPU do the scanning is not counted either. How often faults occur hardly matters, since there is at most one fault per page and more faults simply mean more of the necessary scanning.

Is user-level VM a good idea? Most of the use cases could instead be implemented by adding instructions, for example a compiler-inserted check on each reference in the application thread. In favor of VM, it avoids compiler changes, and the CPU already provides VM support. Against it, it requires OS support, and efficient support: most kernels cannot expose the raw performance of the paging hardware, because the OS imposes much abstraction. Some uses, such as checkpointing and shared virtual memory, Unix does support.

Much has changed between 1991 and 2020. The VM API and its implementations evolve continuously (with continued research, for example at OSDI 2020), with big changes such as five-level page tables, address-space identifiers, and kernel page-table isolation. Switching address spaces is now essentially free thanks to tagged TLBs, and extending addressability no longer matters with 2^{52} bytes of virtual address space.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 17 (Fall 2020).

LECTURE 18: OS ORGANIZATION

What should a kernel do, and what should its abstractions and system calls look like? The answers depend on the application and on the programmer's taste; there is no single best answer, and the topic is more about ideas than about specific mechanisms.

18.1 Traditional kernels

The traditional approach, taken by UNIX, Linux, and xv6, combines powerful abstractions with a MONOLITHIC kernel implementation. The philosophy of powerful abstractions is:

- *portable interfaces*: files rather than disk controller registers, address spaces rather than MMU access;
- *simple interfaces that hide complexity*: all I/O through file descriptors and `read`/`write` rather than an interface specialized for each device, and address spaces with transparent paging to disk;
- abstractions help the kernel *manage and share resources*: the process abstraction puts the kernel in charge of scheduling, and the file and directory abstraction puts it in charge of disk layout;
- abstractions help the kernel *enforce security*: file permissions, and processes with private address spaces;
- *lots of indirection* (file descriptors, virtual addresses, file names, PIDs) helps the kernel virtualize, hide, revoke, and schedule.

Powerful abstractions have led to big monolithic kernels: the kernel is one big program, like xv6. Kernel subsystems cooperate easily, with no irritating boundaries: `exec()` and `mmap()` are part of both the file system and the VM system, and it is relatively easy to add symbolic links, copy-on-write `fork`, `mmap`, and so on. All kernel code runs with high privilege, with no internal security restrictions.

What is wrong with traditional kernels?

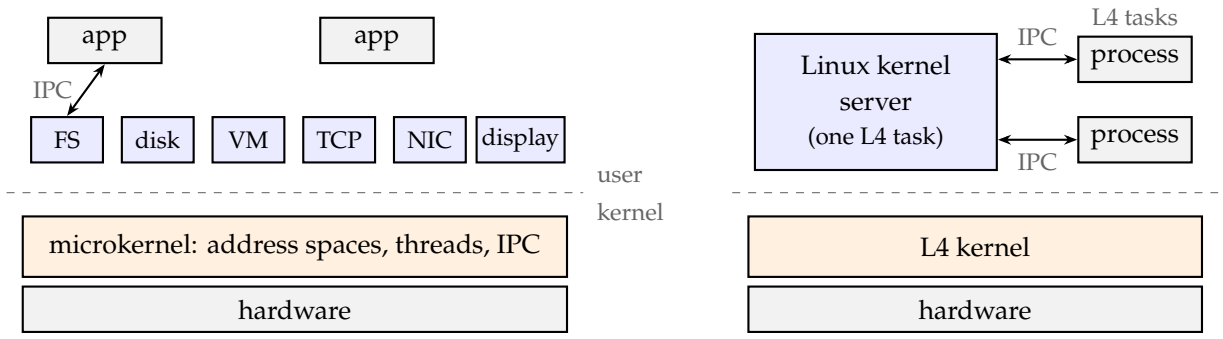
- They are big, hence complex, hence buggy and insecure.
- They may be over-general and therefore slow. Sending one byte through a UNIX pipe executes a surprising amount of code: buffering, locks, sleep and wakeup, the scheduler.
- Many design decisions are baked in and cannot be changed, which may be awkward: an application may want to wait for a process that is not its child, or change another process's

address space, and a database may lay out its B-tree files on disk better than the kernel's file system can.

- It is hard to create kernel extensions that others can use, such as new device drivers or file systems.

18.2 Microkernels

The big idea of a MICROKERNEL is to move most operating system functionality into user-space service processes (Figure 22a). The kernel itself can be small, providing address spaces, threads, and INTER-PROCESS COMMUNICATION (IPC), which lets threads send each other messages. The 1980s saw a big burst of microkernel research, of which CMU's Mach was perhaps the most influential. Microkernels are used today in embedded systems, phone chips, and car entertainment systems, and their ideas (especially user-level servers and IPC) influenced Windows and macOS.



(a) microkernel organization (b) L4Linux

Figure 22: (a) A microkernel provides only address spaces, threads, and IPC; file system, disk, VM, network, and display services run as user-level processes that applications reach by IPC. (b) L4Linux runs the Linux kernel as a user-level server on L4; each Linux process is an L4 task whose system calls are IPCs to the server.

Microkernels attract interest because they are focused, elegant, and a clean slate. Being small, they can be more secure (less code means fewer bugs to exploit), verifiable (as seL4 is), and easier to optimize; applications do not pay for features they do not use, and the kernel avoids forcing design decisions on them. Running services at user level may encourage modularity, makes services easier to extend, customize, or replace, and adds robustness, since an individual service can be restarted; most bugs are in drivers, and this gets them out of the kernel. A microkernel can also run or emulate multiple operating systems, like a virtual machine monitor.

The challenges are to find a minimal kernel API, with simple primitives on which `exec`, `fork`, `mmap`, and the like can be built; to build the rest of the operating system at user level; and to get good performance despite IPC and less integration.

18.3 L4

L4 has evolved over time through many versions and reimplementations, and is used commercially in phones and embedded controllers. It is representative of the microkernel approach and emphasizes minimality: 7 system calls (Linux has 300+, xv6 has 21) and 13,000 lines of code. Its basic abstractions are address spaces (TASKS), threads, and IPC. Its system calls:

- create an address space;
- create or destroy a thread in an address space (possibly another one);
- send or receive a message by IPC, addressed by thread ID;
- map pages of one's memory into another address space, which must agree; this happens via IPC, so one task can modify another task's page table, and is used to create new tasks and share memory;
- intercept another address space's page faults, acting as its PAGER; the kernel delivers the faults by IPC;
- handle device interrupts, which the kernel delivers by IPC (device hardware is accessed directly, without a system call).

The L4 kernel lacks almost everything Linux or even xv6 has: a file system, `fork()`, `exec()`, pipes, device drivers, a network stack. All of these must be user-level code, in a library or a server process.

Thread switching in L4 holds no surprises. The current user-level thread gives up the CPU when an IPC system call waits, on a timer interrupt, or by calling `yield()`. The kernel saves the user thread's registers, picks a runnable thread, restores its registers, switches page tables, and jumps to user space.

Every task has a pager task. On a page fault:

1. the thread faults;
2. the kernel suspends the thread;
3. the kernel sends the fault information to the pager in an IPC;
4. the pager picks one of its own pages;
5. the pager sends that page's virtual address in an IPC reply to the faulting thread;
6. the kernel intercepts the IPC, maps the page into the target, and resumes it.

Pagers can allocate memory (`sigma0` allocates on fault for early tasks), implement copy-on-write `fork` (together with a system call that revokes access), or implement `mmap` of files.

18.4 Fast IPC

Microkernel programs do a lot of IPC, and in early systems IPC was expensive (multiple kernel crossings, TLB misses, context switches); its cost caused many to dismiss microkernels, so L4's designers put huge effort into IPC performance.

A slow design patterns IPC on UNIX pipes, with a message queue in the kernel. `send(id, msg)` appends the message to the queue and returns; `recv(&id, &data)` removes a waiting message and returns, or sleeps. This is called ASYNCHRONOUS and BUFFERED IPC. The usual request-response pattern (RPC) then needs a second queue for replies, four system calls (each side does a `send` and a `recv`, each of which may disturb the CPU's TLB, data, and instruction caches), four message copies (two for the request, two for the reply), two context switches, and two passes through the general-purpose scheduler.

L4's fast IPC (Liedtke, "Improving IPC by Kernel Design," 1993) differs in several ways:

- *Synchronous.* `send()` waits for the target thread's `recv()`. In the common case the target is already waiting, and `send()` jumps into the target's user space as if returning from its `recv()`: no real context switch and no scheduler loop.
- *Unbuffered.* There is no queue in the kernel; since IPC is synchronous, the kernel can copy directly between user buffers.
- *Small messages in registers.* The kernel's `send()` path disturbs few registers (there is no context switch), so a small message needs no copying: it travels in the registers as `send()` jumps into the target's user space.
- *Huge messages as virtual memory grants.* Again there is no copy, though the kernel's `send()` code must change the page table.
- *Combined `call()` and `sendrecv()` system calls.* IPC is almost always used as request-response RPC, so separate `send()` and `recv()` calls are wasteful. A client's `call()` sends a message and waits for the response; a server's `sendrecv()` replies to one request and waits for the next. This halves the number of user/kernel crossings.
- *Careful layout of kernel code* to minimize its cache footprint.

The result was a 20× reduction in IPC cost.

18.5 Running Linux on L4

How can a full operating system be built on a microkernel? For embedded systems it can be fairly simple. General-purpose use, on workstations or web servers, really needs compatibility with existing applications, so the system must mimic something like UNIX. One could reimplement UNIX's kernel services as many user-level services, or one could run an existing Linux kernel as a

process on top of the microkernel, an "OS server." This is perhaps not elegant, but pragmatic, and offers a path to adoption: users might start by just running Linux applications, and then gradually exploit the possibilities of the underlying microkernel.

Härtig et al., "The Performance of μ-Kernel-Based Systems" (1997), take this route (Figure 22b): an L4 kernel, a Linux kernel server, one L4 task per Linux process, and IPC for system calls. The Linux kernel is just a program; the authors modified it to replace hardware access with L4 system calls or IPC in process creation, configuring user page tables, memory allocation, system call handling, and interrupt handling.

Threads. Each Linux process has one or more L4 threads for its user code. The Linux server has just one L4 thread (plus L4 threads waiting for interrupts), which at rest waits for IPCs carrying system calls. The server switches its own L4 thread among the kernel threads of its processes, for example when file system code sleeps waiting for a disk read or a pipe `read()` sleeps waiting for a writer, much as xv6 switches among kernel threads. But these switches are unrelated to user process switching: L4 separately switches among the runnable L4 threads that implement the Linux processes, so the server may be running the kernel thread for P_1 while L4 runs process P_2 on another core. Using L4 threads for the server's kernel threads would cause pain without benefit: it would introduce parallelism inside Linux, but Linux 2.0 did not support SMP (it had no spinlocks), and the authors' hardware had a single core, so there could be no parallel speedup anyway. A drawback is that L4 schedules the user threads, so L4Linux could not enforce Linux's notions of priority.

Memory. The Linux server maps all user memory into its own address space (really, it allocates lots of memory and gives its own memory to user processes). It uses this mapping for `copyin()` and `copyout()`, to dereference user pointers in system calls, which keeps system call IPCs small (they carry a data address, not the data), and for `fork()` and `exec()`. For example, `fork()` works as follows:

1. process P_1 (really an L4 task) calls `fork()`, which its libc turns into an IPC to the Linux server;
2. the server asks L4 to create a new task and thread, P_2;
3. the server allocates as many memory pages as P_1 has;
4. the server uses IPC to tell L4 to map these pages into P_2;
5. the server copies the data from P_1's pages to P_2's pages;
6. the server sends P_2 a special IPC with a stack pointer and program counter, which makes it run;
7. the server sends its reply to P_1 by IPC.

The server is also the pager for user processes: L4 turns their page faults into IPCs to the server, which implements copy-on-write `fork`, lazy allocation, and memory-mapped files. Because L4 does not give direct control over page tables, however, the server could not switch to a page table

that also includes user virtual addresses; until recently Linux used that trick both for performance (no page table switch on a system call) and for convenience in dereferencing system call arguments.

Devices. The server uses Linux's device drivers unchanged, since L4 allows it direct access to device registers; only interrupts arrive differently, as L4 IPCs.

18.6 Evaluation

The paper's main goal is not to argue that microkernels are a good idea, but to show that they can perform well. Two questions matter: is IPC fast (a microbenchmark), and is there some other performance obstacle (a whole-system benchmark)?

In the IPC microbenchmark, `getpid()` is one system call on native Linux but two L4 system calls (an IPC send and receive) on L4Linux. It takes only somewhat more than twice as long on L4Linux, and is far faster than on Mach with a Linux server.

Would system calls that take twice as long be a disaster, or hardly noticeable? The AIM benchmark answers this. It forks a number of processes, each of which randomly uses the disk, allocates memory, uses pipes, computes, and so on, to do a fixed total amount of work; the measured time for all processes to complete is plotted against (a function of) the number of concurrent AIM processes. Only the slope matters: it is the time per unit of work, so lower is better. Native Linux is best, L4Linux is only a little slower, and Mach with Linux is noticeably less efficient. Twice the IPC time thus makes little overall difference: L4Linux is only somewhat slower than Linux and significantly faster than Mach with Linux. These results are not by themselves an argument for using L4, but they are an argument against rejecting it for performance reasons.

18.7 The current situation

Microkernels are sometimes used in embedded computing, in microcontrollers and Apple's "enclave" processor, running custom software. As such, they never caught on for general computing: there was no compelling story for switching from Linux and its peers. Many ideas from microkernel research have, however, been adopted by modern UNIX systems. Mach spurred adoption of sophisticated virtual memory support; virtual machines are partially a response to the OS-server idea; loadable kernel modules respond to the need for extensibility; client/server structure appears in DNS servers and window servers; and macOS has microkernel-style IPC.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 18 (Fall 2020).

Lecture 19: Virtual Machines

A virtual machine is a simulation of a computer accurate enough to run an operating system. A virtual machine monitor (VMM) runs on the hardware, either stand-alone or inside a host operating system such as Linux, and runs guest kernels, which in turn run guest processes.

Virtual machines are used to run many guests per physical machine: individual services often need modest resources and would waste most of a dedicated server, so cloud and enterprise computing need a new layer, under the existing kernel, to manage multiple guests. VMs also isolate better than processes, let one computer run several operating systems (OS X and Windows), provide a kernel development environment (like qemu), and enable tricks such as checkpointing, migration, and monitoring. They have a long history: IBM used them in the 1960s to share big machines, VMware re-popularized them for x86 in the 1990s, and in the 2000s they became widely used in cloud and enterprise computing. Much of the most interesting action in operating system design has shifted to VMs: they overlap heavily with traditional kernels but bring new challenges and opportunities, some problems are solved more easily by inserting a new layer under the operating system, and VMs have affected both operating systems above them and hardware below.

The usual goal is that the guest has no idea it is running in a VM and that the host can support any guest operating system. It must be impossible for the guest to distinguish the VM from a real computer, and impossible for it to escape the VM: a standard operating system must boot and run, and malicious software must be contained. In practice many VMs compromise: the guest knows it is running on a VM and has modifications for efficiency.

19.1 Trap-and-emulate

A VM could be built by writing software that simulates machine instructions: the VMM interprets each guest instruction and maintains the guest's machine state (32 registers, `satp`, the mode, RAM, disk, and so on). This is clearly possible, but slow. It is much faster to execute guest instructions on the real CPU. The difficulty is privileged instructions: if the guest kernel loads a new page table into `satp`, it cannot be given direct access to the supervisor registers.

The solution is to run the guest kernel in user mode, much like an xv6 process, even though it believes it is in supervisor mode (Figure 23). Ordinary instructions, such as adding two registers or calling a function, work fine. Privileged RISC-V instructions are illegal in user mode and trap to the VMM, whose trap handler emulates them: it may apply the operation to the guest's virtual state (for example, reading or writing `sepc`), or transform it and apply it to the real hardware (for example, an assignment to `satp`). This is trap-and-emulate. Such a VMM can be built entirely in software; xv6 could be modified into a trap-and-emulate VMM for RISC-V.

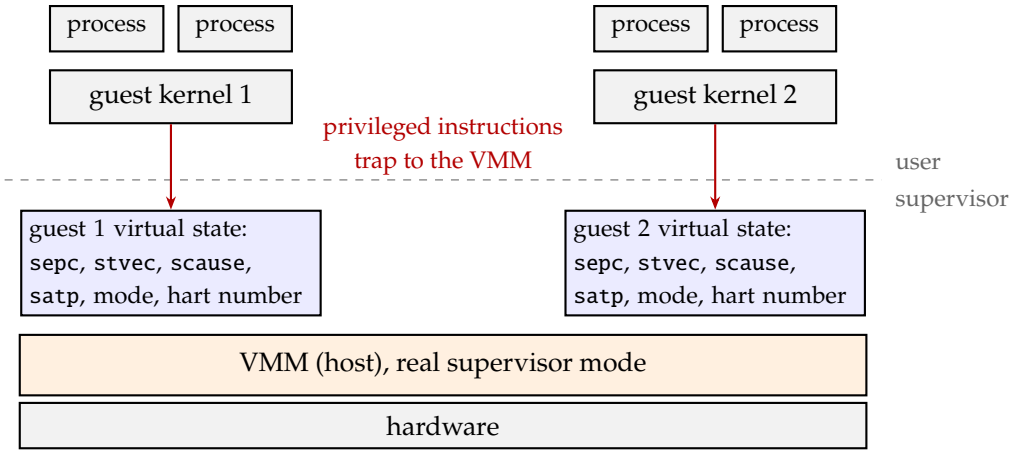


Figure 23: Trap-and-emulate. Guest kernels and their processes all run in real user mode; privileged instructions trap to the VMM, which keeps each guest's virtual supervisor state.

The guest cannot be allowed to see or change the real machine state, so the VMM must virtualize all the `s` registers (`sepc`, `stvec`, `scause`, `satp`, ...), the mode, and the hart number. RISC-V is very convenient here, because every privileged instruction traps when executed in user mode. Not all CPUs are so nice: on 32-bit x86, some privileged instructions do not trap but are silently ignored in user mode.

The VMM's work in common cases:

- *The guest kernel executes* `sret` *to return to user space.* The CPU traps into the VMM (really a trap from user to supervisor mode, saving the guest's PC in `sepc` and perhaps a trapframe). The VMM sets the virtual mode to user and the real `sepc` to the virtual `sepc`, switches page tables (both guest modes run in real user mode, so the guest's `PTE_U` distinctions need different real page tables), and returns from the trap.
- *Guest user code executes* `ecall`. `ecall` always traps, here into the VMM, which sets the virtual `sepc` to the real `sepc`, the virtual mode to supervisor, and the virtual `scause` to "system call"; sets the real `sepc` to the virtual `stvec`; switches page tables for `PTE_U`; and returns from the trap, into the guest kernel's trap handler.
- *The guest kernel reads* `scause` (`csrr a0, scause`). This traps into the VMM, which sets `a0` in the trapframe to the virtual `scause`, advances the real `sepc` by 4, and returns.
- *The guest kernel writes* `satp`. The VMM must ensure that the guest accesses only its own memory, and must remap guest physical addresses. The guest's page table maps guest virtual to guest physical addresses, and the VMM's map for this guest maps guest physical to host physical addresses. The VMM combines them into a SHADOW PAGE TABLE mapping guest virtual to host physical addresses, and installs it in the real `satp` (Figure 24a).
- *The guest kernel modifies a PTE in the active page table.* The VMM need not do anything immediately: the RISC-V specification says PTE modifications take effect only after `sfence.vma`, which traps

to the VMM, and the VMM then generates a new shadow page table.

Simulating devices (disk, network interface, display) is a big challenge, with three strategies:

- *Emulate a common, real device*, which is needed to run an oblivious guest. The VMM intercepts reads and writes of the device's memory-mapped control registers by marking those pages invalid, and turns the resulting page faults into operations on a (different) real device. For example, qemu simulates the UART for xv6 and turns UART reads and writes into characters on your display or ssh connection.
- *A special virtual device tailored for efficiency.* This needs a driver in the guest, so the guest knows it is in a VM, but it can be more streamlined than trapping on control-register accesses. xv6's `virtio_disk.c` is an example; qemu turns its requests into reads and writes of the file `fs.img`.
- *Pass-through access to a real device*: the guest gets direct access to the device hardware, with no traps. This often requires specific support in the device (modern network interfaces have it) and can be very efficient.

Trap-and-emulate works well, but it can be slow because of the many traps into the VMM.

19.2 Hardware-supported virtualization

Intel's VT-x (VMX) and AMD's SVM add virtualization support to x86 CPUs: the CPU directly supports virtual as well as real state, so a guest can execute privileged instructions without trapping. This is faster than trap-and-emulate, makes the VMM software easier to write, and is widely used.

With VT-x the host VMM runs in ROOT MODE, which is nothing special, and the guest runs in NON-ROOT MODE, which has a full set of virtual control registers (the x86 equivalents of `satp`, `stvec`, user versus supervisor mode, and so on). In guest supervisor mode the guest can do everything an operating system kernel expects: switch to guest user mode, take system calls and traps, and so on. Non-root mode *looks* just like an ordinary computer, but limits the guest in some critical ways, mostly through the page table. New instructions switch modes: `VMLAUNCH` and `VMRESUME` go from root to non-root mode, and `VMCALL` from non-root to root mode. Device interrupts (timer, disk) cause an exit from non-root mode, so by default the VMM handles devices. The guest's virtual state is saved in a VM CONTROL STRUCTURE (VMCS) when non-root mode exits, and restored from it by `VMLAUNCH` or `VMRESUME`.

Page tables pose a problem: the guest kernel should control its page table, the guest should be restricted to its allowed physical memory, and trapping into the VMM whenever the guest sets `%cr3` (the x86 `satp`) or modifies a PTE would be expensive. VT-x therefore has *two* layers of address translation in non-root mode (Figure 24b). The VMM sets up an EXTENDED PAGE TABLE (EPT) that contains just the guest physical addresses the guest may use and maps them to the host physical addresses the VMM has allocated for it. The MMU uses `%cr3` to translate a guest virtual address to a guest physical one, then uses the EPT to translate that to a host physical address. The EPT is

invisible to the guest, so the guest can freely read and write `%cr3`, change PTEs, and read dirty bits, while the VMM still provides isolation through the EPT. In a typical setup the VMM allocates some RAM for the guest, maps guest physical addresses 0 to the RAM's size onto it in the EPT, and the guest uses `%cr3` to configure its processes' address spaces.

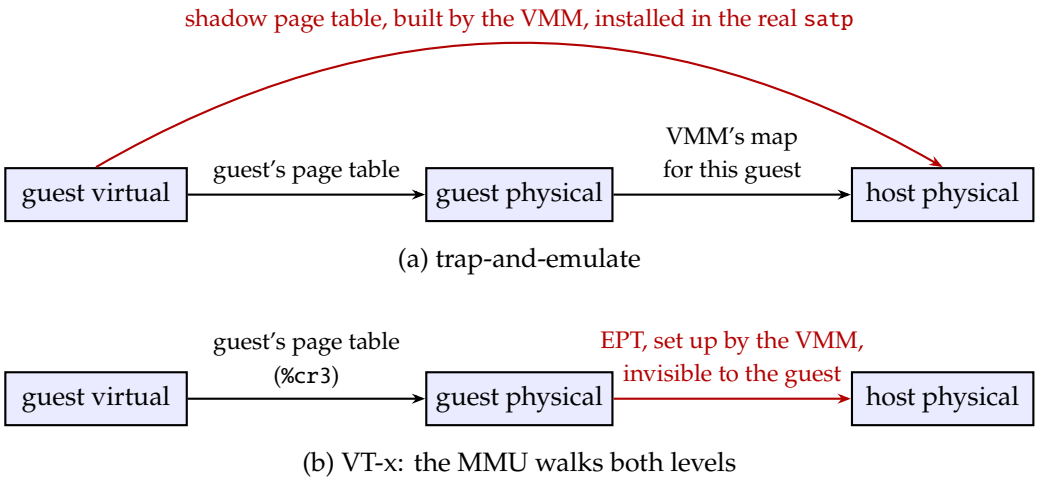


Figure 24: Translating guest addresses. (a) With trap-and-emulate the VMM composes the two mappings into a shadow page table. (b) With VT-x the hardware walks the guest's page table and then the EPT.

19.3 Dune

Dune (Belay et al., 2012) uses VT-x to run Linux *processes* rather than guest kernels, giving user code fast, direct access to page tables, exceptions, and other privileged hardware, so that it can efficiently sandbox untrusted code, or modify its page table and take its own page faults.

Dune is a loadable kernel module for Linux, and an ordinary process can switch into "Dune mode." A Dune-mode process is still a process: it has memory and can make Linux system calls (via `VMCALL`). Its isolation machinery is a little different: it runs in VT-x non-root supervisor mode, its memory is protected by the EPT, and timer interrupts go to Linux rather than to the process, so Linux still controls scheduling. The process gains the ability to read and write its own page table, including the PTE dirty bit, faster than with Linux's `mprotect()`; to handle its own page faults, faster than Linux's upcalls; to switch into guest user mode; and to intercept guest user system calls and page faults.

Sandboxed execution. A web browser may want to run a third-party plug-in, such as a video decoder or ad blocker, that might be malicious or buggy, so it needs a sandbox that executes the plug-in while limiting its system calls and memory accesses. If the browser runs as a Dune process in guest supervisor mode, it creates a page table with `PTE_U` mappings for the memory the plug-in may use and non-`PTE_U` mappings for the rest of its memory, sets `%cr3`, and uses `sret` to enter the untrusted code in guest user mode. The plug-in can read and write its image memory through the page table, and it can execute the system call instruction, but its system calls trap into the browser rather than

the underlying kernel, and the browser decides whether to allow each one.

Garbage collection. Dune was used with a modified Boehm concurrent mark-and-sweep collector, which follows pointers, starting at the registers, to find all live (reachable) objects. Because the collector runs concurrently, the program may modify an object after the collector has traced it, so the collector needs to know which objects were modified in order to revisit them. Under Dune it uses the PTE dirty bit (`PTE_D`) to detect written pages, and clears all dirty bits when a collection is done; direct access to PTEs is much faster than reaching them through Linux system calls. Fast user-level access to VM could help many programs, as Appel and Li argued (Lecture 17).

Costs. Dune can hurt performance: system call overhead is higher because of VT-x entries and exits, faults to the kernel are slower for the same reason, and TLB misses are slower because of the EPT. The authors claim most applications are not much affected, because they do not spend much time in short system calls and the like: Dune is within 5% for most applications of the SPEC2000 benchmark, the exceptions being those that take many TLB misses.

Benefits. Not slowing applications down much is not enough; the question is how much clever use of Dune speeds real applications up. For garbage collection, comparing Dune's dirty-bit collector with the normal one, the overall benefit depends on how fast the program allocates. It is a huge win on three allocation-intensive microbenchmarks, and not a win for an application that allocates little (an XML parser), which the EPT overhead slows down; many real applications allocate more than that.

In short, Dune's key idea is to use VT-x to give processes access to privileged hardware features, and in some situations the result is much faster than using Linux system calls.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 19 (Fall 2020).

Lecture 20: Kernels and HLL

20.1 Why consider a high-level language?

Commodity kernels such as Windows, Linux, and the BSDs are written in C, for good reason: C gives the programmer complete control. It controls memory allocation and freeing, executes almost no implicit, hidden code, allows direct access to memory, and has few dependencies.

But C is hard to use correctly, because memory management is left to the programmer. This causes serious problems for kernel developers: concurrent data structures are challenging (RCU, Lecture 23, is one response), and memory-safety bugs, such as use-after-free bugs (notoriously difficult to debug), buffer overflows (security vulnerabilities), and threads sharing dynamic memory, are common. In 2017, 40 Linux kernel execute-code CVEs (bugs that let an attacker run malicious code in the kernel) were due to memory-safety errors.

A high-level language (HLL) eliminates memory-safety bugs automatically. HLLs offer type safety, automatic memory management with a garbage collector (GC), concurrency, and abstraction; the GC is convenient for the programmer and provides memory safety. The potential downsides are poor performance (bounds, cast, and nil-pointer checks, and garbage collection, which costs CPU cycles, delays execution, and needs extra memory) and incompatibility with kernel programming (no direct memory access, no hand-written assembly, limited concurrency or parallelism). Using an HLL is thus a trade-off between safety and performance. Researchers have built many HLL kernels (Singularity, J-kernel, Taos, Spin, Tock, KaffeOS, House, . . .) to explore new ideas and architectures, and there are many studies of HLL trade-offs for user programs, but kernels differ from user programs (for example, they manage memory more carefully), and no in-depth evaluation had compared an HLL kernel's performance with a fast C kernel.

Measuring the trade-off is tricky, because the comparison must be with a production-grade C kernel such as Linux, and a production-grade HLL kernel cannot be built for the purpose. The most one can do is build an HLL kernel that keeps the important parts the same as Linux, optimize it until its performance is roughly similar to Linux's, and then measure the HLL trade-offs, accepting the risk that measurements of production-grade kernels would differ. The goal is to measure the total effect of using an HLL instead of C on safety, programmability, and performance.

20.2 Biscuit

Biscuit, started in 2014, is a kernel written in Go with a traditional monolithic UNIX organization, to make the comparison with Linux fair (Figure 25). It has about 28,000 lines of Go and 58 POSIX system calls, and runs complex applications such as NGINX and Redis without source modifications.

Its features include multicore support, threads, a journaled file system (7k lines), virtual memory (2k lines), a TCP/IP stack (5k lines), and AHCI and Intel 10 Gb network drivers (3k lines). Go was a good choice: it is easy to call assembly from it, it is compiled to machine code by a good compiler, it has easy concurrency and easy static analysis, and its GC is concurrent mark-and-sweep. Rust might be a fine choice too.

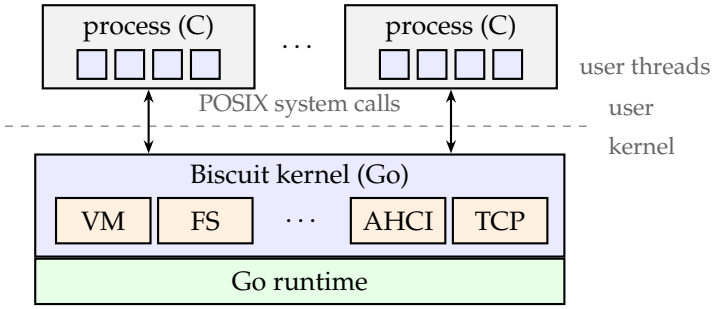


Figure 25: Biscuit: unmodified C processes call a monolithic Go kernel through POSIX system calls; the kernel runs on the Go runtime.

Each process has its own address space, with user and kernel memory isolated by hardware, and each user thread has a companion kernel thread. A system call puts its arguments in registers and executes `SYSENTER`, which passes control to the companion kernel thread; the kernel thread executes the system call and returns with `SYSEXIT`. Biscuit resembles Linux in many ways, with some differences:

- Kernel threads are lightweight goroutines. A context switch between them does not save and restore the page table, so the kernel cannot dereference user pointers directly and must translate them manually.
- The Go compiler freely uses the SSE registers, while Linux avoids them in the kernel and explicitly saves and restores them, so Biscuit saves and restores SSE registers more often.
- Go is not designed to handle interrupts: its runtime does not disable and enable interrupts in critical sections, so calling into a critical section (such as allocation) from an interrupt could deadlock. Biscuit's interrupt handlers therefore do little: they wake up a handling goroutine and return.

Other design puzzles were running the runtime on bare metal and having goroutines run different applications. The biggest difference from Linux, and the hardest puzzle, is handling heap exhaustion.

20.3 Heap exhaustion

Biscuit, Windows, Linux, FreeBSD, and others all face this problem; swap does not solve it, though it may make it less likely. Many kernel operations allocate memory: `open` allocates a file object and `socket` a socket object. The user program decides when to release the resource and free the

memory, so user programs determine how much heap memory the kernel uses, but the machine has limited memory. What if user programs cause the kernel to allocate all of it? This may happen through a buggy program, or a database server intentionally using most of memory plus an unlucky spike in allocations. Then almost no operation can succeed until memory is freed, which is hard or impossible for a user program to handle sensibly: `printf` fails, even `exit` fails, and the memory is held by other processes, so no program can make progress. Without heap memory, nothing works.

To recover, the kernel must free memory by killing the memory-hogging process. It cannot simply kill it, because `exit` must allocate to close files and keep the file system consistent, so the kernel must reserve enough memory for `exit`; Linux and Biscuit both do.

How the kernel copes is important:

- *Strawman 0: panic*, as xv6 does.
- *Strawman 1: wait for memory in the allocator.* Blocking in the allocator is tempting, since the caller then need not handle failure, but it can deadlock. Suppose a well-behaved program holds the lock on a directory, and the memory hog is waiting for that lock. The kernel cannot kill the hog while it waits, the hog waits for the good program, and the good program, blocked in the allocator, waits for the hog to exit.
- *Strawman 2: check and handle allocation failure*, as C kernels do. Linux avoids the deadlock above by failing the good program's allocation instead, so the kernel must handle failure of nearly every allocation, which is hard to get right and full of bugs (hence Linux's "too small to fail" rule). In Go it is not even possible, since Go allocates implicitly and does not expose failed allocations.

Biscuit instead reserves memory: before executing a system call, it waits until enough memory is available for the whole call.

```
reserve()      // no locks held; may evict, kill, wait
sys_read()
  ...
unreserve()
```

There is no waiting in the middle of an operation, and no locks are held while waiting, so there is no deadlock, no checks, and almost no code to handle allocation failures.

The reservation must bound the maximum memory each system call can use; smaller bounds allow more concurrent system calls. Static analysis of Go turns out to be easy, and the authors wrote a tool, MAXLIVE, using Go's static analysis packages. At a high level it is a fancy escape analysis. It inspects the call graph and finds all the allocations of each system call. Objects are of two kinds: those that may be written to a global, which are always live, and those only ever referenced from the stack, which are freed when some stack frame is destroyed. The bound is the sum of the first kind plus, at each leaf of the call graph, the maximum of the second kind. The bound is not exact but

conservative: it may over-estimate the needed memory but never under-estimates it. Annotations handle difficult cases, and applying the tool took about three days of expert effort.

20.4 Experience with Go

Biscuit has 90 uses of Go's `unsafe`. The authors hacked the runtime in a couple of ways (to schedule interrupt goroutines and to count allocations), and some code is non-portable because Go's memory model is unclear. Building Biscuit was otherwise similar to building other kernels. Biscuit does use HLL features: counted per thousand lines, its uses of allocation, maps, slices, channels, strings, multi-valued return, closures, finalizers, `defer`, `go` statements, interfaces, type assertions, and imports are comparable to those of two huge Go projects (Moby and Golang, over a million lines). Slices replace pointer-plus-size, `defer` replaces `goto`s, and closures, maps, strings, multi-valued return, and GC'ed allocation give simpler code overall.

GC significantly simplifies concurrent data structures, whose entries are heap allocated: the hard question of when to free an entry, once all other threads are done with it, disappears. For example, in

```
buf := new(object_t)
// Initialize buf...
go func() {
    process1(buf)
}()
process2(buf)
// When should C code free(buf)?
```

the GC frees `buf` once both goroutines are done, while C code would have to work out when to free it. Locks and reference counts are expensive on hot paths, so read-lock-free structures are good for performance, and there the challenge in C is knowing when an object is free:

```
var Head *Node

func get() *Node {
    return atomic_load(&Head)
}

func pop() {
    Lock()
    v := Head
    if v != nil {
        atomic_store(&Head, v.next)
    }
    Unlock()
}
```

Linux frees such objects safely with RCU, which defers the free until all CPUs have context-switched. The programmer must follow RCU's rules (a prologue and epilogue around accesses, no sleeping or scheduling in between), which is error prone in more complex situations; with GC these challenges disappear, so the HLL significantly simplifies read-lock-free code.

To compete with Linux, Biscuit implements many of Linux's performance optimizations: large pages for kernel text, per-CPU NIC transmit queues, an RCU-like directory cache, executing file system operations concurrently with commit, and padding structures to remove false sharing. Go did not prevent any of them; good OS performance is more about optimizations than about the language.

Would an HLL prevent kernel exploits? Inspecting the fixes for all publicly available execute-code CVEs in the Linux kernel for 2017 and classifying the outcome each bug would have in Biscuit:

Category	Number	Outcome in Go
unknown	11	unknown
logic	14	same
use-after-free or double free	8	disappear due to GC
out-of-bounds	32	`panic` or disappear

A `panic` is likely better than executing malicious code, so an HLL would prevent kernel exploits.

20.5 Performance

The evaluation uses a 4-core 2.8 GHz Xeon X3460 with 16 GB of RAM and hyperthreading disabled, and three demanding applications: the NGINX web server, the Redis key/value store, and CMailbench, a mail-server benchmark exercising `fork`, `exec`, and VM. Together they exercise a 10 Gb NIC, the TCP stack, VM, and the (in-memory) file system. Each runs for a minute with no idle CPU time and spends 79–92% of its CPU time in the kernel; Biscuit gets a 512 MB kernel heap.

Is Biscuit in the same league as Linux? Linux 4.9.82 (Debian 9.4) ran with expensive features disabled (page-table isolation, retpolines, kernel address space layout randomization, transparent huge pages, and others), which speeds Linux up and makes the comparison fairer:

	Biscuit (ops/s)	Linux (ops/s)	Linux/Biscuit
CMailbench (in memory)	15,862	17,034	1.07
NGINX	88,592	94,492	1.07
Redis	711,792	775,317	1.09

Biscuit is within 9% of Linux on these applications. It is not an apples-to-apples comparison: Linux has more features, and it is much faster in other situations, with more than four CPUs or when NUMA awareness matters.

What is the HLL tax? Classifying samples of CPU-time profiles into HLL cost buckets:

	GC cycles	GCs	Prologue cycles	Write-barrier cycles	Safety cycles
CMailbench	3%	42	6%	< 1%	3%
NGINX	2%	32	6%	< 1%	2%
Redis	1%	30	4%	< 1%	2%

Function prologues are the most expensive part. GC costs under 3%: the benchmarks allocate kernel heap rapidly but have few long-lived kernel heap objects.

How much might GC cost? GC cost grows with the amount of live data (the GC must mark and read the pointers in every object) and falls with the amount of free heap memory (a collection runs each time free memory is exhausted), so the total GC cost is proportional to the ratio of live data to free heap RAM. The applications use up to 5 GB of memory and make the kernel allocate rapidly, but the kernel heap holds few objects: small metadata objects, since a separate allocator manages pages (user memory, file content, socket buffers, page-table pages). This reduces the number of kernel objects and increases free heap memory. With two million vnodes of live data, CMailbench gives:

Live (MB)	Free (MB)	Ratio	Throughput	GC
640	320	2	10,448	34%
640	640	1	12,848	19%
640	1280	0.5	14,430	9%

Keeping GC under 10% needs 3× the live data in heap RAM. In general, a large heap trades memory for lower CPU overhead. Few programs allocate millions of resources, though: MIT's big time-sharing machines run 80 users and 800 tasks with 9–16 GB resident and under 2 GB of kernel heap (cached files being an exception, perhaps evictable), so the memory cost seems acceptable in common situations.

GC pauses. The GC must eventually run and could delay latency-sensitive work. Some collectors cause one large pause, but Go's is interleaved with execution (as in Baker's algorithm, Lecture 17), so each allocation during a collection must do some GC work, causing many small delays. Measuring every pause during NGINX and summing them per request, the maximum single pause is 115 μs (marking a large part of the TCP connection table), the maximum total pause during one request is 582 μs, and fewer than 0.3% of requests are paused for more than 100 μs; tuning the GC reduced these pause times. Some programs cannot tolerate rare 582 μs pauses, but many probably can; the 99th-percentile latency in a service in Google's "Tail at Scale" was 10 ms.

Go versus C. To isolate the cost of the language from differences in OS features, the authors made Linux's and Biscuit's code paths nearly identical for two core paths small enough to give the same functionality: pipe ping-pong (system calls and context switching) and the page-fault handler (exceptions and VM). For pipe ping-pong (1.2k lines of Go, 1.8k of C, no allocation and so no GC) CPU-time profiles show both kernels doing the same work, with matching top-ten most expensive

instructions. C is 15% faster (536,193 versus 465,811 ops/s), because prologues and safety checks give Go 16% more instructions; the Go compiler inserts safety checks and write barriers. On the page-fault path C is 5% faster, since kernel entry and exit and copying dominate the other work. Go is slower, but competitive.

Scalability. Can Biscuit use many cores efficiently, or is Go a scalability bottleneck? Running CMailbench on 1 to 20 cores, Biscuit scales well up to 10 cores; it falls well short of perfect scaling at 20 cores because of lock contention in CMailbench and because Biscuit is not NUMA-aware.

20.6 Conclusions

The HLL worked well for kernel development and its performance was pretty good, but C is faster. If performance is paramount, use C (up to 15% faster); to minimize memory use, use C (a smaller memory budget raises GC cost); if safety is paramount, use an HLL (40 CVEs stopped); if performance is merely important, use an HLL and pay about 15% plus memory. Other HLLs, such as Python, would likely perform very differently, and multithreading in Python is harder than in Go. Using an HLL where performance matters less and C where it matters more could work, but sharing GC heap objects with C code is difficult. Biscuit reaches physical addresses without pointer arithmetic through `unsafe` pointer conversions.

Source. Frans Kaashoek, *6.S081: Operating System Engineering*, Lecture 20 (Fall 2020).

Lecture 21: Networking

21.1 Packet formats and protocols

Applications on a host send packets through the host's network interface card (NIC) onto a local area network (LAN), where another host's NIC receives them; to reach distant hosts, packets cross a series of LANs connected by routers.

An ethernet packet (xv6's `struct eth` in `kernel/net.h`) consists of a start flag, a 48-bit destination ethernet address, a 48-bit source ethernet address, a 16-bit ether type, the payload, and an end flag. A simple ethernet LAN broadcasts: every host's NIC sees every packet, and a host uses the destination address to decide whether a packet is really for it. Today's ethernet LAN is a switch with a cable to each host, and the switch uses the destination address to decide which host to send a packet to. Ethernet addresses are assigned by the NIC manufacturer: a 24-bit manufacturer ID followed by a 24-bit serial number.

ARP (`struct arp`) is a request/response protocol that translates an IP address into an ethernet address. It is nested inside an ethernet packet with ether type `0x0806`, and its header says whether it is a request or a response. A request carries the desired IP address and is broadcast to every host on the switch; all hosts process it, but only the owner of that IP address responds, with the corresponding ethernet address. The habit of *nesting* higher-level packets inside lower-level ones (ARP inside ethernet) means a packet usually begins with a sequence of headers: one layer's payload is the next layer's header (Figure 26).

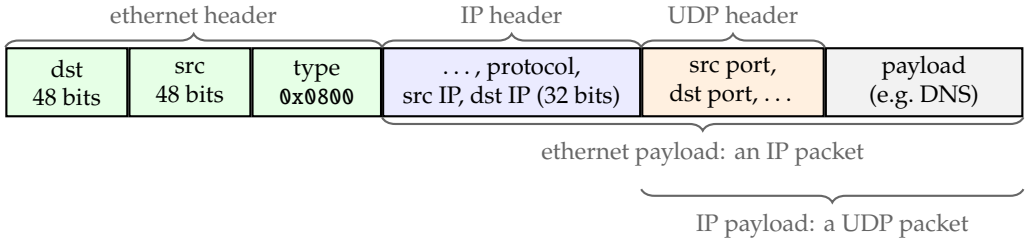


Figure 26: Nested headers in a UDP packet carried over ethernet.

The ethernet header suffices to deliver a packet to a host on the local network, but routing it to a distant Internet host needs more. The IP header (`struct ip`, ether type `0x0800`) contains a lot, but the addresses are most critical: a 32-bit IP address is enough to route to any computer on the Internet, and its high bits hold a *network number* that helps routers decide how to forward the packet. The header's protocol number tells the destination which higher-level protocol to hand the packet to, usually UDP or TCP.

Once a packet reaches the right host, which application should get it? The UDP header (`struct udp`) sits inside the IP packet and contains source and destination PORT numbers. An application uses the socket API system calls to tell the kernel it wants to receive all packets sent to a particular port. Some ports are well known (port 53 is reserved for DNS servers); others are allocated as needed for the client ends of connections. The UDP header is followed by the payload, such as a DNS request or response. TCP is like UDP but adds sequence numbers, so that it can retransmit lost packets and match its sending rate to the capacity of the network and the destination.

21.2 The kernel network stack

A typical kernel network stack is layered: applications (a web browser, a DNS server) on top; the socket layer with its table mapping ports to file descriptors; UDP and TCP; IP, with its routing table, alongside ARP and its table; and the NIC drivers. Packet buffers come from an allocator (`struct mbuf` in `net.h`). On the way in, each layer parses, validates, and strips its header, discarding the packet if there is a problem; on the way out, each layer prepends its header. The software's layer structure partly follows the nesting of headers.

Viewed as control flow, the stack consists of several independent actors, each of which takes input packets, processes them, and produces output (Figure 27). In a typical arrangement (there is much variation, and the lab's is simpler) the NICs buffer packets internally or by DMA; a receive interrupt handler copies packets from the NIC to a software input queue; a transmit interrupt handler copies packets from a software output queue to the NIC; a "network thread" reads packets from the input queue and decides what to do with each (answer ARP, forward it, or put it on a socket queue); and applications read from their per-socket queues.

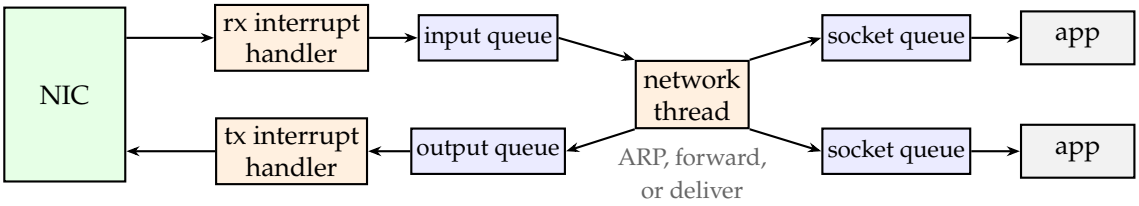


Figure 27: Control flow in a typical kernel network stack: interrupt handlers, a network thread, and applications connected by queues of packet buffers.

The queues absorb temporary bursts of input, keep the output NIC busy while the CPU computes, and let the NICs, the network thread, and the applications run as independent flows of control. Other arrangements are possible and sometimes much better: a user-level stack, direct user access to the NIC (as in Intel's DPDK), or polling.

NICs buffer packets in one of two ways. Some, like the one in the receive-livelock paper below, queue packets in their own memory, and the driver software must copy them to RAM. The Intel e1000 used in the lab DMAs packets into host RAM instead: software prepares a "ring" of buffer pointers for receiving and another for transmitting, and the NIC DMAs each packet to the memory

that successive ring elements point to. DMA is faster than a software copy loop, and it proceeds concurrently with computation.

21.3 Receive livelock

Mogul and Ramakrishnan's "Eliminating Receive Livelock in an Interrupt-Driven Kernel" is a famous and influential paper that illustrates trade-offs in how a kernel network stack is structured; livelock and congestion collapse arise in many situations.

The paper's original system, an interrupt-driven kernel forwarding packets (without the authors' fixes), shows the problem. As the input rate increases, the forwarding rate at first rises with it, since every packet is forwarded. It peaks at a rate set by the CPU time needed per packet: a peak of 5000 packets per second means 200 μs per packet. Beyond the peak, the output rate goes *down*. Receive interrupts take priority over everything else, so as input increases the CPU spends more and more of its time receiving packets and less forwarding them; the packets that are not forwarded are discarded when the queues fill, after CPU time has already been spent on them. How fast the output falls depends on how much processing each discarded packet receives before it is dropped. This is RECEIVE LIVELOCK: the CPU is busy, but little useful work gets done.

The problem needs a source of interrupts that can arrive faster than the CPU can process them, with no feedback. A disk interrupts only for requests the kernel issued, and a host receiving TCP traffic is protected by TCP's matching of the sender's rate to the destination's capacity.

Why not process each packet completely, forwarding it, inside the receive interrupt handler, as the network lab does? An output queue is still needed, the receive work would starve transmit interrupts and other devices' receive interrupts, and there is no story for processing that involves user-level programs. Why not always poll and never use interrupts? At low load polling wastes CPU time checking an idle NIC.

The overall goal is that once the CPU has started to spend time on a packet, it finishes that packet, rather than partially processing it and then discarding it because of overload. For the special case of forwarding, this means giving output priority over input. Interrupts allow no such control.

21.4 The polling solution

The paper eliminates the IP input queue. A NIC receive interrupt just wakes up a thread and leaves interrupts *disabled* for that NIC. The thread does all the processing, re-checks the NIC for more input, and re-enables interrupts only if no input is waiting:

```
NIC interrupt:
  wake up the network thread (but read no packets)
  disable NIC interrupts

network thread loop:
  if packets are waiting in the NIC:
    read a few packets from the NIC        // the polling part
    process each packet completely
  else:
    enable NIC interrupts
    sleep
```

When packets arrive too fast, the thread keeps polling with interrupts disabled, and the excess packets are dropped by the NIC before the CPU has spent anything on them, so every packet the CPU starts on is finished and there is no livelock. When packets arrive slowly, interrupts stay enabled and the thread sleeps between packets. Modern Linux uses a scheme, NAPI, inspired by this paper.

With the authors' system, the forwarding rate rises to its maximum and then levels off as the input rate keeps increasing, because the excess packets are dropped at the NIC at no cost. A polling version *without a quota* on the number of packets processed per round behaves badly: input processing still starves the processing of transmit completions, and the output rate falls immediately to zero rather than gradually. Doing more processing before discarding a packet makes livelock worse: each excess received packet now costs the CPU time of many transmitted packets.

When every packet must also pass through a user-level program (`screend`), polling *without feedback* still behaves badly: there is a queue in front of `screend`, and the input thread can still take 100% of the CPU, leaving 0% for `screend`. Polling *with feedback* behaves well: the input thread sleeps when the queue to `screend` is nearly full and wakes up when it is nearly empty.

The big picture is that the polling loop is a place to exert scheduling control. The paper uses two different mechanisms, a polling thread with quotas and feedback from a full queue. Perhaps feedback could have been used for both, since it needs no magic numbers, but it is hard to retrofit into the existing UNIX structure. In a chain of stages with queues between them, feedback must work its way back from a slow late stage to earlier ones; a structure that splits into parallel paths is not so great, since one slow path blocks all paths.

Two general principles emerge: do not spend time on new work before completing existing work, and design so that efficiency increases with load rather than decreasing, as the switch from interrupts to polling under high load does. Similar phenomena arise with timeouts and retransmissions as network connections multiply, and with spinlocks as cores multiply. Complex, multi-stage systems may need careful scheduling of resources to survive loads close to capacity.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 21 (Fall 2020).

Lecture 22: Meltdown

Security is a critical operating system goal, and the kernel's main strategy is isolation: user and supervisor modes, page tables, defensive system calls, and so on. Meltdown is a surprising and disturbing attack on user/kernel isolation. It is one of a number of recent micro-architectural attacks, which exploit hidden implementation details of CPUs; it is fixable, but people fear an open-ended supply of related surprises.

22.1 The core of the attack

The attack runs this code in user space:

```
char buf[8192]
r1 = <a kernel virtual address>
r2 = *r1
r2 = r2 & 1
r2 = r2 * 4096
r3 = buf[r2]
```

Will `r2` end up holding data from the kernel? The attack assumes the kernel is mapped in the user page table with `PTE_U` clear, which was nearly universal until these attacks were discovered: the kernel occupies the upper half of the address space (high bit set) and the user the lower half (starting at zero), and mapping both makes system calls significantly faster. So `*r1` is meaningful, even though forbidden. How this code can be useful to an attacker depends on two mostly hidden CPU implementation details: speculative execution and caches.

22.2 Speculative execution

Consider ordinary code (C-like; `r0` and so on are registers and `*r0` is a memory reference):

```
r0 = <something>
r1 = valid        // r1 is a register; valid is in RAM
if(r1 == 1){
  r2 = *r0
  r3 = r2 + 1
} else {
  r3 = 0
}
```

Loading `valid` may take hundreds of cycles if it comes from RAM, and the branch needs that

value. Rather than pause until the load completes, the CPU predicts which way the branch will go and continues executing; this is SPECULATION. So the CPU may execute `r2 = *r0` and then `r3 = r2 + 1`. If it mispredicted (say `r1 == 0`), it flushes the results of the incorrect speculation, reverting `r2` and `r3`, and restarts execution in the `else` branch. Speculative execution is a huge win for performance, because it lets the CPU overlap long instructions (divides, memory loads) with subsequent instructions.

What if the CPU speculatively executes the first branch and `r0` holds an illegal pointer? If `valid == 1`, `r2 = *r0` should raise an exception; if `valid == 0`, it should not, even though it was executed speculatively. The CPU RETIRES instructions only once it is sure they will not be canceled by mis-speculation, and it retires them in order, only after all previous instructions have retired, since only then does it know that no previous instruction faulted. Thus the fault of a speculatively executed instruction may not occur until a while after the instruction finishes.

Speculation is, in principle, invisible: part of the CPU's implementation but not of its specification, meant to increase performance without changing the results programs compute. ARCHITECTURAL features are those in the CPU manual, visible to programmers; MICRO-ARCHITECTURAL features are not in the manual and are intended to be invisible.

22.3 Caches

CPU caches are another micro-architectural feature (Figure 28). A load that misses fetches the data and puts it in the cache. The L1 cache is virtually indexed for speed, so after a system call returns to user space, kernel data remains in the L1 cache (assuming the page table maps both user and kernel); each L1 entry probably holds a copy of the PTE permission bits. On an L1 miss the CPU looks up the TLB and then the physically addressed L2 cache. An L1 hit takes a few cycles, an L2 hit a dozen or two, and RAM about 300 cycles, where a cycle is 1/clock rate, for example 0.5 ns.

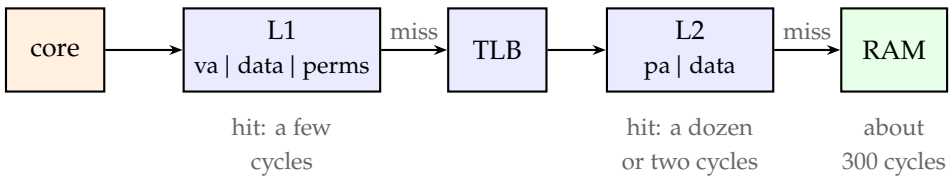


Figure 28: The cache hierarchy relevant to Meltdown, with typical access times.

Having both kernel and user data in the cache is safe in the architectural sense: a user program cannot read kernel data directly out of the cache, because the permission check still applies. In real life, however, micro-architecture is not invisible. It affects how long instructions and programs take, and is of intense interest to people who write performance-critical code and to compiler writers; Intel and others publish optimization guides, usually vague about details.

A useful trick senses whether something is cached. To learn whether a function `f()` uses the

memory at address X, FLUSH+RELOAD proceeds as follows:

1. ensure the memory at X is not cached (Intel CPUs have a `clflush` instruction; alternatively, load enough junk to force everything else out of the cache);
2. call `f()`;
3. record the time, by reading a cycle counter (`rdtsc` on Intel CPUs);
4. load a byte from X (memory fences are needed to ensure the load really happens);
5. record the time again;
6. if the difference is small (say under 50 cycles), the load hit, so `f()` must have used the memory at X; otherwise it did not.

22.4 Meltdown in detail

The complete attack:

```
char buf[8192]

// the Flush of Flush+Reload
clflush buf[0]
clflush buf[4096]

<some expensive instruction like divide>

r1 = <a kernel virtual address>
r2 = *r1
r2 = r2 & 1      // speculated
r2 = r2 * 4096   // speculated
r3 = buf[r2]     // speculated

<handle the page fault from "r2 = *r1">

// the Reload of Flush+Reload
a = rdtsc
r0 = buf[0]
b = rdtsc
r1 = buf[4096]
c = rdtsc
if b-a < c-b:
  low bit was probably a 1
```

The attacker deduces the low bit of the kernel data from which of two cache lines was loaded, `buf[0]` or `buf[4096]`. Three points make this work:

- The fault from `r2 = *r1` is delayed until the load retires, which may take a while (behind the expensive instruction), giving the following instructions time to execute speculatively.
- Apparently `r2 = *r1` does perform the load, even though it is illegal, and puts the result in `r2`, though only temporarily, since the fault at retirement reverts it.
- `r3 = buf[r2]` loads part of `buf` into the cache, even though its change to `r3` is canceled by the mis-speculation, because Intel regards cache contents as hidden micro-architecture.

The attack often does not work: the paper's listings show frequent failures, and it reports only about 10 bytes per second when the kernel data is not cached. Retries help, and thousands may be needed. The conditions for success are not clear; perhaps the attack is reliable if the kernel data is in the L1 cache and not otherwise.

To use Meltdown in a real-world attack, the attacker must run code on the victim machine. On a timesharing system the kernel may hold other users' secrets, such as passwords and keys, in I/O or network buffers, or the kernel may map all of physical memory. In the cloud, some container and VMM systems might be vulnerable, letting one customer steal another's data. A browser runs untrusted code, such as plug-ins, in sandboxes; perhaps a plug-in could steal a password from the kernel. However, Meltdown is not known to have been used in any actual attack.

22.5 Defenses

The software fix is not to map the kernel in user page tables; the paper calls this KAISER, and Linux now calls it KERNEL PAGE-TABLE ISOLATION (KPTI). It requires a page table switch on every system call entry and exit, which is how RISC-V xv6 already works. The switch can be slow because it may require TLB flushes; process-context identifiers (PCIDs) avoid the flush, though with some remaining expense. Most kernels adopted KAISER/KPTI soon after Meltdown became known.

The hardware fix is to return only permitted data from speculative loads: if `PTE_U` is clear, return zero rather than the actual data. This probably costs little or nothing, since each L1 cache line apparently contains a copy of the PTE's `PTE_U` bit. AMD CPUs apparently worked this way all along, and the latest Intel CPUs seem to as well (Intel calls this `RDCL_NO`).

These defenses are deployed and believed to work. Still, it was disturbing that page protections turned out not to be solid, and more micro-architectural surprises keep emerging. Whether the underlying issue is a set of fixable bugs or an error in strategy is still playing out.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 22 (Fall 2020).

LECTURE 23: RCU

23.1 Read-heavy shared data

Parallel execution is crucial to performance on a multi-core machine, and kernels have lots of natural parallelism: different processes' system calls (`fork`, reads and writes of different cached files, pipes, sockets) are often independent at a high level. But the kernel's shared resources (the memory allocator, scheduler, disk cache, inode cache, and so on) often suppress that parallelism; the usual symptom is CPU time wasted spinning in `acquire()`. Early kernels protected all kernel data with a single big lock; later kernels moved to fine-grained locking, and many now also use lock-free approaches. Production kernels put huge effort into eliminating parallel bottlenecks, with many design patterns tailored to different situations.

One important situation is data that is read much more often than it is modified: network routing and ARP tables, file descriptor arrays, and most kinds of system call state. Take a singly linked list, such as a list of processes, VMAs, or cached blocks: a global variable points to the first element, and each element holds some data (say a short string) and a `next` pointer. Most uses are reads that scan the list; occasional writers add an element, delete one, or change the string in one.

23.2 Read/write locks

xv6 would make list readers and writers acquire a spinlock, which is safe but gives no parallelism even when every thread only reads. A READ/WRITE LOCK seems the perfect solution. Its interface is `r_lock(l)`/`r_unlock(l)` and `w_lock(l)`/`w_unlock(l)`, and its semantics are that the lock is held either by one writer and no readers or other writers, or by any number of readers and no writers. A simplified version of Linux's:

```
struct rwlock { int n; };  // 0: not locked; -1: one writer; k>0: k readers
r_lock(l):
  while 1:
    x = l->n
    if x < 0
      continue
    if CAS(&l->n, x, x + 1)
      return
w_lock(l):
  while 1:
    if CAS(&l->n, 0, -1)
      return
```

Here `CAS(p, a, b)` is an atomic compare-and-swap instruction: if `*p == a` it sets `*p = b` and returns true, and otherwise returns false.

Surprisingly, `r_lock()` is very slow when many cores call it often. Suppose N readers and no writers. All N calls read and cache `l->n`. Only one of the N CAS operations succeeds and increments `l->n`, and that CAS does $O(N)$ work, since it must invalidate the $N - 1$ other cached copies of `l->n`. The other cores all retry the read and the CAS, so it takes N loop iterations, each costing $O(N)$, for all N cores to finish `r_lock()`: $O(N^2)$ work in total. This is disappointing, because the list read itself probably takes a few dozen cycles if cached, even with many cores reading, while a popular `r_lock()` can take hundreds or thousands of cycles. `r_unlock()` writes `l->n` too, and while a writer holds the lock, readers must spin. The underlying problem is that `r_lock()` does expensive *writes* to shared data.

23.3 Reading without locks

Can reads of shared read-write data be purely read-only, avoiding even the writes needed to lock, while writers still lock? If readers simply read the list without locking, nothing goes wrong as long as there is no writer. A concurrent writer causes trouble, though: a reader may see a string that is only partly modified, may follow a pointer to an inserted element whose contents are not yet initialized, or may be looking at an element at the moment it is deleted and freed. So readers cannot simply skip locking, but these specific problems can be fixed, and fixing them is what READ-COPY UPDATE (RCU) does. RCU makes reads fast (no locks and no writes) at the cost of slower writes (writers lock and do more work). It helps many, though not all, situations with read-heavy shared data, and is used extensively in Linux, where it is a big part of multi-core performance (with over 10,000 uses of the RCU API).

Idea 1: do not modify data in place; prepare a new copy. With the list Head → E1 → E2 → E3 → nil, a writer that wants to change the string `E2->x`

1. locks,
2. allocates a new element `e`,
3. sets `e->next = E2->next`,
4. sets `e->x` to the new string,
5. sets `E1->next = e`,
6. unlocks.

A reader traversing the list on another core reads `E1->next` either before or after step 5 (this is a simplification; see idea 2). If before, it sees the unmodified E2 and the original string; if after, it sees the new element and the new string. Either way its `next` pointer leads to E3, and it never sees a partially updated string (Figure 29). The update of `E1->next` is a COMMITTING WRITE: before it, it is

as if nothing changed; after it, all the changes are visible. RCU is best suited to data structures, such as lists and trees, in which a single pointer write commits an update.

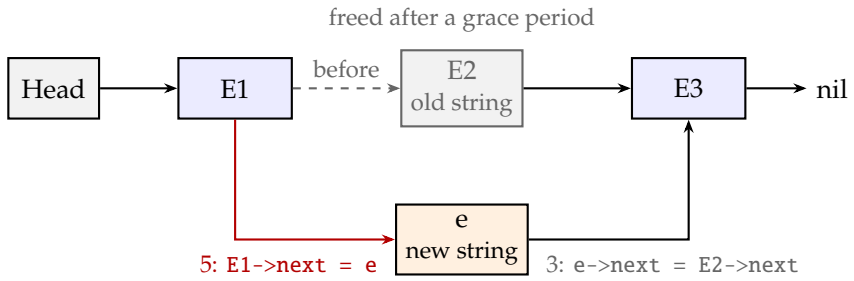


Figure 29: RCU replaces E2 by a prepared copy `e`; the single committing write to `E1->next` makes the change visible to readers, who see either the old or the new element.

Idea 2: readers and writers use memory barriers to enforce order. Neither the compiler nor the machine may move step 3 or 4 after step 5, so the writer needs a barrier just before step 5. The reader needs a barrier between reading `Rx = E1->next` and dereferencing `Rx` to read the element's contents (some CPUs, such as the DEC Alpha, can actually reorder these).

Idea 3: defer freeing. When a writer removes an element, a reader might still be looking at it, so the writer must give readers time to finish after it removes the visible reference. Garbage collection or reference counts could tell when no reader remains, but they are expensive. Instead, RCU imposes two rules:

1. a reader may not hold a pointer to RCU data across a context switch, much as a spinlock may not be held across one;
2. a writer defers the free until every CPU has context-switched.

This waiting period is a GRACE PERIOD. The writer may have to wait a few milliseconds, but writes are assumed to be rare. A simple implementation has the writer force itself to be scheduled on each CPU in turn. Read critical sections must disable preemption: otherwise a preempted task could still hold a pointer to an RCU object, and waiting for every core to context-switch would no longer guarantee that all readers are done.

23.4 The Linux RCU interface

The interface for readers is `rcu_read_lock()`, `rcu_read_unlock()`, and `rcu_dereference(p)`. The interface for writers is `synchronize_rcu()`, `call_rcu(fn, x)`, and `rcu_assign_pointer(pa, p)`.

- `rcu_read_lock()` and `rcu_read_unlock()` do almost nothing: they disable and re-enable timer interrupts, preventing preemption, since a context switch implies that the core is done with its RCU objects. This is very cheap.

- `synchronize_rcu()` returns only after every core has gone through at least one context switch; it can take a long time and can yield the CPU.
- `call_rcu(fn, x)` returns immediately after adding $\langle$`fn`, `x`$\rangle$ to a list of callbacks; `fn(x)` is called after every core has context-switched. It does not delay the writer, but it can be dangerous if called a lot: the list might grow very long, and the unfreed memory it refers to might be large enough to make the kernel run out of memory.
- `rcu_assign_pointer()` and `rcu_dereference()` are C macros that insert memory barriers. `__sync_synchronize()` tells the compiler not to move memory references past it and emits whatever instructions keep the machine from reordering loads and stores across it (a `fence` on RISC-V):

```
// rcu_assign_pointer(a, p) expands to something like
__sync_synchronize();
*a = p;

// rcu_dereference(p) behaves like
temp = p;
__sync_synchronize();
return temp;
```

A list reader:

```
rcu_read_lock();
e = head;
while(e){
  e = rcu_dereference(e);
  // look at e->x ...
  e = e->next;
}
rcu_read_unlock();
```

A writer that replaces the first element:

```
acquire(lock);
old = head;
e = alloc();
e->x = ...;
e->next = head->next;
rcu_assign_pointer(&head, e);
release(lock);
synchronize_rcu();   // wait for readers of old
free(old);
```

The reader may use `e` only between `rcu_read_lock()` and `rcu_read_unlock()`; it cannot, for

example, return `e`. RCU does nothing to stop two writers from interfering with each other, so writers make their own arrangements, typically a spinlock. Mostly, RCU is a set of rules that writers follow so that the data structure can always be read safely, even in the middle of a change; the most useful code RCU itself provides is `synchronize_rcu()`.

23.5 Performance and examples

For readers, RCU imposes nearly zero cost, while read/write locks can take hundreds or thousands of cycles; moreover, RCU readers can proceed while a write is in progress. Writers, on the other hand, can take much longer with RCU, because of `synchronize_rcu()`. RCU therefore makes sense when writes are rare or not time-sensitive: it improves parallel performance when the benefit of letting many readers avoid locking outweighs the cost of occasional writers that wait a long time.

Removing an NMI handler. Non-maskable interrupts (NMIs) cannot be disabled; they are used for CPU profiling, critical hardware errors, and watchdog timers that catch a hung kernel. Kernel code can register handlers to be called during an NMI, and can unregister a handler and free the memory holding its code. NMIs are frequent and registration is rare, so reads far outnumber writes. If some core is executing the handler when it is unregistered, `unregister_nmi_handler()` (1) deletes the handler's list entry, so future interrupts will not see it; (2) calls `synchronize_rcu()` to wait for all current interrupts to complete, since a core does not context-switch until it has left the interrupt; and (3) frees the memory holding the handler code. RCU here defers a free until all uses have finished. Because NMIs cannot be disabled, spinlocks cannot keep them out, so NMI code generally cannot be allowed to acquire locks at all (it could deadlock). RCU is thus not merely faster than locking for the reader; it works where locking would not.

IP options. An IP packet header can contain options (for example, record route), and a kernel socket has optional options that are copied into every outgoing packet. The send code (`udp_sendmsg()`) copies the socket's options without a lock, since options are read much more often than written. `setsockopt()` defers the free of superseded options with `call_rcu(kfree, old)`. The reader is safe because the writer entirely prepares the new options before publishing the pointer, writer and reader use memory barriers, and the writer defers the free until any reader of the old options must have finished. The RCU paper calls this "reference counting," but no references are counted; "garbage collection" might be a better term, since the goal is to free the object after its last reference disappears. Objects with long-term references, such as xv6's `struct file`, still need explicit reference counts.

23.6 Limitations

RCU is not universally applicable:

- it does not help writes, and improves performance only when reads far outnumber writes;

- it does not work for code that must hold references across a yield or sleep (much like spinlocks);
- it does not work if the data structure cannot be updated with a single committing write, for example if readers scan a doubly linked list in both directions, or if updates must be made in place;
- readers can see stale data. A reader may see the old version just after a writer has replaced it, and may see a change if it looks twice (`udp_sendmsg()` might send a packet with the old options, but only if it runs concurrently with `setsockopt()`). That is not a problem here, and the window is short (nanoseconds or microseconds), but it may be in a few cases; programmers must convince themselves that this lack of freshness and atomicity is acceptable. Since RCU readers never write the protected data, lost updates (a stale read followed by a write) cannot occur;
- it adds complexity to writers (Linux keeps a review checklist for RCU patches) and may well be more bug-prone than locking, a price Linux developers often accept for multi-core performance;
- it needs frequent context switches, and needs to know about them: easy in a kernel, harder for user-space threads, and `synchronize_rcu()`, which can block, cannot be called at interrupt time.

xv6 does not use RCU: spinlocks are powerful and general, and it is hard to judge the performance effect of optimizations like RCU under qemu.

23.7 An efficient `synchronize_rcu()`

Scheduling the writer on each CPU is not the only way. In a more efficient design, each core keeps a count of its own context switches and a list of waiting `synchronize_rcu()` and `call_rcu()` requests. Periodically, each core processes a whole batch of RCU writers: it saves its list and empties it, and records all other cores' counts. At every context switch it looks at the other cores' counts again; once all have changed, every core has context-switched, so it wakes or runs everything on the saved list.

23.8 Write-heavy shared data

For shared data that is written often, the best approach is to redesign for no sharing; the second best is semi-private per-core partitions, as in the locking lab's per-CPU `kalloc` free lists and Linux's per-core scheduling lists. Statistics counters are an example: each core keeps its own count under a separate spinlock. To write, a core acquires its local lock and increments its local count; a spinlock is cheap when only one core uses it. To read, acquire *all* the locks and sum all the counts; that is expensive, but writes are assumed to dominate.

Reference counts are a related case. A standard reference count (`kref_get()` with an atomic increment, `kref_put()` with an atomic decrement-and-test that releases the object at zero) requires exclusive ownership of the cache line for every get and put, so a frequently referenced object causes heavy cache-line bouncing and often becomes a scaling bottleneck once other bottlenecks are removed. Per-CPU reference counts keep an array of counters, one per core: `percpu_ref_get()` and `percpu_ref_put()` update only the local core's entry, which only that core writes, so there is no bouncing. `percpu_ref_kill()` reverts to ordinary shared counting; the scheme pays off when most references are added and removed while the killer's own reference is still held, which is often true. Kill works as follows:

1. set the shared count to a bias value;
2. atomically set a "kill" flag in the shared count;
3. wait a quiescent period, after which the flag is visible to all reference counters (a call to `synchronize_rcu()` is perfect for this);
4. sum the per-core counts and atomically add the total to the shared count;
5. atomically subtract the bias value;
6. free the object when the shared count reaches zero.

Per-CPU counting thus trades space for speed, and makes reference counting almost free when objects are long-lived and one task can be designated as the killer.

When can code be made scalable at all? The scalable commutativity rule says that if two operations commute, there exists a scalable (conflict-free) implementation of them: if the order of the operations does not matter, communication between them must be unnecessary. Scalable implementations may still exist for operations that fail the rule, but if they pass it, scalability is definitely possible. The key to scalability is therefore good interface design. For example, POSIX requires `open()` to return the smallest available file descriptor, so two `open()` calls do not commute; an interface that allowed any free descriptor would let them commute. The operations of both RCU and per-CPU reference counting commute.

Source. Robert Morris, *6.S081: Operating System Engineering*, Lecture 23 (Fall 2020).